GO: ZERO TO MASTERY

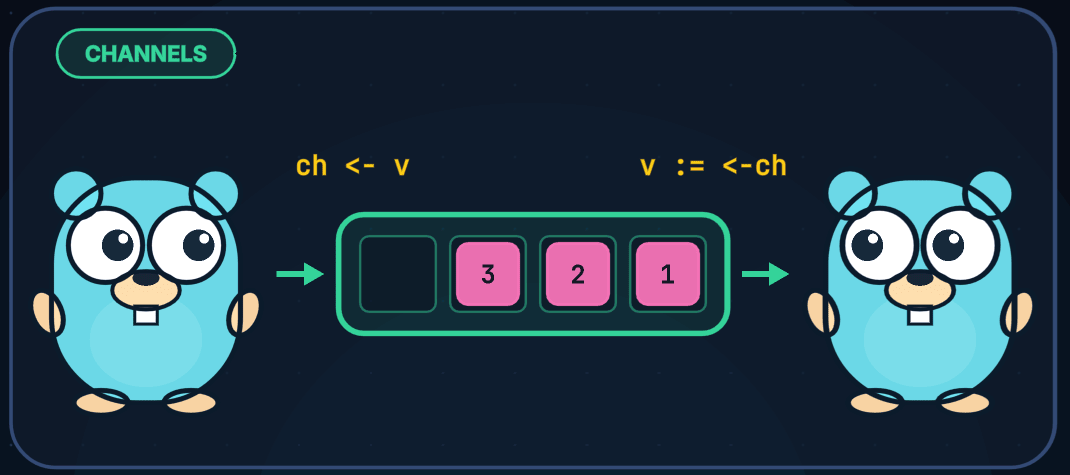
MODULE 8

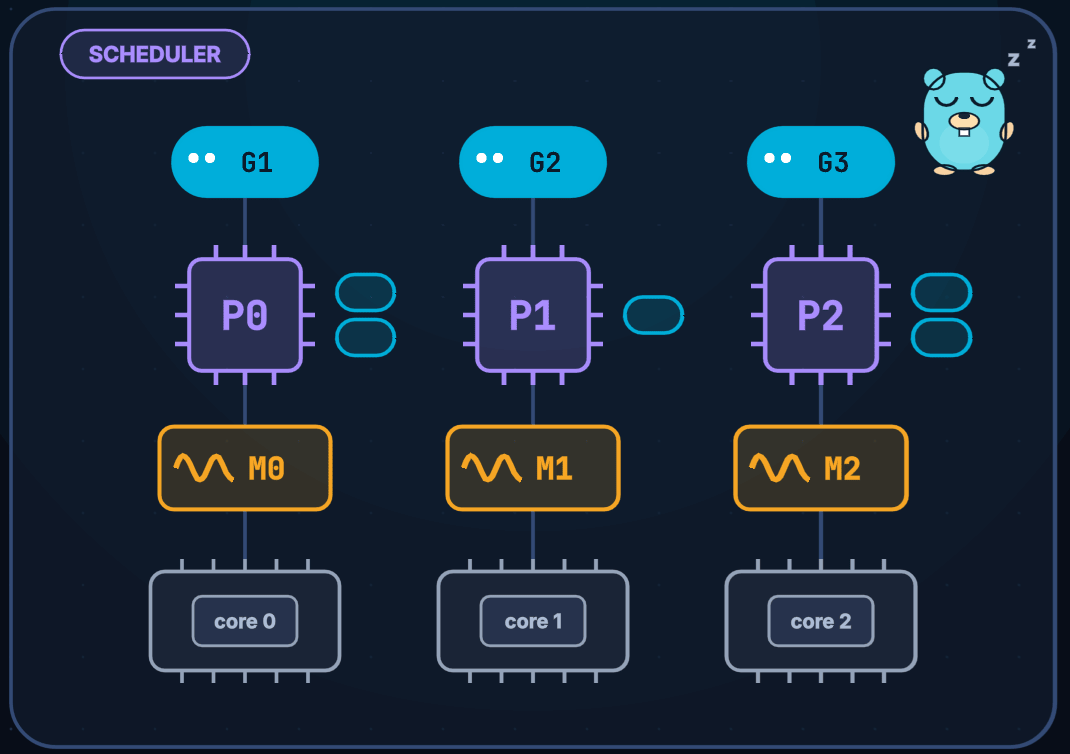
The Concurrency Model

Goroutines, Channels, and the Scheduler

A graphic book for graduate and postgraduate students: from the `go` keyword to the runtime's work-stealing scheduler.

19 chapters · 193 illustrated pages · every listing runs on Go 1.27.3





How to read this book

Every page is a picture first. Learn the cast once and you can read every diagram in the module.

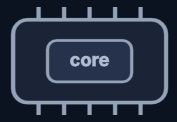
THE CAST

A goroutine, drawn as a gopher: the main character

G states: running, runnable (outline), waiting, dead

M: an operating-system thread (amber)

P: a logical processor that owns a run queue (violet)



A hardware **CPU core**

A **channel** (green) carrying **values** (pink)

A **mutex** (red); **time**: timers, deadlines (blue)

The **operating system** kernel

THE CALLOUTS

- ★ **KEY IDEA** The one idea to remember from the page.
- ≡ **DEFINITION** A precise definition of a term.
- ⚙ **UNDER THE HOOD** What the runtime really does, from the Go source.
- ! **PITFALL** A bug students (and seniors) really write.
- ✓ **PRO TIP** Advice from production code.
- v **GO VERSION NOTE** Behaviour that changed in a Go release.
- ▶ **TRY IT** A small experiment to run yourself.
- ? **THINK ABOUT IT** A question to test your understanding.

THE CODE

listing.go — Go

```
1 ch := make(chan int)
2 go func() { ch <- 42 }()
3 fmt.Println(<-ch)
```

1

- Highlighted lines carry the idea of the listing.
- 1 Numbered markers are explained next to the code.
- Every listing is a real program in `examples/`.

output — terminal

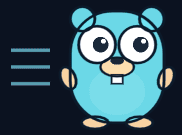
```
$ go run .
42
```

Outputs are **real**: captured by `tools/run_examples.py` with Go 1.27.3 on linux/amd64, 4 vCPUs. When the order can change from run to run, the panel says one possible output.

ONE POSSIBLE OUTPUT

The module at a glance

Six parts, nineteen chapters. Each stop is a chapter; the number under it is its first page.



A Foundations
what concurrency is and what a goroutine costs

- 8.1 Concurrency vs parallelism — p. 5
- 8.2 Threads & goroutines — p. 15
- 8.3 Goroutines in depth — p. 24
- 8.4 Waiting: WaitGroup & errgroup — p. 34

B Channels
communicating by sending values

- 8.5 Channel basics — p. 42
- 8.6 Buffers, close & range — p. 51
- 8.7 Inside a channel — p. 59
- 8.8 select & timers — p. 68

C The runtime
how Go puts goroutines on CPUs

- 8.9 Scheduler I: G, M, P — p. 78
- 8.10 Scheduler II: queues, syscalls, I/O — p. 85
- 8.11 Preemption — p. 93

D Shared memory
when goroutines must share data

- 8.12 sync & atomics — p. 100
- 8.13 The memory model — p. 115

E Production
cancellation, patterns, bugs, tests

- 8.14 context — p. 129
- 8.15 Patterns I — p. 140
- 8.16 Patterns II — p. 150
- 8.17 Pitfalls & debugging — p. 158
- 8.18 Testing concurrency — p. 167

F Capstone & reference
build it all; keep the cheat sheets

- 8.19 Capstone: a crawler — p. 175
- **Cheat sheets** — p. 186
- **Glossary & exercises** — p. 189

What you will be able to do

Outcomes are written as things you can do, from explaining an idea to designing a system.

EXPLAIN the difference between concurrency and parallelism, and why goroutines are cheap

APPLY goroutines, channels, `select`, `sync` and `context` to real programs

ANALYZE how the G-M-P scheduler places work, and read `schedtrace` and race reports

EVALUATE channels versus mutexes versus atomics for a given problem, with data

CREATE leak-free, cancellable, testable concurrent systems such as a crawler

deeper

BEFORE YOU START

- ✓ Functions, **closures** and methods
- ✓ Structs, **interfaces** and error values
- ✓ Slices and maps (and why they are not safe to share)
- ✓ Generics: type parameters such as `[T any]`
- ✓ Packages, modules and `go test`

YOUR TOOLCHAIN

shell — terminal

```
$ go version
go version go1.27.3 linux/amd64
$ cd examples/ch05_pingpong && go run .
pong hits 1
...
$ CGO_ENABLED=0 go run -race .
go: -race requires cgo; enable cgo by setting CGO_ENABLED=1
```

Go 1.25 or newer runs almost every listing; a few use Go 1.27 features and say so. The race detector (`-race`) needs cgo and a C compiler.

EVERY CHAPTER FOLLOWS THE SAME ARC

Opener → Concepts → Code + real output → Under the hood → Pitfalls → Check yourself

CHAPTER

8.1

Concurrency vs Parallelism

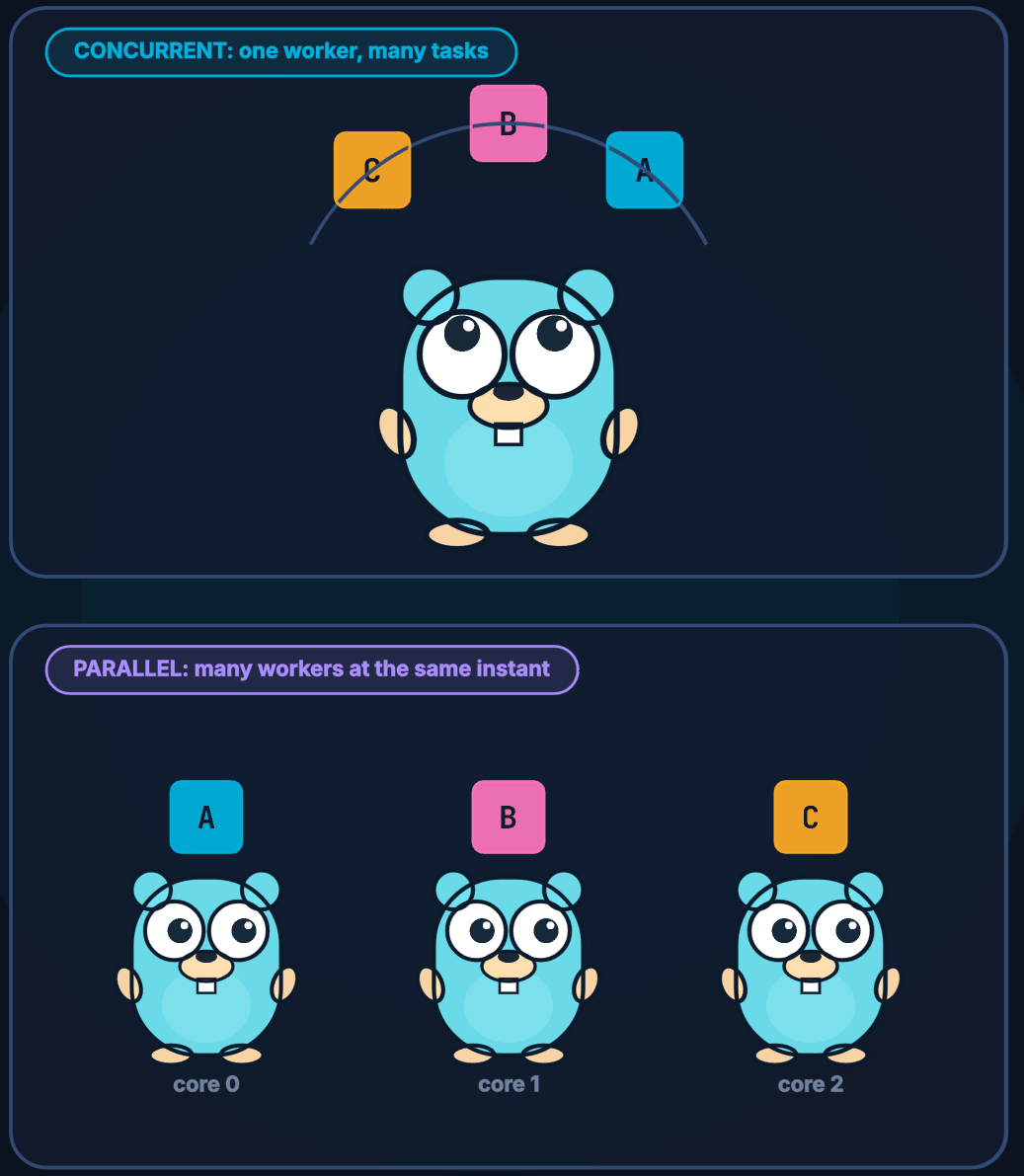
Two words that people use as synonyms. They are not, and the difference shapes every design decision in Go.

IN THIS CHAPTER YOU WILL

- ✓ Define concurrency (structure) and parallelism (execution)
- ✓ Tell I/O-bound from CPU-bound work, and what each one gains
- ✓ Bound the speed-up of any program with Amdahl's law
- ✓ Place Go's CSP model among threads, actors and event loops
- ✓ Name the tools of Go concurrency you will meet in this module

? THE BIG QUESTION

If your machine had a single CPU core, could a Go program still be concurrent? Could it be parallel?



8.1 Two different ideas

Concurrency is about **structure**: dealing with many things at once. Parallelism is about **execution**: doing many things at once.

CONCURRENCY

Concurrency is the composition of independently executing tasks. A program is concurrent when it is built from tasks whose lifetimes **overlap**, whether or not they ever run at the same instant.

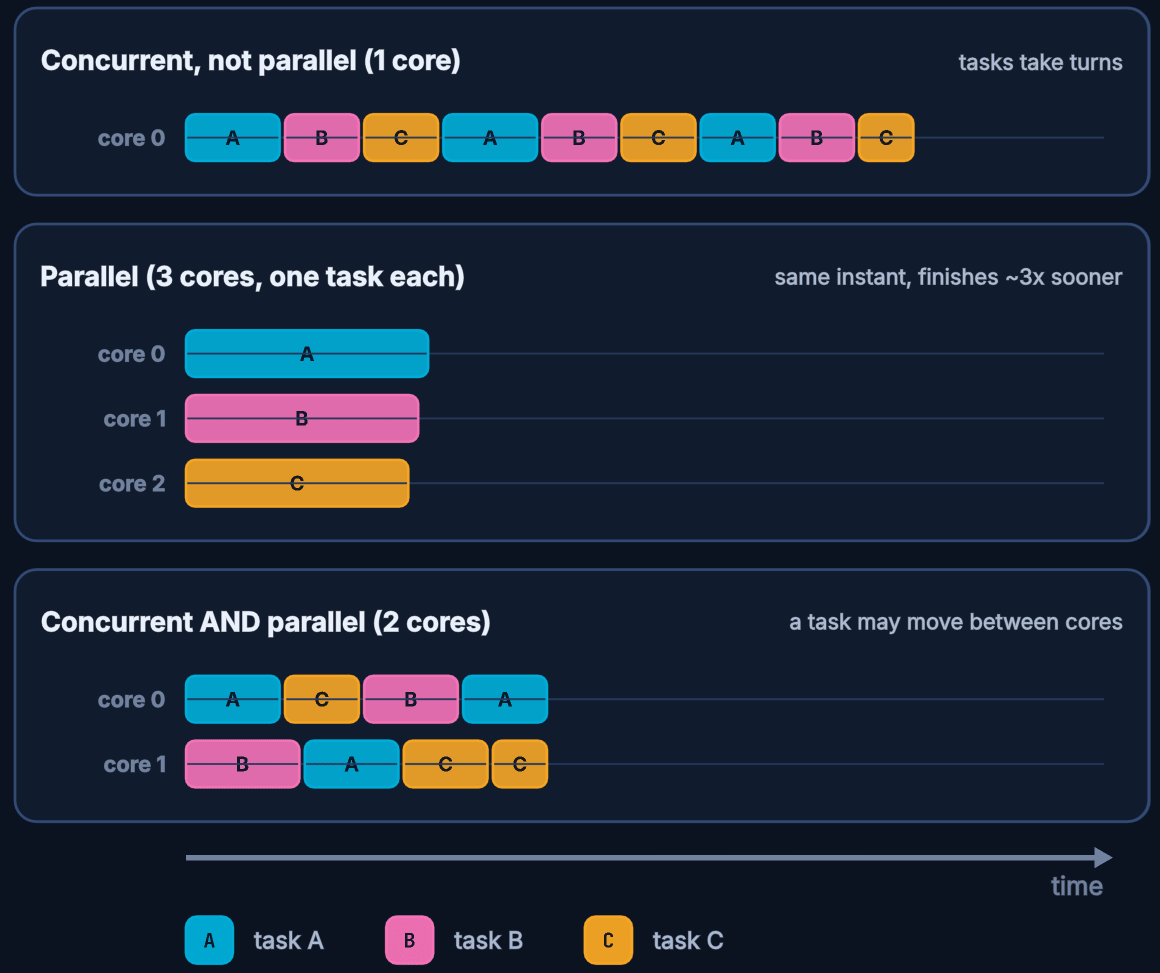
PARALLELISM

Parallelism is the **simultaneous** execution of computations on several cores (or machines). It is a property of a particular run on particular hardware, not of the source code.

> *Concurrency is about dealing with lots of things at once. Parallelism is about doing lots of things at once.*
>
> — Rob Pike, "Concurrency is not Parallelism", 2012

KEY IDEA

Concurrency is a way to **structure** a program; parallelism is one way to **run** it. A well-structured concurrent Go program becomes parallel for free when the machine has more cores.



8.1 The 2 × 2 of concurrency and parallelism

The two properties are independent. Every program sits in one of four boxes; the hardware decides between the two on the right.

PARALLEL? no → yes

Parallel, not concurrent

One instruction stream, many data lanes: SIMD maths, a GPU kernel.

Concurrent and parallel

Many tasks on many cores: a Go web server on 32 cores.

Sequential

One task, one core. A plain `for` loop over files.

Concurrent, not parallel

Tasks interleave on one core: Go with `GOMAXPROCS=1`, a JS event loop.

CONCURRENT? no → yes

THE BIG QUESTION, ANSWERED

With a single core a Go program can be fully **concurrent** (its goroutines take turns) but never **parallel**. Give it four cores and the same binary runs four goroutines at once, without a single code change.

- Concurrency is chosen by the **programmer**: how the code is split into tasks.
- Parallelism is granted by the **runtime and hardware**: how many cores run them.
- In Go the dial is `GOMAXPROCS`: how many goroutines may execute Go code at the same instant.
- Parallel-but-not-concurrent code is rare in Go; it lives in vector instructions and GPUs.

PRO TIP Design for concurrency first. Then measure, and tune parallelism with benchmarks.

8.1 I/O-bound work: concurrency hides waiting

Most server time is spent waiting for networks and disks. While one goroutine waits, another can run, even on one core.

examples/ch01_latency/main.go — Go

```go
// fetch pretends to call a slow service: it only waits.
func fetch(id int) string {
    time.Sleep(100 * time.Millisecond)
    return fmt.Sprintf("page-%d", id)
}

func main() {
    start := time.Now()
    for i := range 5 {
        fetch(i)
    }
    fmt.Println("sequential:", time.Since(start).Round(10*time.Millisecond))

    start = time.Now()
    var wg sync.WaitGroup
    for i := range 5 {
        wg.Go(func() { fetch(i) })
    }
    wg.Wait()
    fmt.Println("concurrent:", time.Since(start).Round(10*time.Millisecond))
}
```

(1) (2)

output — terminal

```
$ go run .
sequential: 500ms
concurrent: 100ms
```

WHAT THE CPU SEES

sequential: 5 × 100 ms in a row

concurrent: five goroutines wait together

(1) each call blocks for 100 ms while the CPU sits idle

(2) `wg.Go` starts one goroutine per call, so all five wait at once

8.1 CPU-bound work: only parallelism helps

Counting primes never waits. Splitting it into goroutines pays off only when there are cores to run them at the same time.

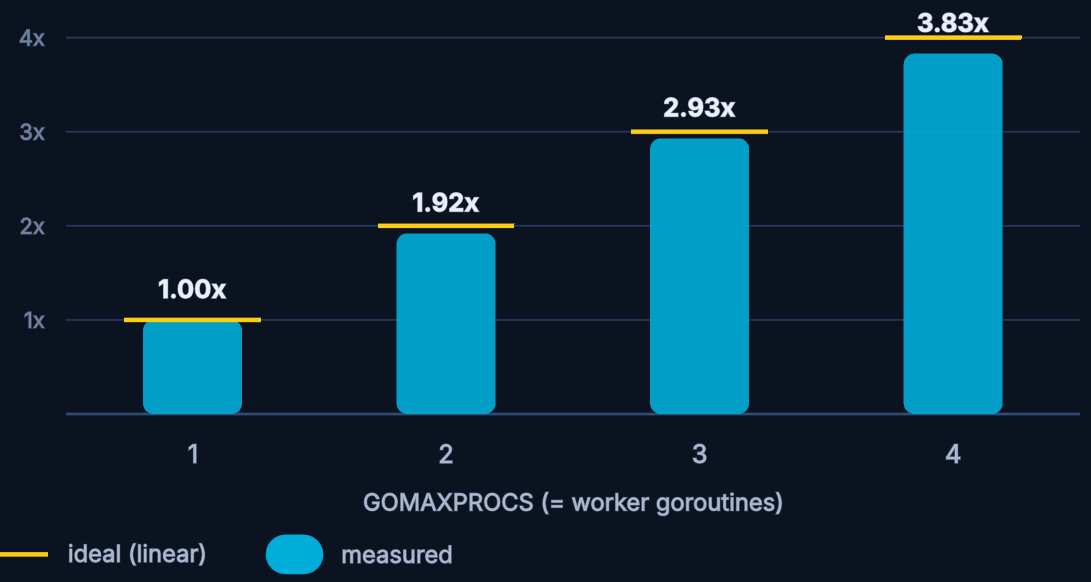
examples/ch01_speedup/main.go — Go

```go
func run(workers, limit int) (int, time.Duration) {
    start := time.Now()
    counts := make([]int, workers)
    var wg sync.WaitGroup
    for w := range workers {
        wg.Go(func() {
            // interleave numbers so every worker gets similar work
            for lo := w * 1000; lo < limit; lo += workers * 1000 {
                counts[w] += countPrimes(lo, min(lo+1000, limit))   (1)
            }
        })
    }
    wg.Wait()
    total := 0
    for _, c := range counts {
        total += c
    }
    return total, time.Since(start)
}
```

measured on this machine — terminal

```
$ go run .
CPUs: 4
GOMAXPROCS=1  primes=216816  time=1.101s  speedup=1.00x
GOMAXPROCS=2  primes=216816  time=573ms   speedup=1.92x
GOMAXPROCS=3  primes=216816  time=375ms   speedup=2.93x
GOMAXPROCS=4  primes=216816  time=287ms   speedup=3.83x
```

SPEED-UP vs GOMAXPROCS (measured)



(1) each worker writes only its own slot `counts[w]`, so no lock is needed

- Ideal is 4x; this cloud VM reached **3.83x**: nearly linear, because the work splits evenly and the workers share nothing.
- Speed-up depends on what else runs: an earlier run of this very program, with two stray busy loops on the same VM, stalled at about **2x**.

★ **KEY IDEA** Parallel speed-up is a property of the **hardware** you run on. Measure it; never assume it.

8.1 Amdahl's law: the serial part sets the ceiling

If a fraction p of the work can run in parallel, n cores can never make the program more than 1 / (1 – p) times faster.

$$S(n) = \frac{1}{(1 - p) + p / n}$$

- **S(n)**: speed-up on n cores
- **p**: fraction of the run time that can be parallelised
- **1 – p**: the serial fraction (start-up, merging results, a lock everyone needs)
- **limit**: as n → ∞, S → 1 / (1 – p)

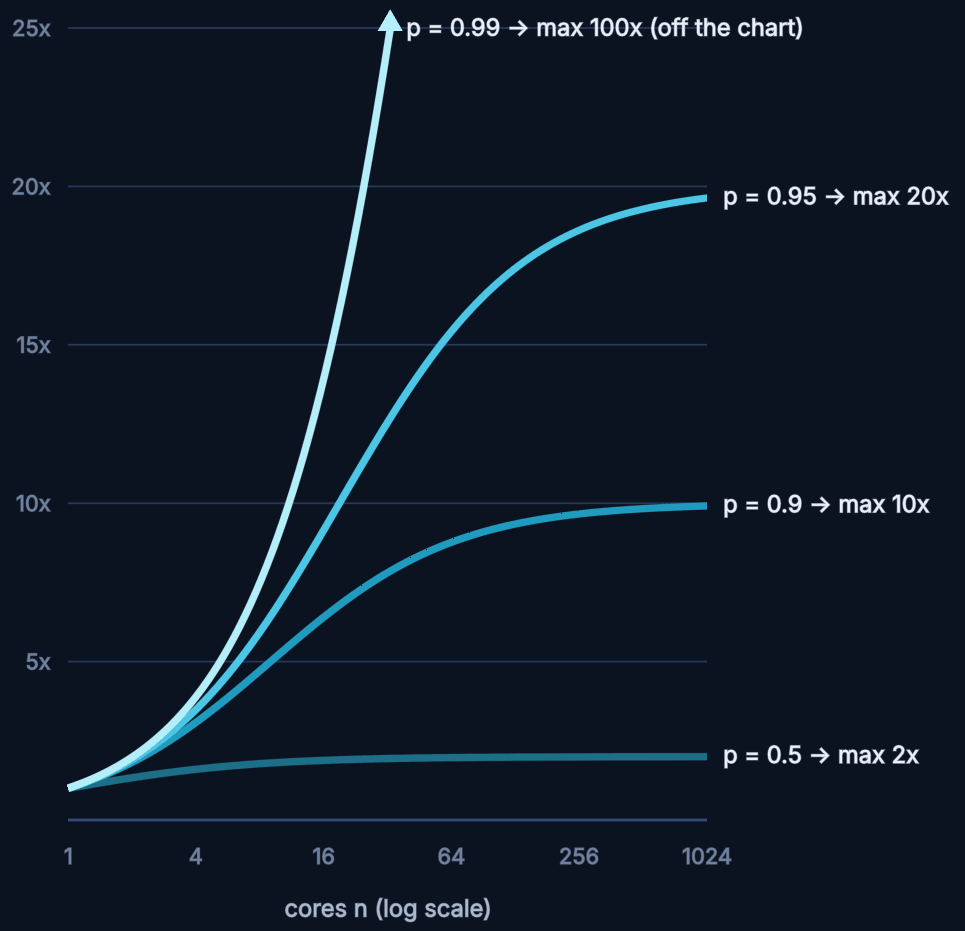
WORKED EXAMPLE

p = 0.9 on 8 cores: S = 1 / (0.1 + 0.9/8) = 1 / 0.2125 = **4.7x**, not 8x. On 1,000 cores: **9.9x**. The 10% serial part dominates.

NOTE

Gustafson's law looks at it the other way round: with more cores we usually grow the problem (more requests, bigger data), so the parallel part grows and the useful speed-up keeps rising.

SPEED-UP S(n) FOR FOUR VALUES OF p



8.1 Four ways to structure a concurrent program

Languages differ in who does the work, how tasks talk to each other, and who decides when they run.

Threads + locks

Tasks share memory; locks protect it.

C, C++, Java, Rust std

Actors

Each actor owns its state and a mailbox; messages go to a named actor.

Erlang/Elixir, Akka

Event loop

One thread runs callbacks or **async** tasks whenever I/O is ready.

JavaScript, Python asyncio

CSP

Anonymous processes; values travel over typed channels.

Go, occam, Clojure core.async

	Threads + locks	Actors	Event loop	CSP (Go)
Unit of work	OS thread	actor (has identity)	callback / async task	goroutine (anonymous)
Communication	shared memory	message to a mailbox	shared memory, one thread	value sent on a channel
Blocking style	blocks a thread	never blocks the mailbox owner	must never block the loop	blocks the goroutine only
Who schedules	the OS kernel	the actor runtime	the event loop	the Go runtime (M:N)

★ **KEY IDEA** Go is **CSP-first** but not CSP-only: package sync gives you threads-and-locks tools too. This module teaches when to use each.

8.1 Go's choice: Communicating Sequential Processes

In 1978 Tony Hoare proposed building programs from sequential processes that synchronise only by passing messages. Go is its most popular descendant.

1978 — CSP paper
C. A. R. Hoare, CACM

1983 — occam
language for the Transputer

1989 — Newsqueak
Rob Pike: channels as values

1992 — Alef
Plan 9 systems language

1995 — Limbo
Inferno OS

2009 — Go
open-sourced, Nov 2009

2012 — Go 1.0
the compatibility promise

Communicate by sharing memory

Both goroutines touch the same memory; a lock decides who goes first.

Share memory by communicating

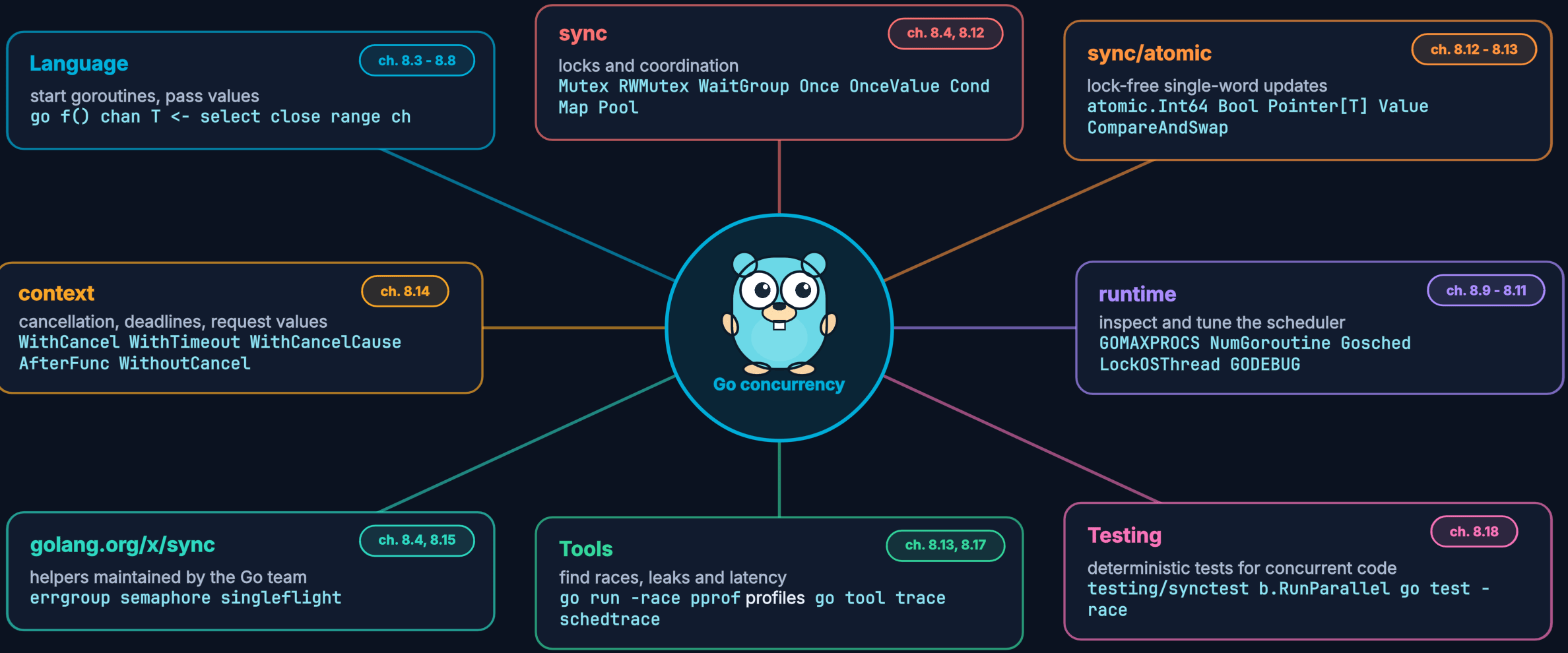
The value travels through a channel and ownership travels with it.

> *Do not communicate by sharing memory; instead, share memory by communicating.*
>
> — Effective Go

KEY IDEA
A send is a **hand-off of ownership**: after `ch <- buf` the sender must not touch `buf` again.

8.1 The toolkit you will master in this module

Language keywords, five standard packages, an extension module and the runtime's own instruments.



8.1 Recap and check yourself

Five ideas to keep from this chapter, and four questions to make sure they stuck.

REMEMBER

- ✓ **Concurrency** = structure (tasks with overlapping lifetimes). **Parallelism** = simultaneous execution.
- ✓ Concurrency helps **I/O-bound** work even on one core by overlapping waits (500 ms → 100 ms).
- ✓ **CPU-bound** work only speeds up with more cores, and only as much as the hardware allows: 3.83x on our 4 vCPUs.
- ✓ **Amdahl**: speed-up ≤ 1 / (1 − p). Shrink the serial part before buying cores.
- ✓ Go follows **CSP**: goroutines plus channels, with sync for the cases where sharing is simpler.

Concurrency is not parallelism... but it **enables** it.

CHECK YOURSELF

Q1. A web crawler spends 95% of its time waiting for HTTP responses. Will raising GOMAXPROCS from 4 to 64 make it 16x faster?

Answer. No. It is I/O-bound: more goroutines (more requests in flight) help; more cores barely do.

Q2. A job is 80% parallelisable. What is the best possible speed-up on infinitely many cores?

Answer. 1 / (1 − 0.8) = 5x.

Q3. Name one program that is parallel but not concurrent.

Answer. A SIMD loop that adds two vectors: one instruction stream, many data lanes.

Q4. In CSP, how do processes find each other?

Answer. They don't: they share a channel. Actors, by contrast, send to a named mailbox.

CHAPTER

8.2

Threads and Goroutines

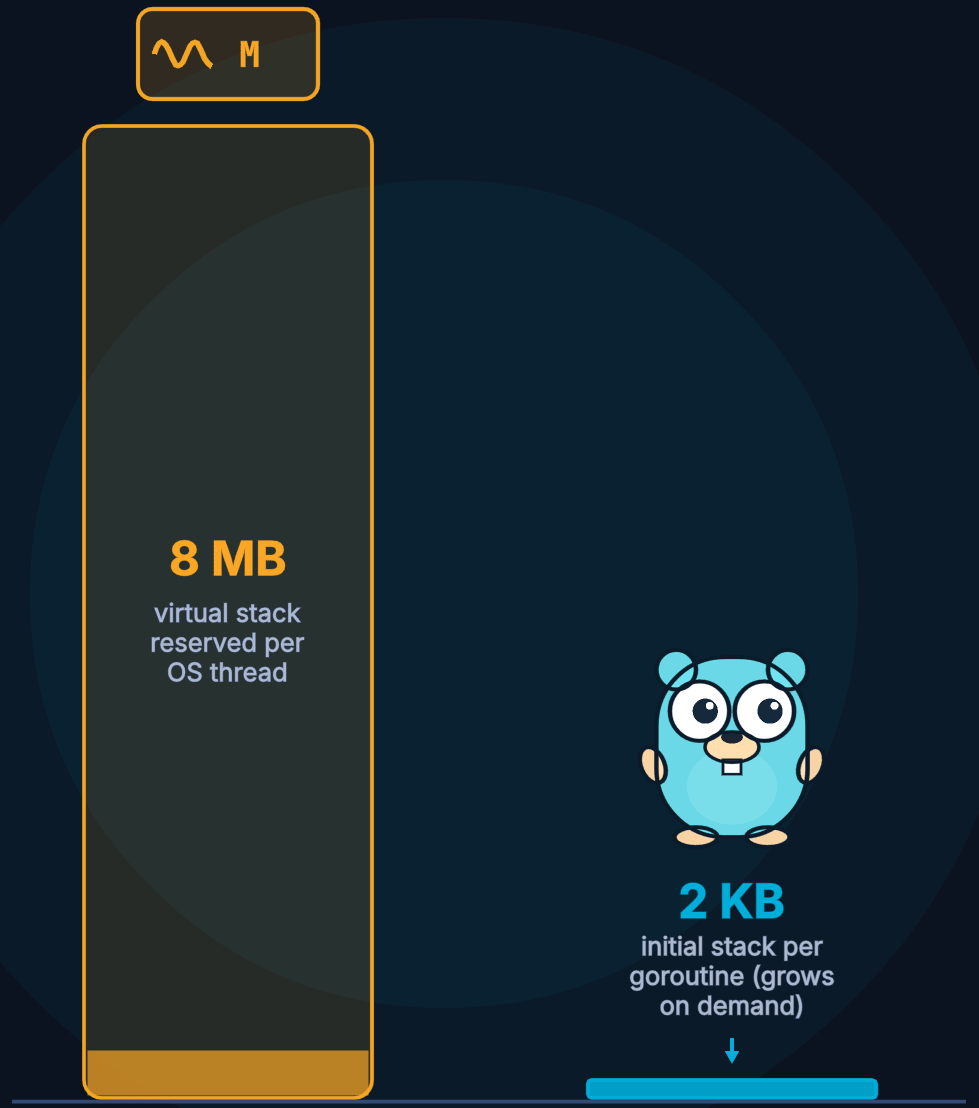
Why can a Go program keep a million goroutines alive when a C program struggles with ten thousand threads? Follow the memory and the context switches.

IN THIS CHAPTER YOU WILL

- ✓ Draw a process: its address space, its threads and what they share
- ✓ Measure what an OS thread costs to create, to switch and to keep
- ✓ Compare the 1:1, N:1 and M:N threading models
- ✓ Read the runtime's g structure and see why a switch is cheap
- ✓ Estimate how many goroutines a server can afford

? THE BIG QUESTION

A goroutine and a thread both run a function on their own stack. Where does the 54x difference in switching cost come from?



drawn to the same scale: the goroutine stack is 1/4096 of the thread's

8.2 Inside a process

A process owns an address space and kernel resources. Its threads share all of it, except their stacks and CPU registers.

virtual address space

THREADS SHARE

- ✓ code, globals and the **heap**
- ✓ open files and sockets (the fd table)
- ✓ the address space and its page tables
- ✓ signal handlers, working directory, user id

EACH THREAD OWNS

- a **stack** (8 MB reserved by default on Linux)
- CPU **registers**: PC, SP, flags, vector registers
- a thread ID, a signal mask, thread-local storage
- a **kernel** stack and task_struct (about 20 KB)

KEY IDEA

Goroutines live **inside one process** and share its heap like threads do. That is why they need channels or locks to cooperate safely.

UNDER THE HOOD

On Linux a thread is a task made by `clone(CLONE_VM | CLONE_FILES | ...)`. The kernel schedules tasks; a process is a group of tasks sharing one memory map.

	Process	Thread
Address space	its own	shared with sibling threads
Create	`fork`/`exec`: copy page tables	`clone`: share almost everything
Talk to others	pipes, sockets, shared mmap	ordinary memory (needs synchronisation)
A crash kills	only itself	the whole process

8.2 What an OS thread really costs

Every switch between threads goes through the kernel. We measured it with a small C program on this machine.

examples/ch02_thread_cost_c/threads.c

```c
static int a2b[2], b2a[2];
static const int ROUNDS = 200000;
static void *pong(void *arg) {
    char c;
    for (int i = 0; i < ROUNDS; i++) {
        if (read(a2b[0], &c, 1) != 1) break;
        if (write(b2a[1], &c, 1) != 1) break;
    }
    return arg;
}
```

measured on this machine — terminal

```
$ ./a.out
thread create+join: 83.4 us
default thread stack (virtual): 8192 KB
thread switch (pipe ping-pong): 12.36 us per switch
```

INDIRECT COST
The hidden cost: after a switch the CPU caches and TLB hold the **other** thread's data. The first microseconds run slowly while they refill.

NOTE Limits on this machine: `pid_max` = 32768, `threads-max` = 128594, and every thread stack uses one of 65530 memory maps.

83 µs
to create and join one thread

12 µs
per switch (pipe ping-pong, includes the syscalls)

8 MB
virtual stack reserved per thread

8.2 What a goroutine costs (measured)

Park 100,000 goroutines on a channel and ask the runtime how much memory they use.

examples/ch02_goroutine_cost/main.go — Go

```go
func main() {
    const n = 100_000
    var before, after runtime.MemStats
    runtime.GC()
    runtime.ReadMemStats(&before)

    block := make(chan struct{})
    var started sync.WaitGroup
    t0 := time.Now()
    for range n {
        started.Add(1)
        go func() {
            started.Done()
            <-block // park until released
        }()
    }
    started.Wait()
    spawn := time.Since(t0)

    runtime.ReadMemStats(&after)
    fmt.Println("goroutines alive:", runtime.NumGoroutine())
    fmt.Printf("spawn time: %v total, %v per goroutine\n",
        spawn.Round(time.Millisecond), spawn/n)
    stack := after.StackInuse - before.StackInuse
    total := after.Sys - before.Sys
    fmt.Printf("stack memory: %.1f MB (%d bytes per goroutine)\n",
        float64(stack)/1e6, stack/n)
    fmt.Printf("all runtime memory: %.1f MB (%d bytes per goroutine)\n",
        float64(total)/1e6, total/n)
```

output — terminal

```
$ go run .
goroutines alive: 100001
spawn time: 244ms total, 2.438µs per goroutine
stack memory: 204.9 MB (2048 bytes per goroutine)
all runtime memory: 276.2 MB (2762 bytes per goroutine)
```

1. each goroutine parks on `<-block`: alive, but using no CPU
2. `StackInuse` grows by exactly the new goroutines' stacks

HOW MANY FIT IN 1 GiB OF RAM?

388,755 goroutines

at 2,762 B each (measured)

128 threads, if the 8 MB stacks were real

virtual reservation

43,691 threads, kernel memory only

about 24 KB: kernel stack + task_struct

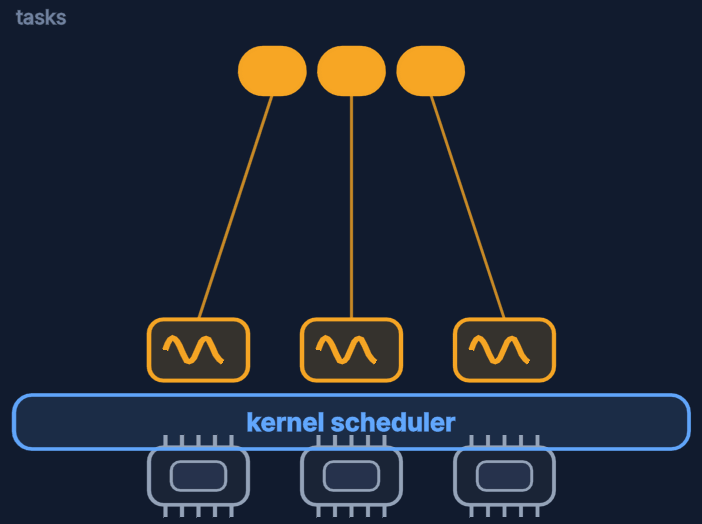
KEY IDEA

Goroutines are cheap because they are **just data** in the Go heap: a small growable stack plus a few hundred bytes of bookkeeping. The kernel never sees them.

8.2 Three ways to map tasks onto OS threads

Who schedules what? The answer decides how cheap a task is and whether tasks can use many cores.

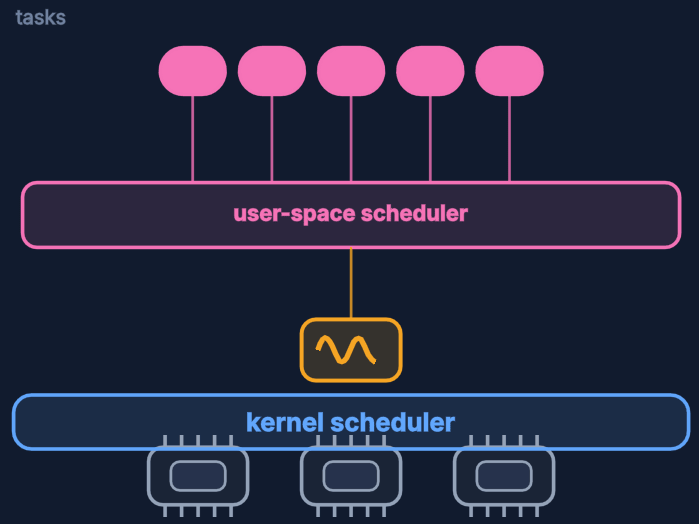
1 : 1 kernel threads



Every task is an OS thread. Simple and parallel, but each one costs kernel memory and a kernel switch.

C pthreads, Java platform threads, Rust std::thread

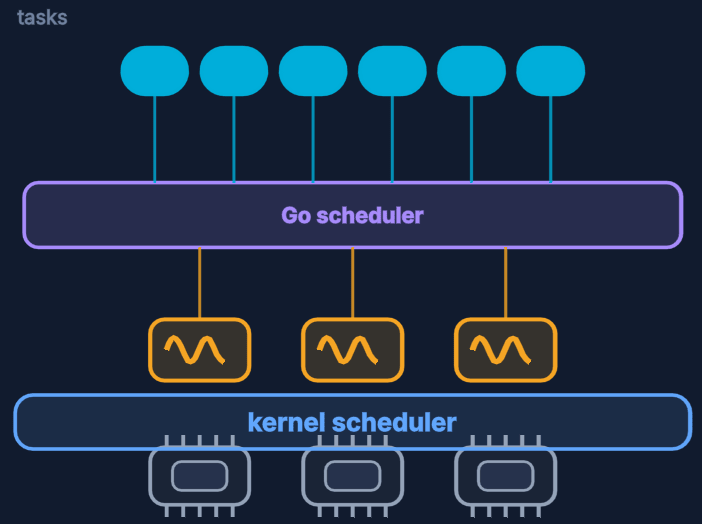
N : 1 green threads



Many user tasks share one OS thread. Very cheap switches, but no parallelism, and one blocking syscall stalls them all.

early Java (1.1), GNU Pth, Python asyncio's loop

M : N hybrid (Go)



M goroutines run on N threads chosen by a runtime scheduler. Cheap like green threads, parallel like 1:1.

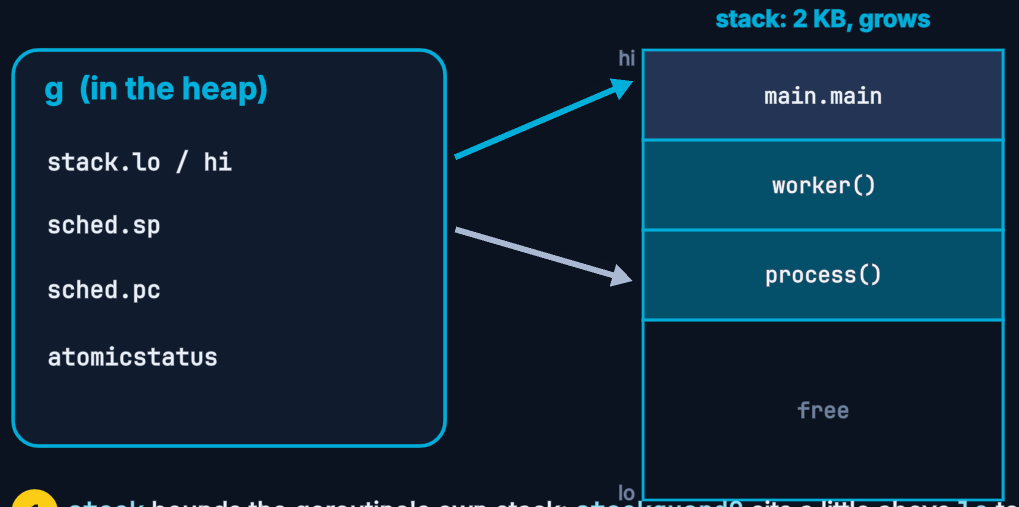
Go, Erlang/BEAM, Java 21 virtual threads, Tokio

8.2 What a goroutine is made of

To the runtime a goroutine is a struct called g (in runtime/runtime2.go) plus a stack it owns. Here are the fields that matter.

runtime/runtime2.go (Go 1.27, abridged) — Go

```
type g struct {
    stack       stack   // [lo, hi): this goroutine's stack          (1)
    stackguard0 uintptr // checked in every function prologue
    _panic      *_panic // innermost panic
    _defer      *_defer // innermost deferred call
    m           *m      // the OS thread running it (if any)
    sched       gobuf   // saved SP, PC, BP when not running        (2)
    atomicstatus atomic.Uint32 // _Grunnable, _Grunning, ...        (3)
    goid        uint64  // id shown in panics and traces
    waitreason  waitReason // "chan receive", "sleep", ...
    preempt     bool    // a request to yield soon
    lockedm     muintptr // set by runtime.LockOSThread
    parentGoid  uint64  // who started it
    startpc     uintptr // the function passed to go
    waiting     *sudog  // channel waits it is part of
    // ... about 60 more fields
}

type gobuf struct {
    sp, pc, bp uintptr // the whole "context" of a switch
    g    guintptr
    ctxt unsafe.Pointer
    lr   uintptr       // link register (ARM)
}
```



1. `stack` bounds the goroutine's own stack; `stackguard0` sits a little above `lo` to detect overflow
2. `sched` is where SP, PC and BP are saved when the goroutine stops running
3. `atomicstatus` is the state machine of chapter 8.3

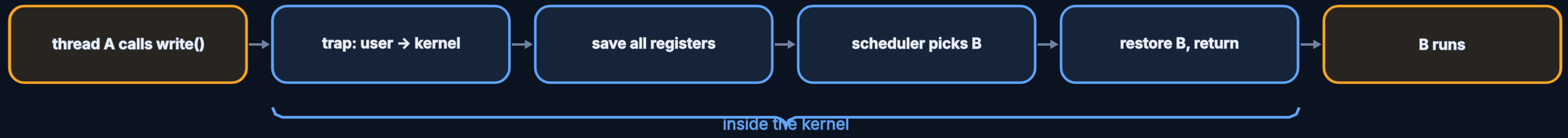
KEY IDEA

A goroutine switch saves **three registers** (SP, PC, BP) into `gobuf` and loads three others. No kernel entry, no page-table change.

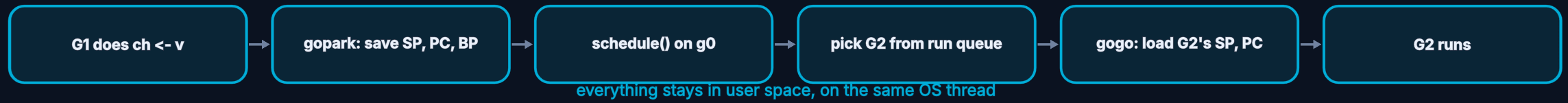
8.2 Why a goroutine switch is cheap

Both programs bounce a value between two workers. The goroutines never leave user space.

OS thread switch



goroutine switch



MEASURED ON THIS MACHINE (lower is better)

54x
cheaper to switch goroutines

Thread numbers come from `threads.c`, goroutine numbers from `examples/ch02_switch_bench` (`GOMAXPROCS=1`). The thread figure includes two syscalls per hop, which is what a real hand-off costs.

8.2 Putting it together: cost per unit of concurrency

Each comparison has its own scale. Lower is better in every row.

create and finish one

97x less for a goroutine

switch between two

54x less for a goroutine

stack reserved at start

4,096x less for a goroutine

BACK-OF-THE-ENVELOPE

A chat server with **1,000,000** connections, one reader and one writer goroutine each:

2,000,000 × 2,762 B ≈ **5.5 GB**

The same design with OS threads would need 2,000,000 threads, far beyond this machine's `pid_max` of 32,768, and 16 TB of reserved stack.

PITFALL
Cheap is not free. A leaked goroutine holds its stack and everything it references forever (chapter 8.17).

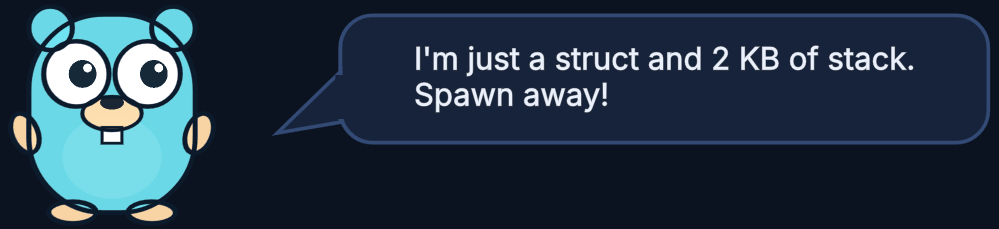
PRO TIP
Goroutine stacks start small and **grow by copying**; deep recursion is fine (chapter 8.3).

8.2 Recap and check yourself

Threads are the kernel's unit of execution; goroutines are the Go runtime's.

REMEMBER

- ✓ Threads of a process share heap, globals and files; each has its own stack and registers.
- ✓ An OS thread costs about 83 µs to create and 12 µs per hand-off here, plus 8 MB of reserved stack.
- ✓ A goroutine starts with a 2 KB stack, costs about 2,762 bytes in total and switches in about 229 ns.
- ✓ Go uses **M:N** scheduling: many goroutines on a few threads, picked by the runtime.
- ✓ A goroutine is a `g` struct plus a stack; a switch saves SP, PC and BP into `g.sched`.



CHECK YOURSELF

Q1. Why can't an N:1 green-thread system use four cores?
Answer. All its tasks share one OS thread, and the kernel only schedules threads.

Q2. Two goroutines in the same program write to the same slice. Are they isolated like processes?
Answer. No. They share the heap exactly like threads; you need a channel or a lock.

Q3. Which three registers does a goroutine switch save?
Answer. The stack pointer, the program counter and the frame pointer (gobuf.sp, pc, bp).

Q4. Roughly how much memory do 50,000 idle goroutines need?
Answer. About 50,000 × 2,762 B ≈ 138 MB.

CHAPTER

8.3

Goroutines in Depth

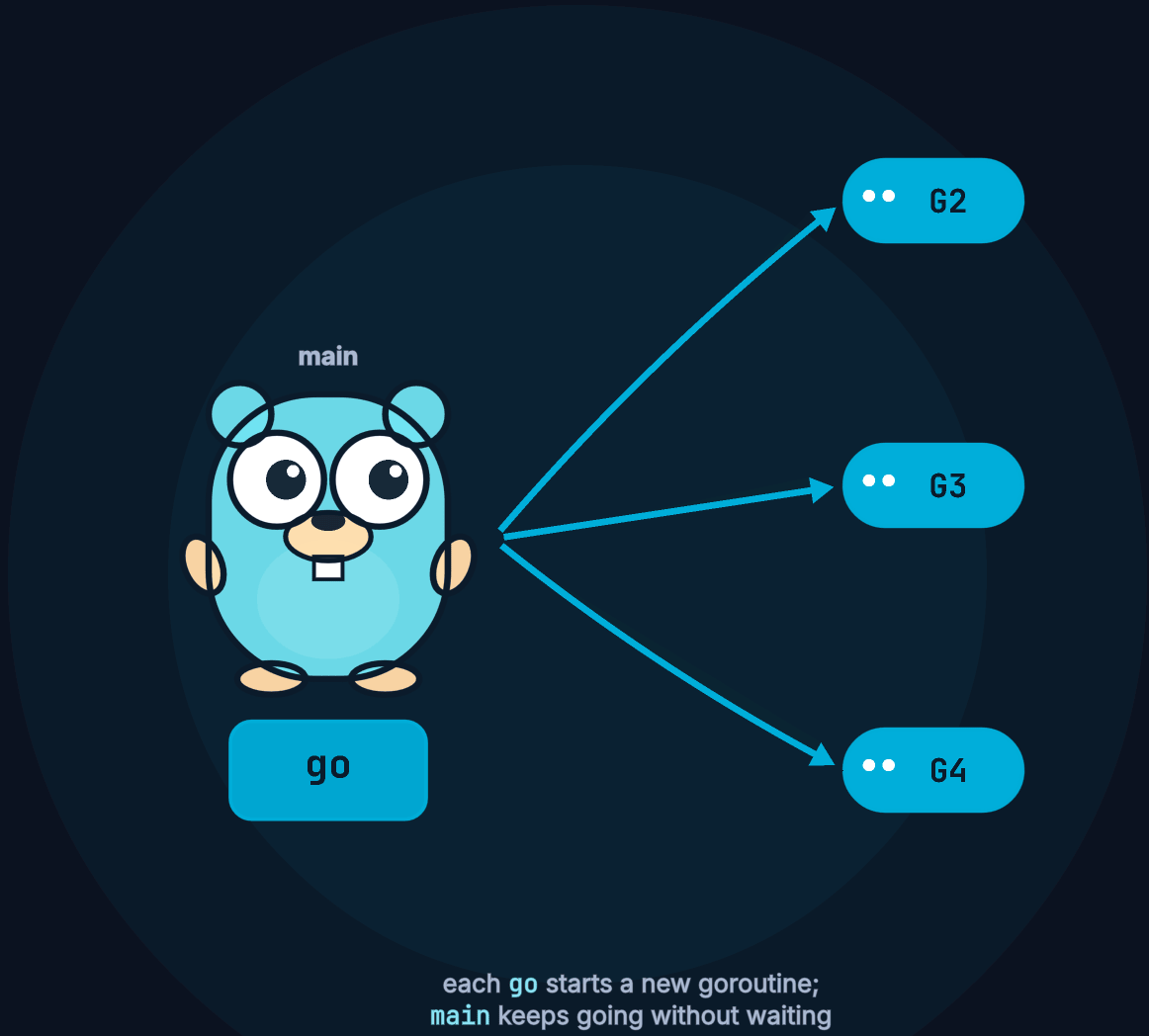
One keyword starts a goroutine. Knowing exactly what it evaluates, when it dies and how its stack grows separates working code from flaky code.

IN THIS CHAPTER YOU WILL

- ✓ Apply the evaluation rules of the `go` statement
- ✓ Explain why returning from `main` kills every goroutine
- ✓ Capture loop variables safely, before and after Go 1.22
- ✓ Trace a goroutine through the runtime's state machine
- ✓ Describe growable stacks, and why goroutines have no ID

? THE BIG QUESTION

A program starts a goroutine that prints three lines, yet none of them appear. Nothing crashed. Where did they go?



each `go` starts a new goroutine;
`main` keeps going without waiting

8.3 The go statement

go + a function call. The call's arguments are evaluated now, in the current goroutine; the body runs later, in a new one.

the forms of go — Go

```go
go worker(id, jobs)       // a named function
go srv.handle(conn)       // a method value
go func(n int) {          // a function literal...
    fmt.Println("got", n)
}(42)                     // ...called on the spot
go fmt.Println("hi")      // any function at all
go close(done)            // ok: close, copy, panic, print

// go len(s)              // NOT allowed: len, cap, make...
// v := go f()            // NOT allowed: go is a statement
```

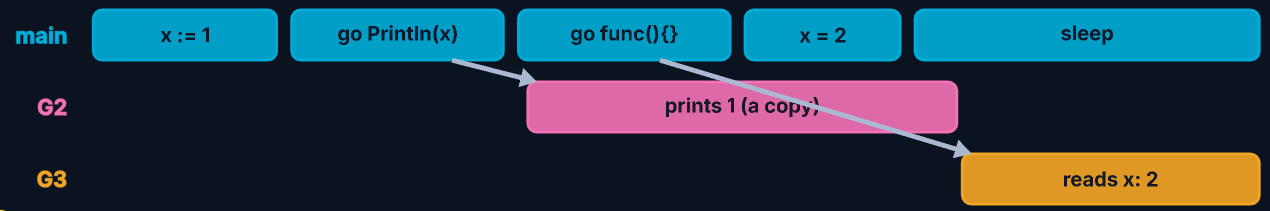
1. **Evaluate** the function value and every argument, right now, in the calling goroutine.
2. **Create** a new goroutine (a g with a fresh stack) and put it on a run queue.
3. **Continue** the caller immediately; it does not wait for the new goroutine.
4. **Discard** the function's return values; use a channel to get results back.

examples/ch03_args/main.go — Go

```go
8  func main() {
9      x := 1
10     go fmt.Println("argument:", x)            // x evaluated NOW   1
11     go func() { fmt.Println("closure:", x) }() // x read LATER      2
12     x = 2
13     time.Sleep(50 * time.Millisecond) // crude wait, fine for a demo
14 }
```

one possible output — terminal

```
$ go run .
closure: 2
argument: 1
```



1. x is an **argument**: copied at the go statement, so G2 prints 1
2. the closure **captures the variable**; G3 reads it later and sees 2

! **PITFALL** The closure version is also a **data race**: main writes x while G3 reads it. Pass values as arguments, or synchronise.

8.3 When main returns, every goroutine dies

A Go program ends the instant `main.main` returns. The runtime does not wait for other goroutines, and their deferred calls never run.

examples/ch03_main_exits/main.go — Go

```go
package main

import "fmt"

func say(s string) {
    for i := range 3 {
        fmt.Println(s, i)
    }
}

func main() {
    go say("goroutine") // scheduled, not waited for   (1)
    fmt.Println("main is done")
} // return -> exit(0) -> every goroutine dies   (2)
```

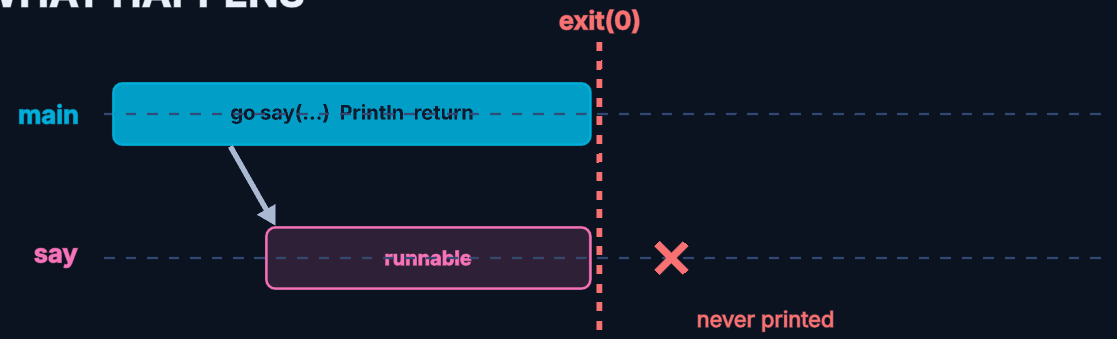
output — terminal

```
$ go run .
main is done
```

THE BIG QUESTION, ANSWERED

The three lines from `say` were never printed: the goroutine was still waiting for a CPU when `main` returned and the process exited.

WHAT HAPPENS



1. `go say(...)` only **schedules** the goroutine
2. `main` returns, the runtime calls `exit(0)`; all goroutines vanish mid-flight

PRO TIP

To wait, use a `sync.WaitGroup` or receive from a channel (chapter 8.4). `time.Sleep` is a guess, not synchronisation: it fails on a slow or busy machine.

UNDER THE HOOD

`os.Exit` also ends the process without running defers. And `runtime.Goexit` in `main` lets the others finish, then crashes with "no goroutines (main called runtime.Goexit) - deadlock!".

8.3 Closures and the loop variable: Go 1.22 changed the rules

Before Go 1.22 a `for` loop had one variable shared by all iterations. Since 1.22 each iteration gets its own. The `go` line in go.mod decides.

ch03_loopvar_old/main.go (go.mod: go 1.21) — Go

```go
func main() {
    var wg sync.WaitGroup
    for i := 0; i < 3; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            fmt.Print(i, " ")
        }()
    }
    wg.Wait()
    fmt.Println()
}
```

output — terminal

```
$ go run .
3 3 3
```

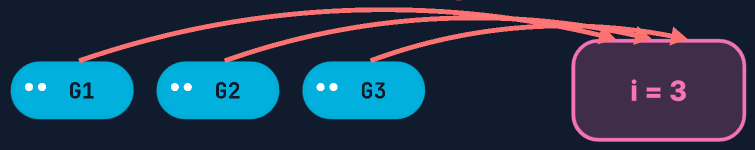
ch03_loopvar/main.go (go.mod: go 1.27) — Go

```go
func main() {
    var wg sync.WaitGroup
    for i := 0; i < 3; i++ { // 1.22+: new i each time
        wg.Go(func() { fmt.Print(i, " ") })
    }
    wg.Wait()
    fmt.Println()
}
```

one possible output — terminal

```
$ go run .
2 1 0
```

Go ≤ 1.21: one i for the whole loop



all three closures read one variable, after the loop ended

Go ≥ 1.22: a new i per iteration

each closure captures its own copy; the order still varies

8.3 Loop variables: rules you can rely on

New code gets the safe behaviour automatically. Old modules, and code you copy from old blogs, still need care.

✖ DON'T

```
// module says "go 1.21" or older
for i := 0; i < 3; i++ {
    go func() {
        fmt.Print(i) // shared i
    }()
}
```

Prints `3 3 3` (or `2 3 3`...): the goroutines run after the loop has moved on.

✓ DO

```
// works with ANY Go version
for i := 0; i < 3; i++ {
    go func(i int) { // a parameter
        fmt.Print(i)  // copied per call
    }(i)
}
```

Passing `i` as an argument copies it at the `go` statement (rule 1 of the go statement).

RULES

- The semantics come from the `go` line in **go.mod**, not from the compiler version.
- Go 1.22+ applies it to `for i := ...` and `for k, v := range ...` loops.
- The old fix `i := i` inside the body still works, but is no longer needed.
- `go vet` (check **loopclosure**) flags the old pattern in old modules.

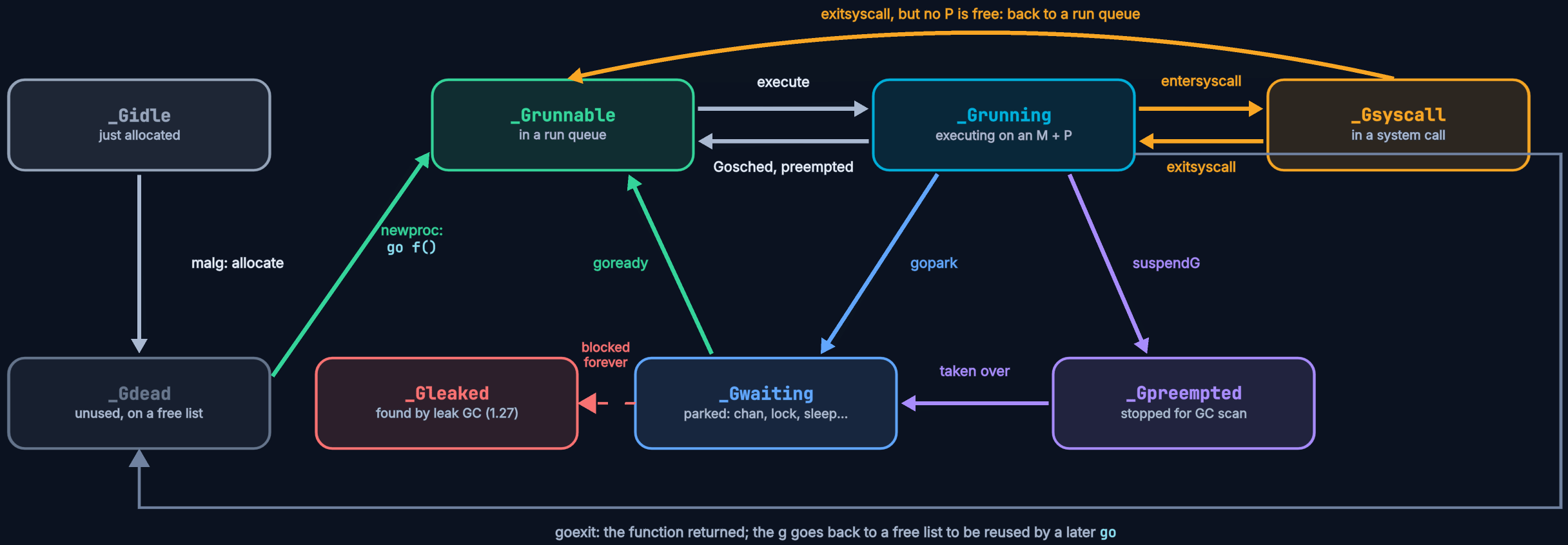
GO VERSION NOTE

Go 1.22 (Feb 2024) made loop variables per-iteration. Go 1.22 also added `for i := range 10` over integers, used throughout this book.

★ **KEY IDEA** Even with per-iteration variables, **goroutines run in any order**. The output `0 2 1` is just one possibility.

8.3 The life of a goroutine: the runtime's state machine

Every goroutine is always in exactly one state, stored in g.atomicstatus. These names appear in the runtime source and its tracebacks.



A goroutine in **_Gwaiting always carries a reason**, shown in tracebacks as `goroutine 7 [chan receive]`:

`chan receive` `chan send` `select` `sleep` `IO wait` `sync.Mutex.Lock` `sync.WaitGroup.Wait` `semacquire`

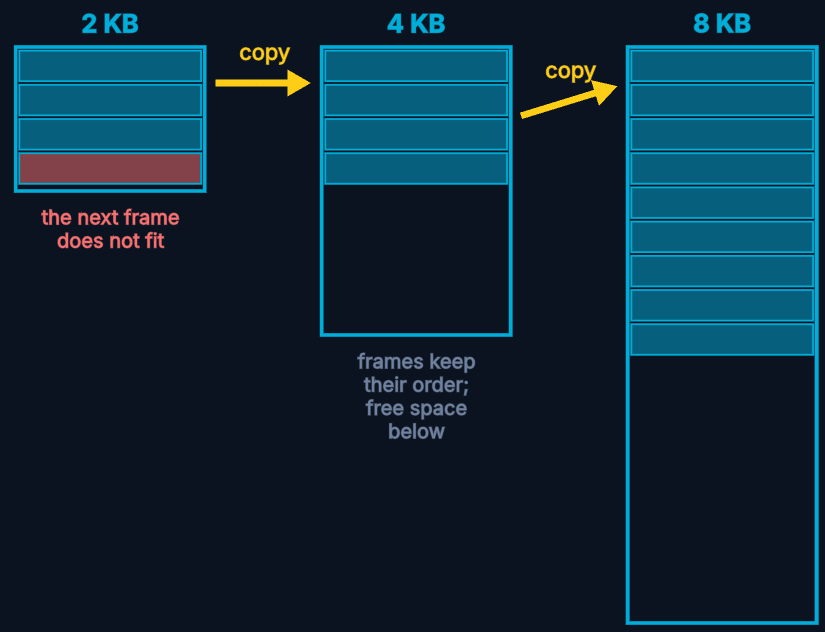
`_Gcopystack` (not drawn) is a brief state while a stack is moved.

8.3 Growable stacks: start small, double, copy

Every Go function starts by comparing the stack pointer with a guard. If the frame will not fit, the runtime moves the stack to a block twice as big.

`go build -gcflags=-S` (real output, trimmed)

```
TEXT main.depth(SB), ABIInternal, $80-8
    CMPQ  SP, 16(R14)  // R14 = g, 16(R14) = g.stackguard0
    JLS   76           // too close to the end? grow first
    PUSHQ BP
    MOVQ  SP, BP
    SUBQ  $72, SP      // now it is safe to claim the frame
    ...
76: MOVQ  AX, 8(SP)    // spill the argument
    CALL  runtime.morestack_noctxt(SB)
    MOVQ  8(SP), AX
    JMP   0            // start the function again
```



1. **Check** the prologue compares SP with `g.stackguard0` (a little above `stack.lo`).
2. **Grow** `morestack` → `newstack` allocates a block **2x** the old size.
3. **Copy** `copystack` copies every frame and rewrites pointers that point into the stack.
4. **Retry** the old stack is freed and the function starts again on the new one.

- Max stack: **1 GB** on 64-bit (250 MB on 32-bit). Beyond it: `fatal error: stack overflow`.
- The GC **shrinks** a stack to half when its goroutine uses less than a quarter.
- Measured: one goroutine recursing 1,000,000 deep held **135 MB** of stack, with no settings.
- Since Go 1.19 new goroutines start at the **average** stack size seen so far (at least 2 KB).

GO VERSION NOTE

Go ≤ 1.2 used **segmented** stacks (linked chunks). A call right at a chunk edge could allocate and free a chunk in a tight loop ("hot split"). Go 1.3 switched to copying.

8.3 No name, no handle, no kill switch

Thread libraries give you an ID, a join handle and a way to cancel. Goroutines offer none of these on purpose; you build them from channels and context.

You want to...	Threads (pthreads, Java)	Go
start a task	`pthread_create`, `new Thread()`	`go f()`
wait for it to finish	`pthread_join`, `t.join()`	`sync.WaitGroup`, a done channel
get its result	join + a shared variable	a result channel, `errgroup`
stop it from outside	`pthread_cancel`, `t.interrupt()`	ask it to stop: close a channel, cancel a `context`
know who it is	thread ID, `currentThread()`	no public ID; pass what you need
store per-task data	thread-local storage	parameters, `context.Value`
set a priority	`pthread_setschedparam`	not possible

? **WHY SO MINIMAL?**

A goroutine ID invites goroutine-local state, which hides data flow and breaks as soon as work moves to another goroutine. Killing from outside would leave locks held and data half-written. So Go makes cancellation **cooperative**: the goroutine checks a signal and exits cleanly.

THE runtime PACKAGE

- `NumGoroutine()`: how many exist now
- `Gosched()`: yield, stay runnable
- `Goexit()`: end this goroutine (defers run)
- `LockOSThread()`: pin to one OS thread

ch03_numgoroutine — Go

```go
func main() {
    fmt.Println("start:", runtime.NumGoroutine())
    stop := make(chan struct{})
    for range 10 {
        go func() { <-stop }()
    }
    fmt.Println("go x10:", runtime.NumGoroutine())
    close(stop) // release all ten
    time.Sleep(10 * time.Millisecond)
    fmt.Println("closed:", runtime.NumGoroutine())
}
```

output — terminal

```
$ go run .
start: 1
go x10: 11
closed: 1
```

8.3 A panic in any goroutine kills the whole program

`recover` only works in the goroutine that is panicking. A panic that reaches the top of any goroutine's stack crashes the process.

examples/ch03_panic/main.go — Go

```go
func main() {
    defer func() {
        // runs in main's goroutine only
        if r := recover(); r != nil {                    (1)
            fmt.Println("recovered:", r)
        }
    }()
    go func() {
        panic("boom in a worker") // nobody recovers HERE   (2)
    }()
    time.Sleep(100 * time.Millisecond)
    fmt.Println("unreachable")
}
```

output — terminal

```
$ go run .
panic: boom in a worker

goroutine 19 [running]:
main.main.func2()
    ./main.go:16 +0x25
created by main.main in goroutine 1
    ./main.go:15 +0x3b
exit status 2
```

EACH GOROUTINE UNWINDS ITS OWN STACK

(1) recover waits here, but nothing panics in main

(2) unwinds to its top, finds no recover: crash

pattern: recover where the panic happens — Go

```go
// recover INSIDE the goroutine
func safeGo(f func(), onPanic func(any)) {
    go func() {
        defer func() {
            if r := recover(); r != nil {
                onPanic(r) // log, count, restart
            }
        }()
        f()
    }()
}
```

GO VERSION NOTE `WaitGroup.Go` (Go 1.25) re-panics instead of calling `Done`, so a crashing task can never make `Wait` return early.

8.3 Recap and check yourself

The goroutine itself: how it starts, lives, grows and ends.

REMEMBER

- ✓ `go f(x)` evaluates `f` and `x` **now**; the body runs later in a new goroutine; results are discarded.
- ✓ When `main` returns the process exits: wait with a WaitGroup or channel, never with `time.Sleep`.
- ✓ Go 1.22+ gives each loop iteration its own variable (controlled by go.mod's `go` line).
- ✓ States: runnable → running → waiting/syscall → runnable … → dead (then reused).
- ✓ Stacks start at 2 KB (or the running average), double by copying, and can reach 1 GB.
- ✓ No IDs, no join, no kill: cancellation is cooperative. A panic anywhere kills everything.

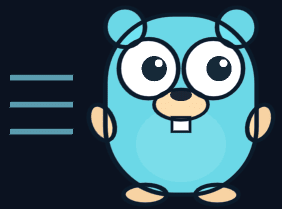
CHECK YOURSELF

Q1. `x := 1; go fmt.Println(x); x = 2`. What does the goroutine print?
Answer. 1. Arguments are evaluated at the go statement.

Q2. Your program prints nothing from its goroutines on a CI server but works on your laptop. First suspect?
Answer. main returns (or a Sleep-based wait expires) before they run.

Q3. Which state does a goroutine blocked in `<-ch` have, and which wait reason?
Answer. _Gwaiting with reason "chan receive".

Q4. Why can Go grow stacks by copying when C cannot?
Answer. The runtime knows every pointer into the stack (from compiler metadata) and can fix them up.



Started by `go`, finished by return. Never by force.

CHAPTER

8.4

Waiting for Goroutines

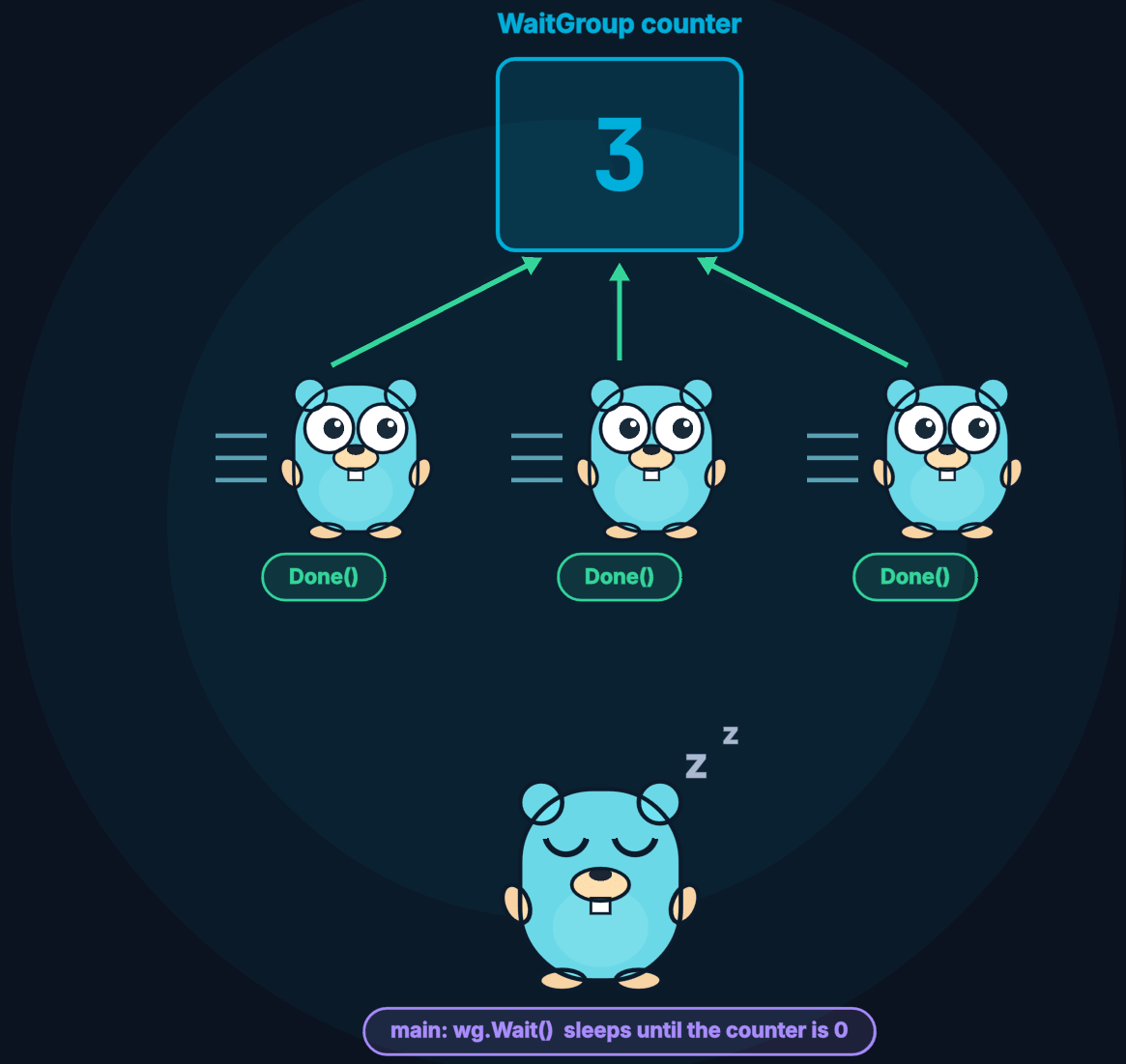
Starting goroutines is easy. Knowing when they have all finished, and collecting what they produced, is where real programs go wrong.

IN THIS CHAPTER YOU WILL

- Use `sync.WaitGroup` correctly: Add, Done, Wait
- Let Go 1.25's `wg.Go` do the bookkeeping for you
- Recognise the four classic WaitGroup bugs
- Collect results safely in three different shapes
- Use `errgroup` to wait, fail fast, cancel and limit

? THE BIG QUESTION

Five goroutines each fetch a page. How does main know that all five are finished, and which one failed?



8.4 sync.WaitGroup: a counter you can wait on

`Add(n)` raises the counter, `Done()` lowers it by one, and `Wait()` blocks until it reaches zero.

examples/ch04_waitgroup/main.go — Go

```go
func worker(id int, wg *sync.WaitGroup) {
    defer wg.Done() // always runs, even on panic
    time.Sleep(time.Duration(id) * 10 * time.Millisecond)
    fmt.Println("worker", id, "done")
}

func main() {
    var wg sync.WaitGroup
    for id := 1; id <= 3; id++ {
        wg.Add(1) // BEFORE the go statement
        go worker(id, &wg)
    }
    wg.Wait() // blocks until the counter is 0
    fmt.Println("all workers finished")
}
```

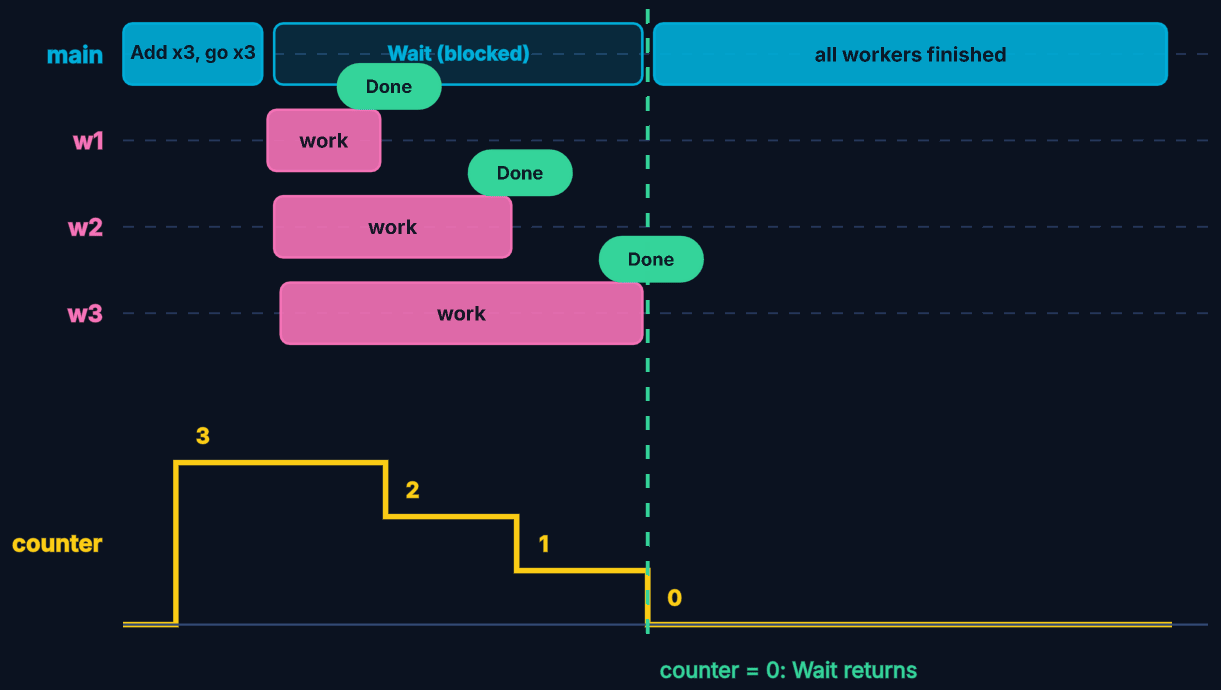
(1) (2) (3)

output — terminal

```
$ go run .
worker 1 done
worker 2 done
worker 3 done
all workers finished
```

1. `defer wg.Done()` first, so the count drops even on an early return
2. `Add` **before** `go`, in the goroutine that will call `Wait`
3. pass `&wg`: a WaitGroup must never be copied

THE COUNTER OVER TIME



KEY IDEA

WaitGroup answers one question: **are they all done?** It carries no results and no errors; combine it with a channel or a slice for those.

8.4 Go 1.25: wg.Go does the bookkeeping

`wg.Go(f)` is `Add(1)` + `go` + `defer Done()` in one call, so the two classic mistakes become impossible to write.

before Go 1.25 — Go

```go
var wg sync.WaitGroup
for _, url := range urls {
    wg.Add(1)              // must come first
    go func() {
        defer wg.Done()    // must not be forgotten
        fetch(url)
    }()
}
wg.Wait()
```

Go 1.25 and later — Go

```go
var wg sync.WaitGroup
for _, url := range urls {
    wg.Go(func() {         // Add, go and Done in one
        fetch(url)
    })
}
wg.Wait()
```

sync/waitgroup.go, Go 1.27 (comments trimmed) — Go

```go
func (wg *WaitGroup) Go(f func()) {
    wg.Add(1)
    go func() {
        defer func() {
            if x := recover(); x != nil {
                panic(x)  // f panicked: crash, no Done
            }
            wg.Done()     // f returned (or Goexit)
        }()
        f()
    }()
}
```

examples/ch04_wggo/main.go — Go

```go
    for i := 1; i <= 100; i++ {
        wg.Go(func() { // Add(1) + go + defer Done()
            mu.Lock()
            sum += i
            mu.Unlock()
        })
    }
    wg.Wait()
    fmt.Println("sum =", sum)
```

output — terminal

```
$ go run .
sum = 5050
```

GO VERSION NOTE `WaitGroup.Go` arrived in **Go 1.25** (August 2025). Its doc says: "The function f must not panic."

8.4 Four ways to misuse a WaitGroup, and what you see

The first three outputs are real runs. The first bug is silent; the others crash or hang.

1. Add inside the goroutine

```
    for range 5 {
        go func() {
            wg.Add(1) // BUG: may run after Wait
            defer wg.Done()
            time.Sleep(10 * time.Millisecond)
            finished.Add(1)
        }()
    }
    wg.Wait() // the counter may still be 0 here
```

`Wait` may run before any `Add`: it sees 0 and returns at once. **Silent and wrong.**

```
finished before Wait
    returned: 5 of 5
```

2. Copying the WaitGroup

```
// BUG: wg is passed by value (a copy)
func worker(id int, wg sync.WaitGroup) {
    defer wg.Done() // lowers the copy
    fmt.Println("worker", id, "done")
}

func main() {
    var wg sync.WaitGroup
    wg.Add(1)
    go worker(1, wg)
    wg.Wait() // the original never reaches 0
}
```

`Done` lowers the copy, so the original never reaches 0: the program deadlocks. `go vet` catches it:

```
main.go:9:24: worker
    passes lock by value:
    sync.WaitGroup
    contains sync.noCopy
```

3. One Done too many

```
func main() {
    var wg sync.WaitGroup
    wg.Add(1)
    wg.Done()
    wg.Done() // one Done too many
}
```

The counter would go below zero, so `Done` panics:

```
panic: sync: negative
    WaitGroup counter
```

4. Reusing it too early

```
wg.Add(1)
go job(&wg)
go wg.Wait()   // waiter A
wg.Add(1)      // new batch: too early
```

A new `Add` while an old `Wait` is still returning panics with (from sync/waitgroup.go):

```
panic: sync: WaitGroup is
    reused before
    previous Wait has
    returned
```

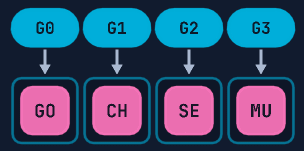
§ **RULE** **Add** before `go`, in the parent. **Done** with `defer`, in the child. **Pass a pointer**, never a copy. **One batch** per Wait. Or simply use `wg.Go`.

8.4 Collecting results: three safe shapes

A WaitGroup tells you when; it does not carry values. Pick a shape for the values, and make sure no two goroutines write the same memory.

A slot per goroutine

Pre-size a slice; goroutine i writes only `out[i]`. Different elements are different memory: no lock, and the order is kept.

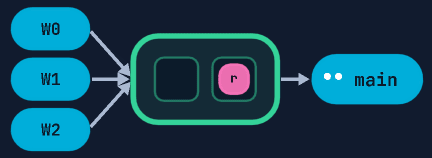



```
13    var wg sync.WaitGroup
14    for i, w := range words {
15        wg.Go(func() {
16            upper[i] = strings.ToUpper(w) // own slot: no race
17        })
18    }
19    wg.Wait()
20    fmt.Println(upper)
```

```
[GO CHAN SELECT MUTEX]
```

A results channel

Workers send; a closer goroutine waits for all of them, then closes; main ranges. Values arrive in finish order.

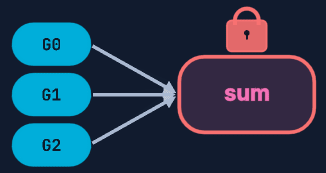



```
11    for id := 1; id <= 3; id++ { // three senders
12        wg.Go(func() {
13            results <- fmt.Sprintf("worker %d", id)
14        })
15    }
16
17    go func() { // closer: wait for ALL, then close
18        wg.Wait()
19        close(results)
20    }()
```

```
received from worker 1
received from worker 2
received from worker 3
total: 3
```

A shared total + lock

Workers update one variable under a mutex. Simple, but every worker queues for the same lock.




```
12    for i := 1; i <= 100; i++ {
13        wg.Go(func() { // Add(1) + go + defer Done()
14            mu.Lock()
15            sum += i
16            mu.Unlock()
17        })
18    }
19    wg.Wait()
20    fmt.Println("sum =", sum)
```

```
sum = 5050
```

8.4 errgroup: wait, fail fast, cancel the rest

`golang.org/x/sync/errgroup` adds three things a WaitGroup lacks: the first error, a context that is cancelled on failure, and a concurrency limit.

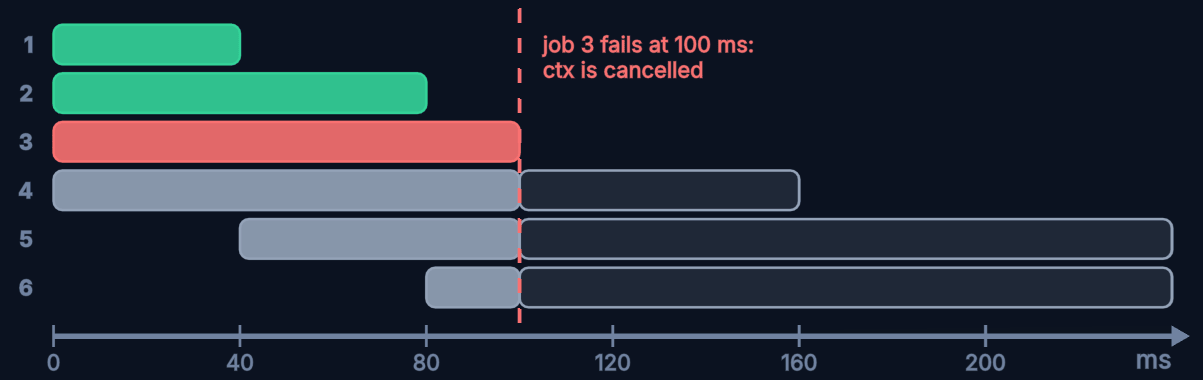
examples/ch04_errgroup/main.go — Go

```go
func fetch(ctx context.Context, id int) error {
    if id == 3 {
        time.Sleep(100 * time.Millisecond)
        return errors.New("fetch 3: server error")
    }
    select {
    case <-time.After(time.Duration(id) * 40 * time.Millisecond):
        fmt.Println("fetched", id)
        return nil
    case <-ctx.Done(): // a sibling failed: stop early
        fmt.Println("cancelled", id)
        return ctx.Err()
    }
}

func main() {
    g, ctx := errgroup.WithContext(context.Background())
    g.SetLimit(4) // at most 4 goroutines at once
    for id := 1; id <= 6; id++ {
        g.Go(func() error { return fetch(ctx, id) })
    }
    err := g.Wait() // first non-nil error
    fmt.Println("Wait returned:", err)
}
```

(1) (2) (3) (4)

one possible output — terminal

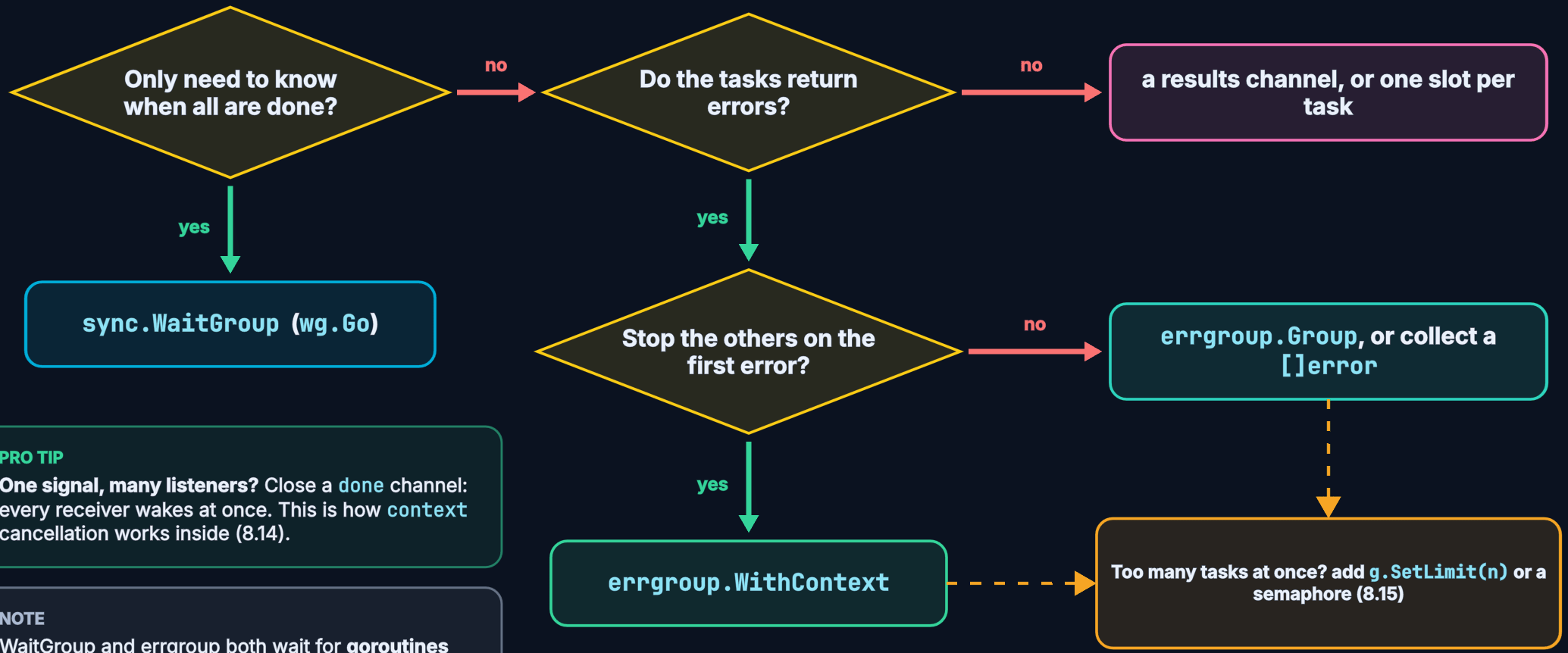
```
$ go run .
fetched 1
fetched 2
cancelled 4
cancelled 6
cancelled 5
Wait returned: fetch 3: server error
```



1. each task watches `ctx.Done()` and returns early
2. `WithContext`: ctx is cancelled on the first error
3. `SetLimit(4)`: `g.Go` blocks while 4 tasks run
4. `Wait` returns the **first** non-nil error

8.4 Which waiting tool do I need?

Follow the questions from the top left. Each answer is the simplest tool that does the job.



PRO TIP

One signal, many listeners? Close a `done` channel: every receiver wakes at once. This is how `context` cancellation works inside (8.14).

NOTE

WaitGroup and errgroup both wait for **goroutines you started**. To wait for an event (a value, a deadline), use a channel and `select` (8.8).

8.4 Recap and check yourself

Waiting is synchronisation too: get it wrong and the program either exits early or hangs.

REMEMBER

- ✓ `Add` before `go`, `defer Done()` inside, `Wait` in the parent. Or use `wg.Go` (Go 1.25+).
- ✓ Never copy a WaitGroup; `go vet` reports "passes lock by value".
- ✓ A WaitGroup carries no data: use a slot per goroutine, a channel, or a locked aggregate.
- ✓ `errgroup.WithContext` returns the first error and cancels the siblings' context.
- ✓ `g.SetLimit(n)` caps how many tasks run at once; `g.Go` blocks when the limit is reached.

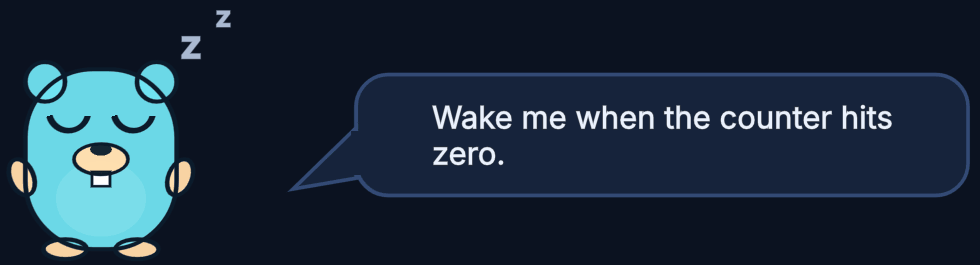
CHECK YOURSELF

Q1. Why does calling `wg.Add(1)` inside the new goroutine sometimes work in tests?
Answer. Luck: the goroutines ran before `Wait`. On a loaded machine `Wait` sees 0 and returns early.

Q2. Ten goroutines append to one shared slice under a WaitGroup. Is it safe?
Answer. No. `append` reads and writes the slice header: that is a data race. Use slots, a channel or a mutex.

Q3. errgroup task 2 fails while task 5 is blocked in a database call that ignores ctx. When does Wait return?
Answer. Only after task 5 finishes: cancellation is cooperative, so pass ctx all the way down.

Q4. What does `wg.Go` do if f panics?
Answer. It re-panics without calling Done, so the program crashes.



CHAPTER

8.5

Channel Basics

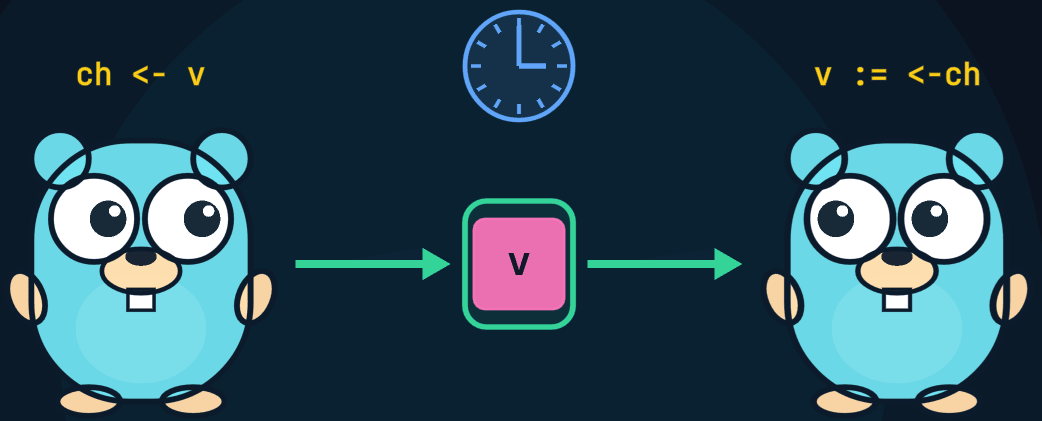
A channel is a typed pipe between goroutines. Sending and receiving on it is how Go programs hand off work and synchronise.

IN THIS CHAPTER YOU WILL

- ✓ Create channels, send, receive and use comma-ok
- ✓ Explain why an unbuffered send waits for a receiver
- ✓ Read and fix the runtime's deadlock error
- ✓ Restrict channels with direction types: `chan<-` and `<-chan`
- ✓ Use channels as values: in structs, as replies, as signals

? THE BIG QUESTION

If a send on an unbuffered channel blocks until someone receives, how can two goroutines ever exchange a value without both getting stuck?



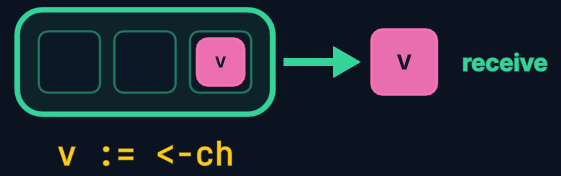
the hand-off: both goroutines meet at the channel, the value moves, then both continue

8.5 Making, sending and receiving

The arrow `<-` always points the way the data flows: into the channel for a send, out of it for a receive.

make	`ch := make(chan int)`	an unbuffered channel of int
make	`ch := make(chan int, 8)`	buffered: room for 8 values (8.6)
send	`ch <- v`	put v in; may block
receive	`v := <-ch`	take a value out; may block
receive	`v, ok := <-ch`	ok is false once ch is closed and empty
receive	`<-ch`	wait and throw the value away
close	`close(ch)`	no more values will be sent (8.6)
inspect	`len(ch), cap(ch)`	queued values, buffer size
loop	`for v := range ch`	receive until closed and drained

READ THE ARROW



- The type is `chan T`; the zero value is **nil** (never ready).
- A channel value is a **pointer** to a runtime struct (`hchan`, 8.7): copying it shares the same channel.
- Channels are comparable: `ch1 == ch2` is true only for the same channel.
- Any type can travel, including other channels and functions.

8.5 An unbuffered channel is a rendezvous

A send completes only when a receiver takes the value, and a receive completes only when a sender provides one. Whoever arrives first waits.

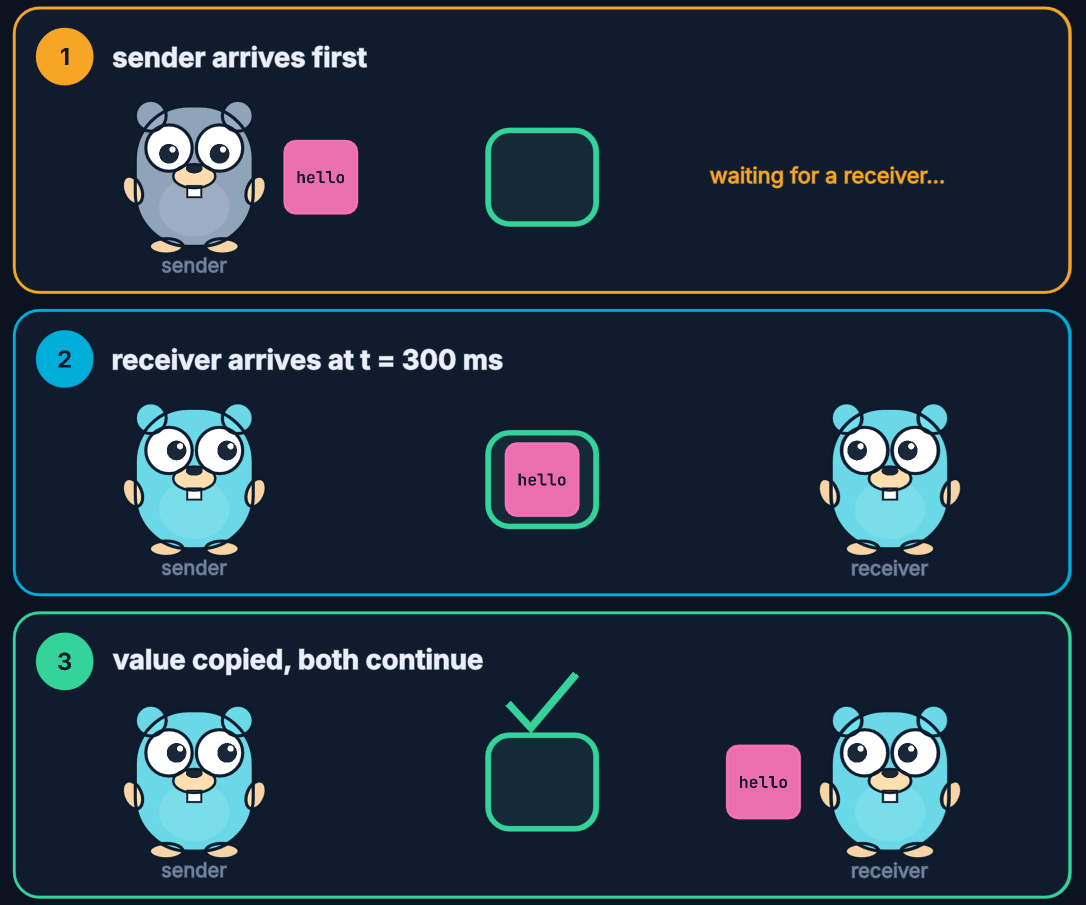
examples/ch05_handoff/main.go — Go

```go
func main() {
    start := time.Now()
    ms := func() int64 { return time.Since(start).Milliseconds() / 100 * 100 }
    ch := make(chan string)

    go func() {
        time.Sleep(300 * time.Millisecond) // receiver arrives late
        fmt.Printf("t=%dms receiver: ready\n", ms())
        v := <-ch
        fmt.Printf("t=%dms receiver: got %q\n", ms(), v)
    }()

    fmt.Printf("t=%dms sender:   sending...\n", ms())
    ch <- "hello" // blocks until the receiver takes it
    fmt.Printf("t=%dms sender:   send completed\n", ms())
    time.Sleep(10 * time.Millisecond)
}
```

output — terminal

```
$ go run .
t=0ms sender:   sending...
t=300ms receiver: ready
t=300ms receiver: got "hello"
t=300ms sender:   send completed
```

KEY IDEA Communication **and** synchronisation: when the send returns, the receiver has the value.



8.5 Ping-pong: two goroutines, one channel

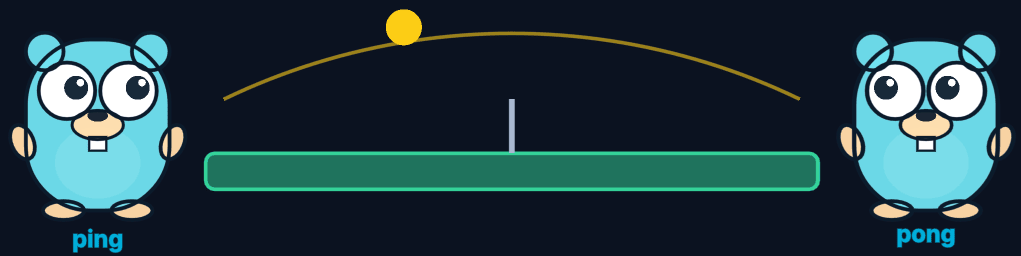
One unbuffered channel can carry values in both directions. Each pass of the ball is a hand-off from one goroutine to the other.

examples/ch05_pingpong/main.go — Go

```go
package main

import "fmt"

func player(name string, table chan int, done chan struct{}) {
    for ball := range table { // receive the ball
        if ball == 6 {
            close(done)
            return
        }
        fmt.Println(name, "hits", ball)
        table <- ball + 1 // send it back
    }
}

func main() {
    table := make(chan int) // unbuffered: every pass is a hand-off
    done := make(chan struct{})
    go player("ping", table, done)
    go player("pong", table, done)
    table <- 1 // serve
    <-done
}
```

output — terminal

```
$ go run .
pong hits 1
ping hits 2
pong hits 3
ping hits 4
pong hits 5
```



1. `range table` receives the ball (blocks until it is sent)
2. `table <- ball + 1` sends it back; the other player is waiting in its range
3. main serves the first ball: the two players start in any order
4. main blocks on `<-done` until a player closes it

THINK ABOUT IT

Who prints first, ping or pong? Whoever's receive the runtime matches with main's serve. Here it was pong; another run may differ.

8.5 fatal error: all goroutines are asleep - deadlock!

If every goroutine is blocked and nothing (a timer, the network) can wake one, the runtime stops the program and prints every goroutine's stack.

examples/ch05_deadlock/main.go — Go

```go
package main

import "fmt"

func main() {
    ch := make(chan int) // unbuffered
    ch <- 42             // blocks: nobody will ever receive   (1)
    fmt.Println(<-ch)
}
```

output — terminal

```
$ go run .
fatal error: all goroutines are asleep - deadlock!

goroutine 1 [chan send]:
main.main()
    ./main.go:7 +0x36
exit status 2
```

(1) main sends, but no goroutine will ever receive; main is the only goroutine

- `[chan send]` in the trace is the wait reason of chapter 8.3's state machine

HOW THE RUNTIME KNOWS

Whenever a thread goes idle, `checkdead()` counts the threads still able to run Go code. If none is left and no timer is pending, nothing can ever happen again: the runtime throws this fatal error. It cannot be recovered.

> ! **PITFALL**
> Detection is **global**. If any other goroutine is alive (an HTTP server, a ticker, a goroutine in a syscall), a stuck goroutine just hangs silently: a **leak** (8.17).

- ✖ send with no receiver (here)
- ✖ receive with no sender, or from a nil channel
- ✖ `wg.Wait()` with a counter that never reaches 0
- ✖ two goroutines each waiting for the other (8.17)

8.5 Channel directions: chan<- and <-chan

A function can promise to only send, or only receive. The compiler enforces it, and the signature documents who owns which end.

`chan T` — **bidirectional** — ✓ send ✓ recv

send and receive

`chan<- T` — **send-only** — ✓ send ✗ recv

the arrow points INTO chan: data goes in

`<-chan T` — **receive-only** — ✗ send ✓ recv

the arrow comes OUT of chan: data comes out

NOTE
A `chan T` converts **implicitly** to either direction (as in `go producer(ch)`). The reverse conversion is not allowed.

examples/ch05_direction_error/main.go — Go

```go
package main

func producer(out chan<- int) { // send-only
    out <- 1
    _ = <-out // compile error: receive from send-only channel
}

func consumer(in <-chan int) { // receive-only
    in <- 2 // compile error: send to receive-only channel
}

func main() {
    ch := make(chan int)
    go producer(ch) // chan int converts to chan<- int
    consumer(ch)    // chan int converts to <-chan int
}
```

1 2

the compiler says no — terminal

```
$ go build .
# gozm/module08/ch05_direction_error
./main.go:5:8: invalid operation: cannot receive from send-only channel chan<- int out
    (variable of type chan<- int)
./main.go:9:2: invalid operation: cannot send to receive-only channel <-chan int in
    (variable of type <-chan int)
```

PRO TIP Return `<-chan T` from producers (8.15): callers cannot send or close by mistake.

8.5 Channels are values: a channel of replies

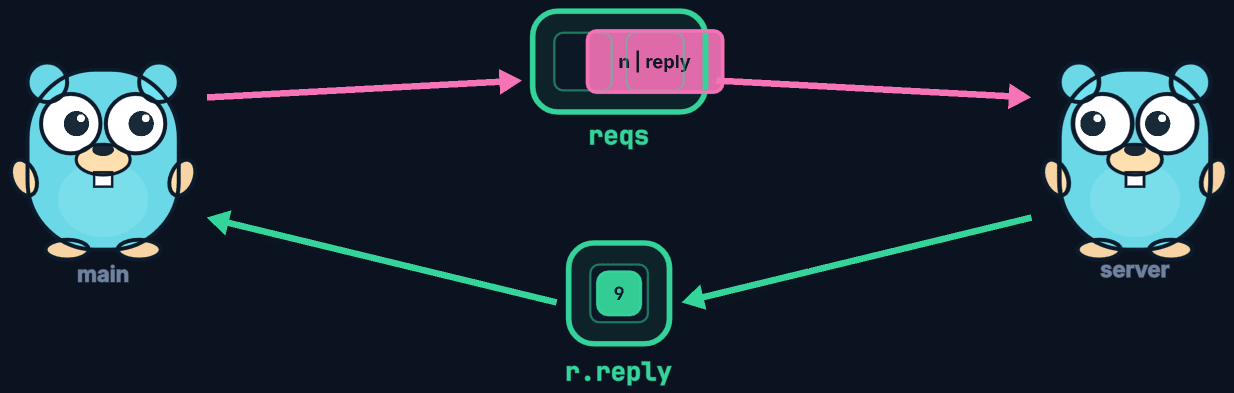
A channel can travel inside a struct or through another channel. Sending a request with its own reply channel gives you a tiny server.

examples/ch05_reply/main.go — Go

```go
type request struct {
    n     int
    reply chan int // where to send the answer
}

// server owns its work; clients reach it only through reqs.
func server(reqs <-chan request) {
    for r := range reqs {
        r.reply <- r.n * r.n
    }
}

func main() {
    reqs := make(chan request)
    go server(reqs)
    for _, n := range []int{3, 4, 5} {
        r := request{n, make(chan int)}
        reqs <- r                              // ask...
        fmt.Println(n, "squared is", <-r.reply) // ...and wait
    }
    close(reqs)
}
```

output — terminal

```
$ go run .
3 squared is 9
4 squared is 16
5 squared is 25
```



1. the request carries **its own reply channel**
2. the server answers on exactly that channel
3. send the request...
4. ...then block until the answer arrives

KEY IDEA

The server's state is touched by **one goroutine only**: no locks are needed. This is the "share memory by communicating" idea from chapter 8.1, in code.

8.5 Signals: chan struct{} and the close broadcast

Sometimes the message is the event itself. `struct{}` takes zero bytes, and closing a channel wakes every receiver at once.

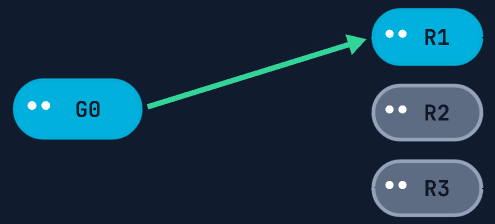
examples/ch05_signal/main.go — Go

```go
func main() {
    var empty struct{}
    fmt.Println("size of struct{}:", unsafe.Sizeof(empty), "bytes")

    ready := make(chan struct{}) // a pure signal: no data
    var config string
    go func() {
        time.Sleep(20 * time.Millisecond)
        config = "loaded" // (1) write
        close(ready)      // (2) signal: wakes EVERY receiver
    }()
    for i := 1; i <= 3; i++ {
        go func() {
            <-ready // (3) all three wait here
            fmt.Println("reader", i, "sees config =", config)
        }()
    }
    time.Sleep(50 * time.Millisecond)
}
```

output — terminal

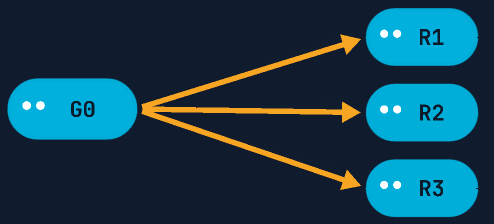
```
$ go run .
size of struct{}: 0 bytes
reader 2 sees config = loaded
reader 1 sees config = loaded
reader 3 sees config = loaded
```

send: wakes ONE receiver



`ch <- struct{}{}` is matched with exactly one waiting receiver.

close: wakes ALL receivers



`close(ch)` makes every current and future receive return at once.

1. the writer stores the data **before** signalling
2. `close(ready)` is synchronised before every receive that sees it closed
3. so each reader is guaranteed to see `config == "loaded"`

§ **RULE**

Go memory model: **a send happens before the matching receive completes**, and **a close happens before a receive that returns because the channel is closed.** Chapter 8.13 builds on these rules.

8.5 Recap and check yourself

Channels carry values and synchronise the goroutines at both ends.

REMEMBER

- ✓ `make(chan T)` is unbuffered: a send waits for a receiver and vice versa (a rendezvous).
- ✓ The arrow `<-` shows the data flow. `v, ok := <-ch` tells you whether the channel is closed.
- ✓ All goroutines blocked = fatal deadlock; one goroutine blocked while others run = a silent leak.
- ✓ `chan<- T` is send-only, `<-chan T` is receive-only; `chan T` converts to either.
- ✓ Channels are values: put reply channels in requests, return `<-chan T` from producers.
- ✓ `chan struct{}` + `close` is a zero-byte broadcast signal with a memory-model guarantee.

CHECK YOURSELF

Q1. `ch := make(chan int); ch <- 1; fmt.Println(<-ch)` in main. What happens?

Answer. Deadlock: the send blocks forever because main itself is the only possible receiver.

Q2. What does a receive from a nil channel do?

Answer. It blocks forever (useful in select, 8.8).

Q3. Can a function with parameter `in <-chan int` close `in`?

Answer. No: closing a receive-only channel is a compile error. Only the sending side closes.

Q4. Why is `chan struct{}` preferred over `chan bool` for signals?

Answer. It costs zero bytes per element and says clearly: the event matters, not a value.

Don't wait for me... unless you're unbuffered.

CHAPTER

8.6

Buffers, close and range

A buffer lets a sender run ahead of its receiver. `close` says that no more values are coming. Together they shape the life of every channel.

IN THIS CHAPTER YOU WILL

- ✓ Use buffered channels as bounded FIFO queues
- ✓ Choose between unbuffered, size 1 and size N, with data
- ✓ Close channels, drain them, and loop with `range`
- ✓ Recite the channel axioms, verified by experiment
- ✓ Decide who closes a channel when there are many senders

? THE BIG QUESTION

A channel has a buffer of 3 and nobody is receiving. What happens on the fourth send?

full: the 4th send waits

the receiver is busy elsewhere

8.6 Buffered channels: a queue with a capacity

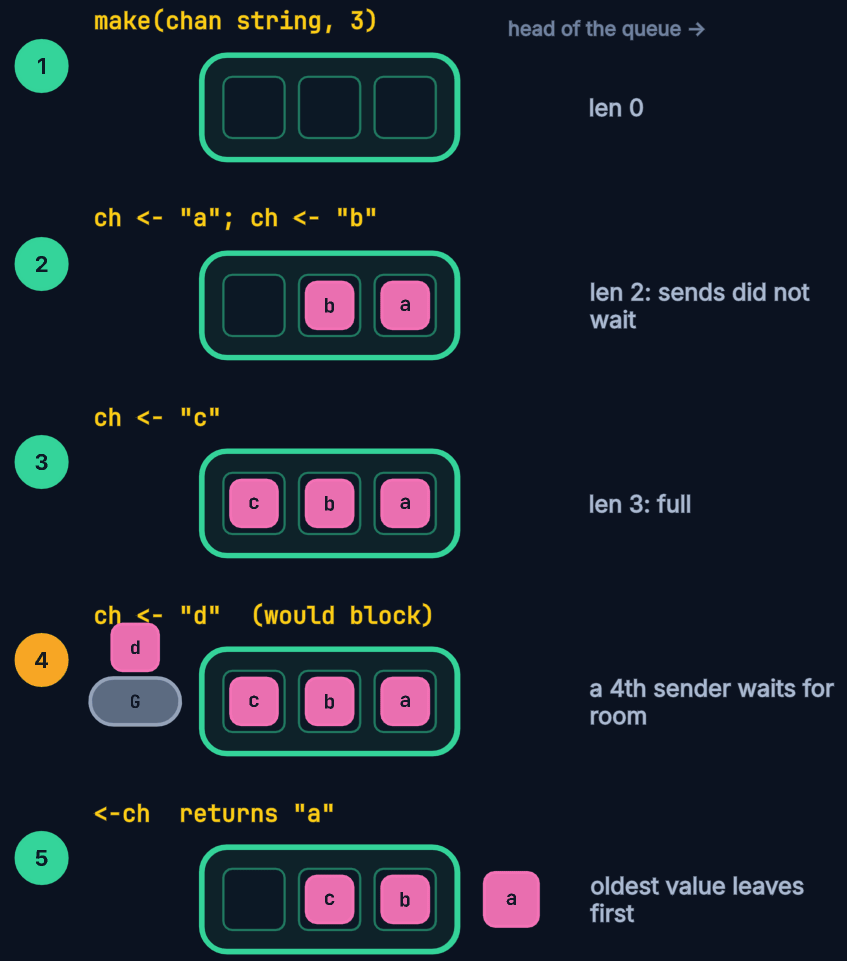
`make(chan T, n)` holds up to n values. A send blocks only when the buffer is full; a receive blocks only when it is empty. Order is FIFO.

examples/ch06_buffered/main.go — Go

```go
func main() {
    ch := make(chan string, 3) // capacity 3
    fmt.Printf("empty:  len=%d cap=%d\n", len(ch), cap(ch))

    ch <- "a"
    ch <- "b" // no receiver needed: there is room
    fmt.Printf("2 sent: len=%d cap=%d\n", len(ch), cap(ch))

    ch <- "c"
    fmt.Printf("full:   len=%d cap=%d (a 4th send would block)\n", len(ch), cap(ch))

    fmt.Println("recv:", <-ch, "(FIFO: first in, first out)")
    fmt.Printf("now:    len=%d cap=%d\n", len(ch), cap(ch))
}
```

output — terminal

```
$ go run .
empty:  len=0 cap=3
2 sent: len=2 cap=3
full:   len=3 cap=3 (a 4th send would block)
recv: a (FIFO: first in, first out)
now:    len=2 cap=3
```



8.6 Unbuffered, size 1 or size N?

A buffer changes timing and throughput, never correctness. Choose the size for a reason you can write down.

Capacity	What it guarantees	Typical use
0 (unbuffered)	the receiver **has** the value when the send returns	hand-off, request/reply, strict lock-step
1	one value can wait; the sender never blocks once	a single result from a goroutine that may be abandoned (8.8 timeout)
N (known)	exactly N sends never block	N workers each send one result; a semaphore of N slots (8.15)
N (large)	absorbs bursts while the consumer catches up	smoothing a bursty producer; batching

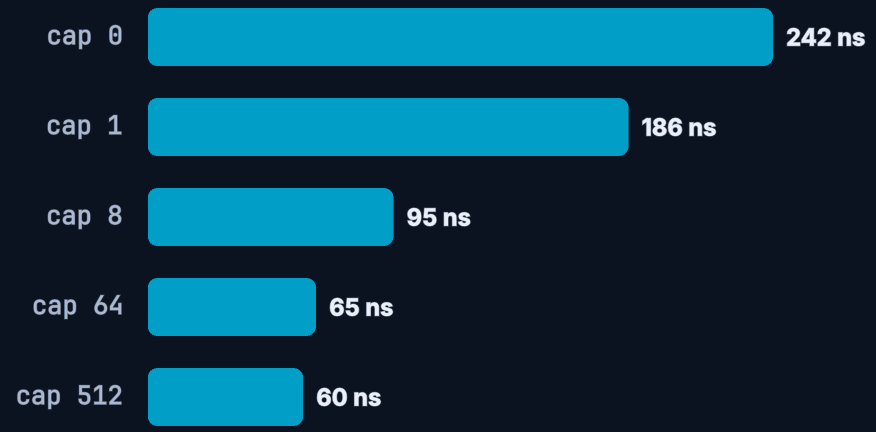
PITFALL

A buffer does not fix a deadlock, it **postpones** it: if the receiver never comes, the (N+1)th send blocks forever just like the first one would have.

PRO TIP

Unbuffered is the default to reach for. Add a buffer when you can say **why**, and size it from the design ("one slot per worker"), not from a guess like 100.

COST PER VALUE (measured)



One producer sends 2,000,000 ints to one consumer (best of 3 runs, `examples/ch06_bufbench`).

- Unbuffered: every value needs both goroutines to meet, so they wake each other constantly.
- A small buffer lets the producer run ahead and the consumer drain in batches.
- Beyond ~64 slots it stops helping: the per-value cost of the lock and the copy remains.

8.6 close: no more values are coming

After `close`, receivers still get every buffered value, then the zero value with ok == false. A `range` loop ends at exactly that point.

examples/ch06_close_range/main.go — Go

```go
func main() {
    ch := make(chan int, 3)
    go func() {
        for i := 1; i <= 3; i++ {
            ch <- i * i
        }
        close(ch) // "no more values" - only the sender closes   (1)
    }()

    for v := range ch { // loops until ch is closed AND drained   (2)
        fmt.Println("got", v)
    }

    v, ok := <-ch // receive from a closed, empty channel   (3)
    fmt.Printf("after close: v=%d ok=%v\n", v, ok)
}
```

output — terminal

```
$ go run .
got 1
got 4
got 9
after close: v=0 ok=false
```

```go
for v := range ch { use(v) }     // is shorthand for:
for { v, ok := <-ch; if !ok { break }; use(v) }
```

before close

receive gets 1

`v=1 ok=true`

closed, 2 left

receive still gets 4, then 9

`v=4 ok=true`

closed and empty

every receive returns at once

`v=0 ok=false`

1. the **sender** closes, after its last send
2. `range` keeps receiving until closed **and** drained
3. after that, every receive succeeds instantly with ok == false

> **NOTE**
> Closing is a **signal**, not cleanup. An unreachable channel is garbage collected whether or not it was closed; close only when receivers need to know the stream has ended.

8.6 The channel axioms, verified by experiment

examples/ch06_axioms tries every operation on every kind of channel (buffer size 1) and records whether it blocks, succeeds or panics.

channel state	send `ch <- 1`	receive `v, ok := <-ch`	close `close(ch)`
nil	⏸ blocks forever	⏸ blocks forever	! panic: close of nil channel
open, empty	✓ ok	⏸ blocks (until a partner comes)	✓ ok
open, full	⏸ blocks (until a partner comes)	✓ `v=7 ok=true`	✓ ok
closed, empty	! panic: send on closed channel	✓ `v=0 ok=false`	! panic: close of closed channel
closed, 1 left	! panic: send on closed channel	✓ `v=7 ok=true`	! panic: close of closed channel

✓ completes ⏸ blocks ! panics

★ **KEY IDEA** Learn the three panics: **close(nil)**, **close twice**, **send after close**. And the two forever-blocks on a **nil** channel, which `select` turns into a feature (8.8).

8.6 Why closing can panic

A panic here is the runtime telling you that two goroutines disagree about who owns the channel.

examples/ch06_send_closed/main.go — Go

```go
package main

func main() {
    ch := make(chan int, 1)
    close(ch)
    ch <- 1 // panic: send on closed channel
}
```

output — terminal

```
$ go run .
panic: send on closed channel

goroutine 1 [running]:
main.main()
    ./main.go:6 +0x3a
exit status 2
```

examples/ch06_close_closed/main.go — Go

```go
package main

func main() {
    ch := make(chan int)
    close(ch)
    close(ch) // panic: close of closed channel
}
```

output — terminal

```
$ go run .
panic: close of closed channel

goroutine 1 [running]:
main.main()
    ./main.go:6 +0x30
exit status 2
```

- ✖ A **send after close** means a sender did not know the stream had ended: some other goroutine closed it.
- ✖ A **double close** means two goroutines both believed they owned the channel.
- ✖ There is no `isClosed(ch)` function on purpose: by the time it returned, the answer could be stale.
- ✖ Recovering from these panics hides the real bug. Fix the ownership instead (next page).

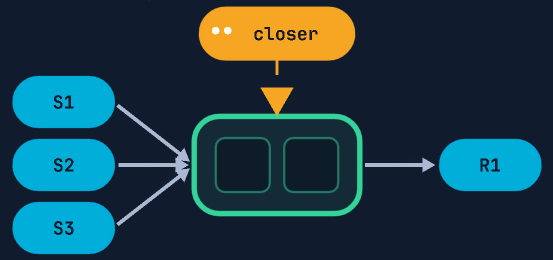
8.6 Who closes the channel?

The rule: **the goroutine that sends closes**, and only when it is sure nobody else will send. Three topologies, three answers.

1 sender → many receivers

The sender closes after its last send. Every receiver's `range` ends.

many senders → 1 receiver



No sender may close alone. A **closer** goroutine waits for all of them (WaitGroup), then closes.

many senders → many receivers

Do not close the data channel at all. Stop everyone with a separate signal: close a `done` channel or cancel a `context`.

examples/ch06_multi_sender: the closer pattern — Go

```go
    for id := 1; id <= 3; id++ { // three senders
        wg.Go(func() {
            results <- fmt.Sprintf("worker %d", id)
        })
    }

    go func() { // closer: wait for ALL, then close
        wg.Wait()
        close(results)
    }()
```

output — terminal

```
$ go run .
received from worker 1
received from worker 2
received from worker 3
total: 3
```

§ **RULE**

Close from the sending side, exactly once, after the last send. When ownership is unclear, signal with a separate channel instead.

8.6 Recap and check yourself

Buffers decouple timing; close ends a stream; range consumes it.

REMEMBER

- A buffered channel is a bounded FIFO: send blocks when full, receive blocks when empty.
- Buffers change speed, not correctness: 242 ns per value unbuffered vs 65 ns with 64 slots here.
- After `close`, buffered values are still delivered; then receives return zero, false.
- `for v := range ch` stops when the channel is closed and drained.
- Panics: close(nil), close twice, send after close. A nil channel blocks forever.
- The sender closes. Many senders: a closer goroutine. Many to many: a separate done signal.

CHECK YOURSELF

Q1. `ch := make(chan int, 2); ch <- 1; ch <- 2; close(ch)`. What do three receives return?
Answer. 1 true, 2 true, then 0 false.

Q2. Is `len(ch) == 0` a safe way to know a channel is drained?
Answer. Only as a hint: another goroutine may send right after you look.

Q3. Five workers send results on `results`. Which goroutine should close it?
Answer. None of the workers: a closer goroutine that waits for all five (WaitGroup), then closes.

Q4. Your program deadlocks; adding a buffer of 100 "fixes" it. Is it fixed?
Answer. No. It deadlocks on the 101st value instead. Find the missing receiver.

CHAPTER

8.7

Inside a Channel

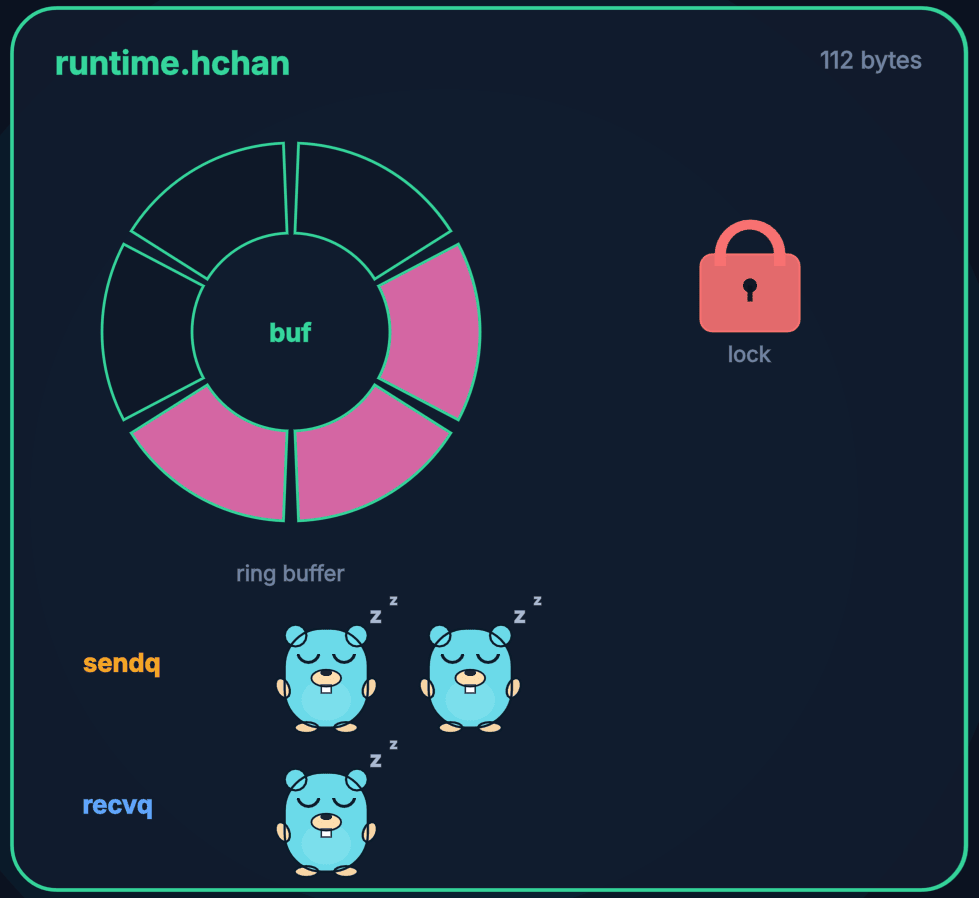
Open the runtime: a channel is a 112-byte struct holding a lock, a ring buffer and two queues of parked goroutines.

IN THIS CHAPTER YOU WILL

- Map every field of `runtime.hchan`
- Follow `chansend` and `chanrecv` case by case
- Explain sudogs, `gopark` and `goready`
- Trace a buffered channel through a full storyboard
- See why a direct send writes into another goroutine's stack

? THE BIG QUESTION

A sender finds a receiver already waiting. Why does the value skip the buffer entirely?



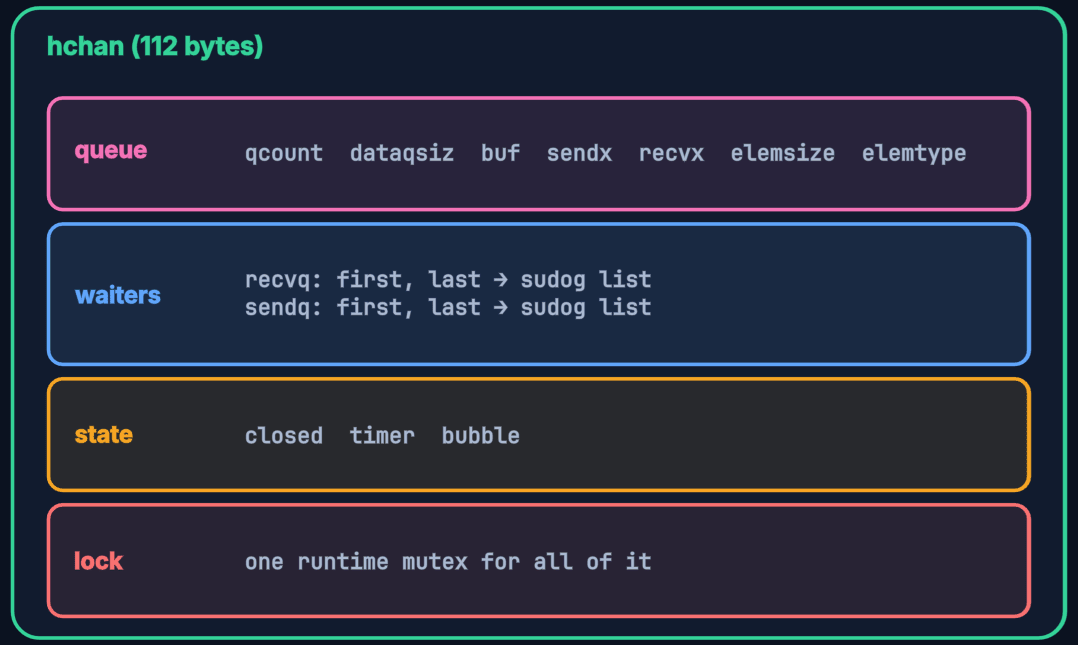
8.7 runtime.hchan, field by field

`make(chan T, n)` returns a pointer to this struct. Everything a channel does is a few operations on these fields, under one lock.

runtime/chan.go (Go 1.27, comments shortened) — Go

```go
type hchan struct {
    qcount   uint           // values now in the buffer
    dataqsiz uint           // buffer capacity
    buf      unsafe.Pointer // the ring buffer
    elemsize uint16
    closed   uint32
    timer    *timer         // set for time.Timer channels
    elemtype *_type
    sendx    uint           // next slot to write
    recvx    uint           // next slot to read
    recvq    waitq          // parked receivers
    sendq    waitq          // parked senders
    bubble   *synctestBubble
    lock     mutex          // protects all of the above
}

type waitq struct {
    first *sudog
    last  *sudog
}
```

Sizes read from the DWARF debug info of a Go 1.27.3 binary: hchan 112 B, sudog 104 B, waitq 16 B.

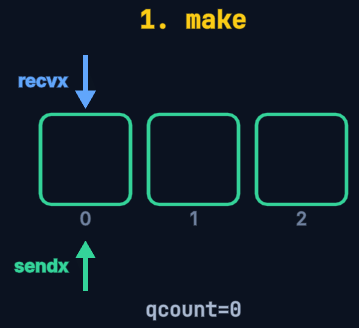


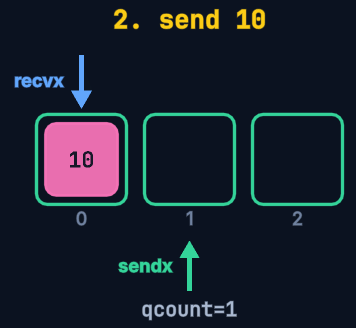
WHERE make PUTS THE BUFFER

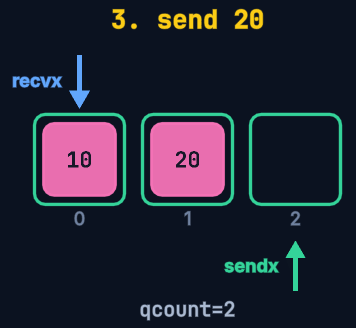
Element type	Allocation
size 0 (e.g. `struct{}`) or cap 0	just the 112-byte hchan
no pointers (int, [8]byte...)	hchan and buffer in **one** block
contains pointers	hchan + a separate, GC-scanned buffer

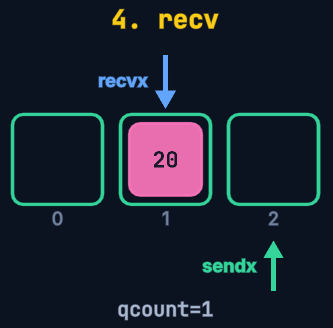
8.7 The ring buffer, observed live

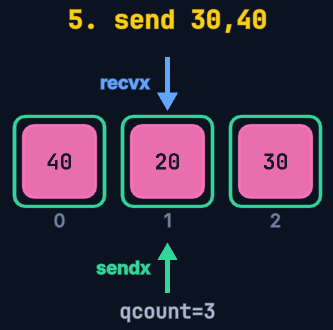
examples/ch07_peek reads the real hchan fields with unsafe after each operation. Watch sendx and recvx chase each other around a 3-slot ring.

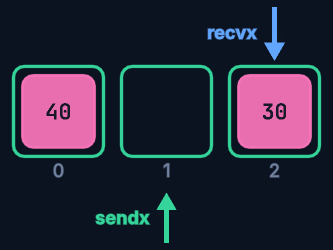


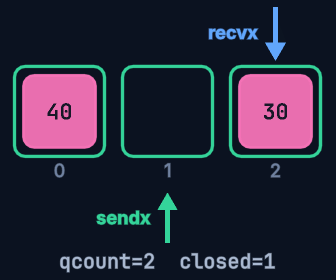












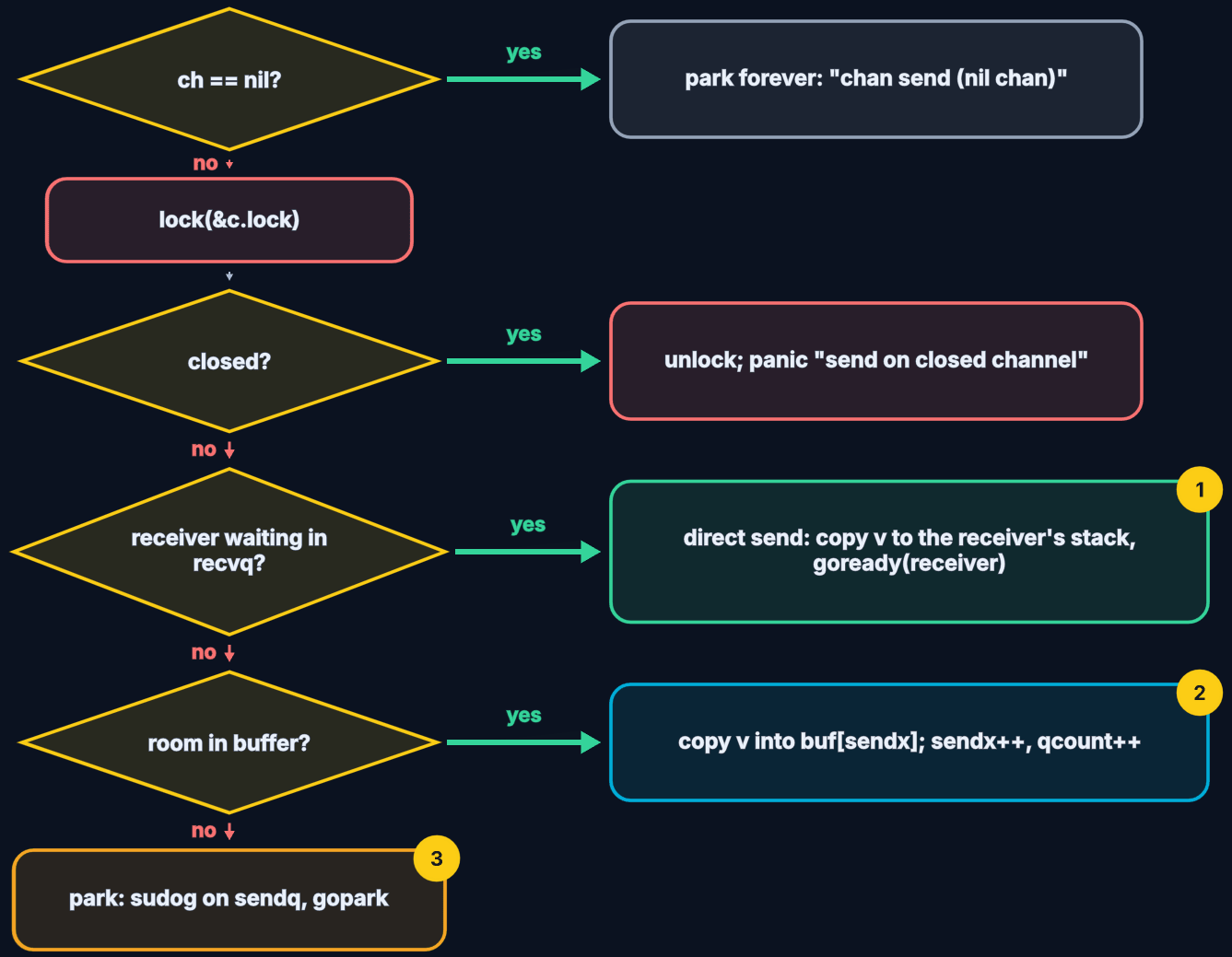
- **sendx** is where the next send writes; **recvx** where the next receive reads.
- Both wrap to 0 at the end: frame 5 wrote 40 into slot 0.
- A receive clears its slot (`typedmemclr`), so the GC never sees stale pointers.
- Grey values are dead slots: the runtime ignores them.

examples/ch07_peek output (uses unsafe: for learning only) — terminal

```
$ go run .
make       qcount=0 sendx=0 recvx=0 buf=[0 0 0] closed=0
send 10    qcount=1 sendx=1 recvx=0 buf=[10 0 0] closed=0
send 20    qcount=2 sendx=2 recvx=0 buf=[10 20 0] closed=0
recv       qcount=1 sendx=2 recvx=1 buf=[0 20 0] closed=0
send 30,40 qcount=3 sendx=1 recvx=1 buf=[40 20 30] closed=0
recv       qcount=2 sendx=1 recvx=2 buf=[40 0 30] closed=0
close      qcount=2 sendx=1 recvx=2 buf=[40 0 30] closed=1
```

8.7 chansend: four cases, in order

What `ch <- v` compiles to: a call to runtime.chansend1, which checks these cases in this exact order (runtime/chan.go).



1. **fastest**: no buffer write at all; one copy, straight into the waiting goroutine's variable
2. the common buffered case: a copy and two counter bumps
3. slow path: the sender sleeps until a receiver takes its value, or the channel closes

UNDER THE HOOD

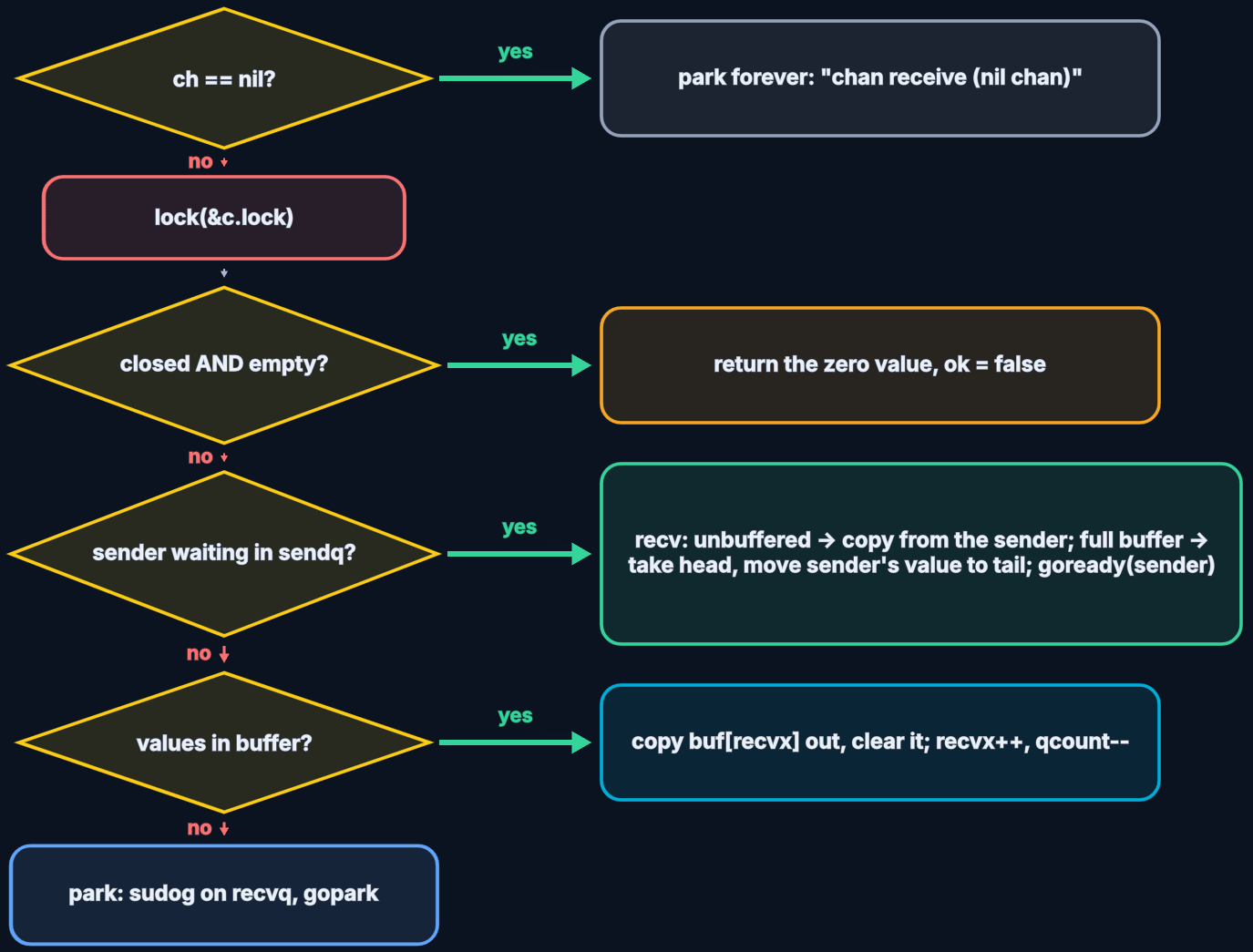
When the parked sender wakes, it checks `sudog.success`. If it was woken by `close` instead of a receiver, it panics with "send on closed channel" right there.

NOTE

A `select` with `default` first does a **lock-free** check: open and full means "would block", so it returns false without touching the lock.

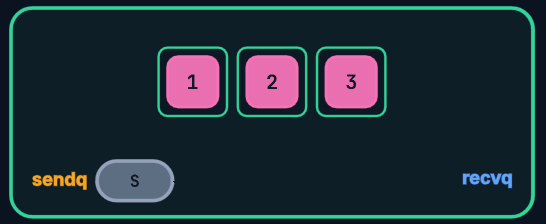
8.7 chanrecv: the mirror image, with one twist

`v, ok := <-ch` calls runtime.chanrecv2. The twist: when a sender is parked on a full buffer, the receiver takes the head and moves the sender's value to the tail.



THE TWIST, DRAWN

before: full, sender S parked with 4



after one receive

got 1

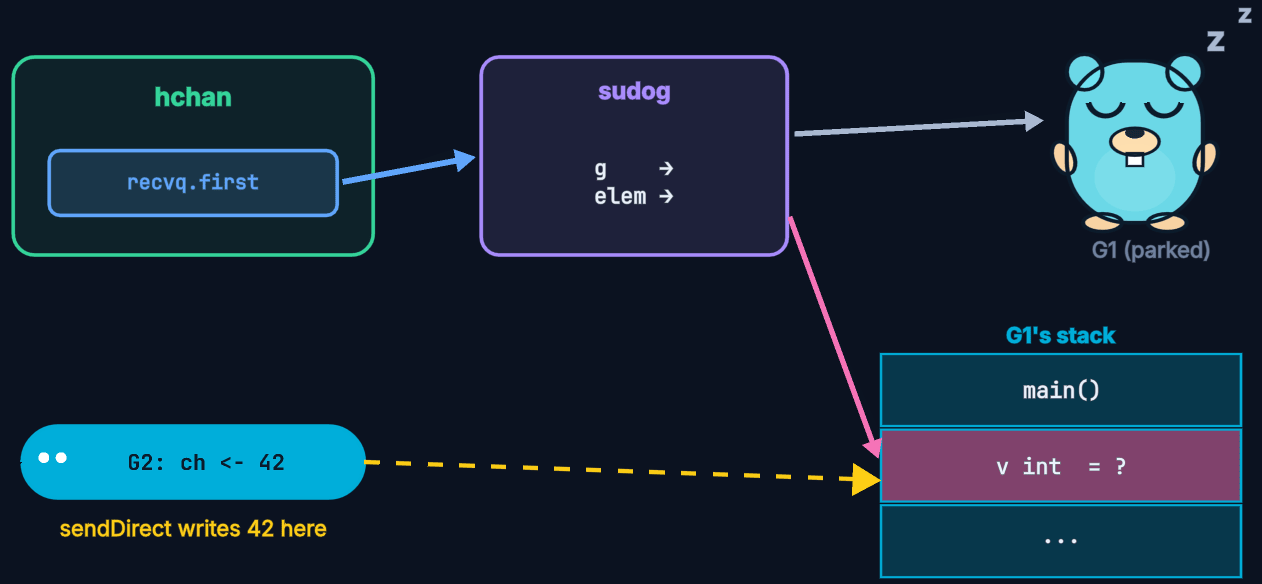
Slot 0 held the oldest value (1), which goes to the receiver. Since the buffer was full, slot 0 is also where the next send would write, so S's value 4 goes there directly and S is woken. One copy in, one copy out, FIFO kept.

8.7 Parking: sudog, gopark and goready

A goroutine that must wait wraps itself in a sudog (104 bytes), joins the channel's queue and parks. The partner that arrives later wakes it up.

runtime/runtime2.go (abridged) — Go

```
type sudog struct {
    g     *g                 // the parked goroutine
    next *sudog              // doubly linked
    prev *sudog              //   wait queue
    elem maybeTraceablePtr   // its variable: often ON ITS STACK
    isSelect bool            // part of a select? (8.8)
    success  bool            // woken by a partner, not by close
    c     maybeTraceableChan // which channel
    // + times, ticket, waitlink, parent, ...
}
```



1. **G1 receives** `v := <-ch` finds the buffer empty: G1 fills a sudog with `elem = &v`, enqueues it on `recvq` and calls `gopark`. G1 is now _Gwaiting.
2. **G2 sends** `ch <- 42`: chansend finds G1's sudog on `recvq` and dequeues it.
3. **sendDirect** copies 42 straight into G1's `v`, **on G1's stack**. The buffer is never used.
4. **goready(G1)** G1 becomes runnable, usually in G2's P's `runnext` slot, so it runs very soon.

UNDER THE HOOD

runtime/chan.go notes that `sendDirect` is **the only place in the runtime where one goroutine writes to the stack of another**. It is safe because G1 is parked and the copy happens while holding the channel lock, with no preemption point in between.

8.7 Storyboard: a fast producer meets a slow consumer

`ch := make(chan int, 2)`. P sends 1, 2, 3 then closes; C receives slowly. Each panel shows the hchan and both goroutines.

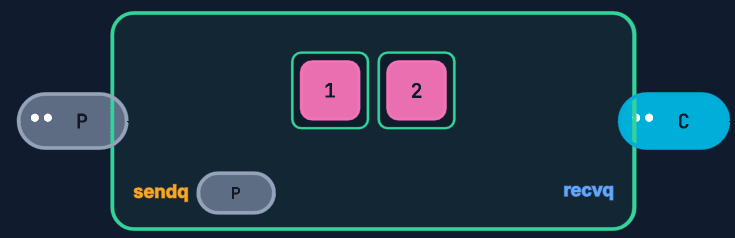
1. make(chan int, 2)

empty buffer, no waiters

2. P: ch <- 1; ch <- 2

buffered; P never blocked

3. P: ch <- 3



full: P parks on sendq with its 3

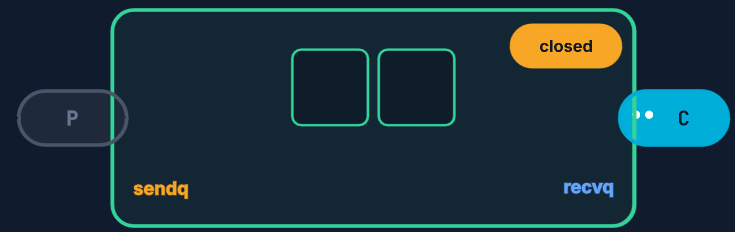
4. C: <-ch gets 1

head 1 out; P's 3 moved into slot 0; P woken

5. C: <-ch, <-ch get 2, 3

buffer empty again (FIFO kept)

6. P: close(ch); C: <-ch



closed + empty: C gets 0, ok == false

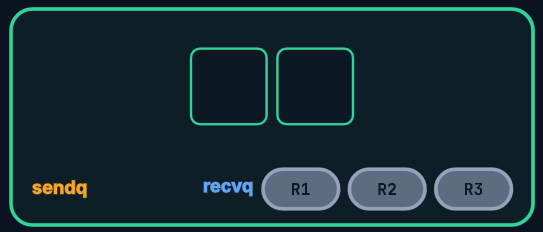
★ **KEY IDEA** Every panel is a handful of field updates under `c.lock`. The scheduler only gets involved when somebody parks (panel 3) or is woken (panel 4).

8.7 closechan: wake everybody, then step back

`close(ch)` flips one flag and releases every parked goroutine. Receivers get zero values; senders will panic when they run.

1. **Check** `ch == nil` → panic "close of nil channel"; lock; already closed → unlock, panic "close of closed channel".
2. **Mark** `c.closed = 1`.
3. **Release receivers** for each sudog on `recvq`: zero its `elem`, `success = false`, collect its G.
4. **Release senders** for each sudog on `sendq`: `success = false`, collect its G (it will panic when it runs).
5. **Unlock, then ready** `unlock(&c.lock)`, then `goready` every collected G, outside the lock.

before close: three receivers parked on an empty channel



after close: queues empty, all three runnable

KEY IDEA

This is why `close` works as a **broadcast** (chapter 8.5): one call wakes any number of waiters, and every later receive returns immediately.

THINK ABOUT IT

Why goready **after** unlocking? Readying a goroutine can involve the scheduler and other locks. Holding the channel lock across that could deadlock with stack shrinking; the source says so.

8.7 Recap and check yourself

A channel is ordinary data plus a lock. Its cleverness is in the order of the checks.

REMEMBER

- ✓ `hchan` = ring buffer (buf, sendx, recvx, qcount) + recvq/sendq of sudogs + closed + one lock.
- ✓ Send order: nil → closed (panic) → waiting receiver (direct) → buffer room → park.
- ✓ Receive order: nil → closed and empty (zero, false) → waiting sender → buffer → park.
- ✓ A direct send copies into the receiver's stack: the only cross-stack write in the runtime.
- ✓ Parked goroutines wait in sudogs; `goready` puts the partner back on a run queue.
- ✓ `close` marks the flag and readies every waiter: receivers get zero values, senders panic.

CHECK YOURSELF

Q1. Can a channel have values in its buffer and receivers parked on recvq at the same time?
Answer. No. The runtime keeps the invariant: qcount > 0 implies recvq is empty.

Q2. Why is an unbuffered hand-off often faster than going through a buffer?
Answer. One copy straight to the receiver, instead of a copy in and a copy out.

Q3. A sender is parked when the channel is closed. What happens?
Answer. closechan readies it with success = false; when it runs it panics "send on closed channel".

Q4. How much memory does `make(chan [64]byte, 4)` request?
Answer. One block: 112 + 4 × 64 = 368 bytes, rounded up to the allocator's 384-byte size class.

CHAPTER

8.8

select and Timers

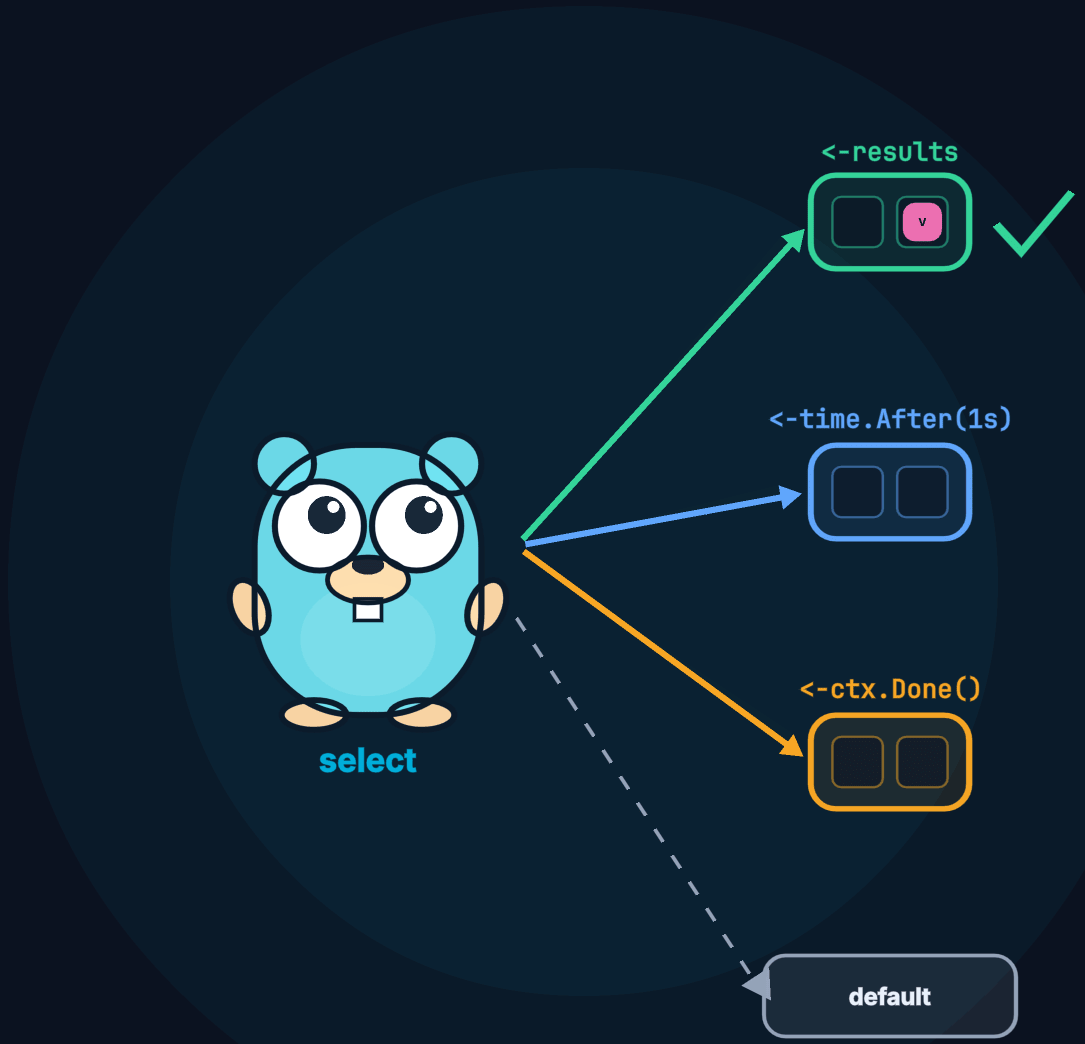
`select` waits on several channel operations at once and proceeds with whichever is ready first. It is how Go programs do timeouts, cancellation and non-blocking I/O.

IN THIS CHAPTER YOU WILL

- State the six rules of `select`
- Add timeouts, and try-send / try-receive with `default`
- Disable cases with nil channels
- Measure select's fairness and add priorities yourself
- Explain selectgo's three passes, and Go 1.23's timers

? THE BIG QUESTION

Two channels are ready at the same instant. Which case does select pick, and may you rely on it?



8.8 select: proceed with the first ready operation

A select statement lists channel sends and receives. It runs exactly one of them, and blocks only if none can proceed.

the shape of select — Go

```
select {
case v := <-in:              // a receive
    use(v)
case out <- result:          // a send
    sent++
case <-time.After(time.Second):
    timeout()                // a receive from a timer
default:                     // optional: never block
    idle()
}
```

1. **Evaluate** every channel expression and every value to send, once, top to bottom, before anything else.
2. **Choose** if one or more cases can proceed, pick **one at random**, uniformly.
3. **Default** if none can proceed and there is a `default`, run it immediately.
4. **Block** otherwise wait until any case can proceed, then run that one.
5. **Nil** a case on a nil channel can never proceed (it is ignored).
6. **Empty** `select {}` has no cases and blocks forever.

PITFALL

Rule 1 bites: `case out <- expensive():` calls `expensive()` **every time** the select runs, even if another case wins. Likewise `time.After(...)` creates a new timer on each loop iteration.

KEY IDEA

`select` is a **switch for communication**: cases are channel operations, and the order they are written in means nothing.

8.8 Timeouts: race the work against a timer

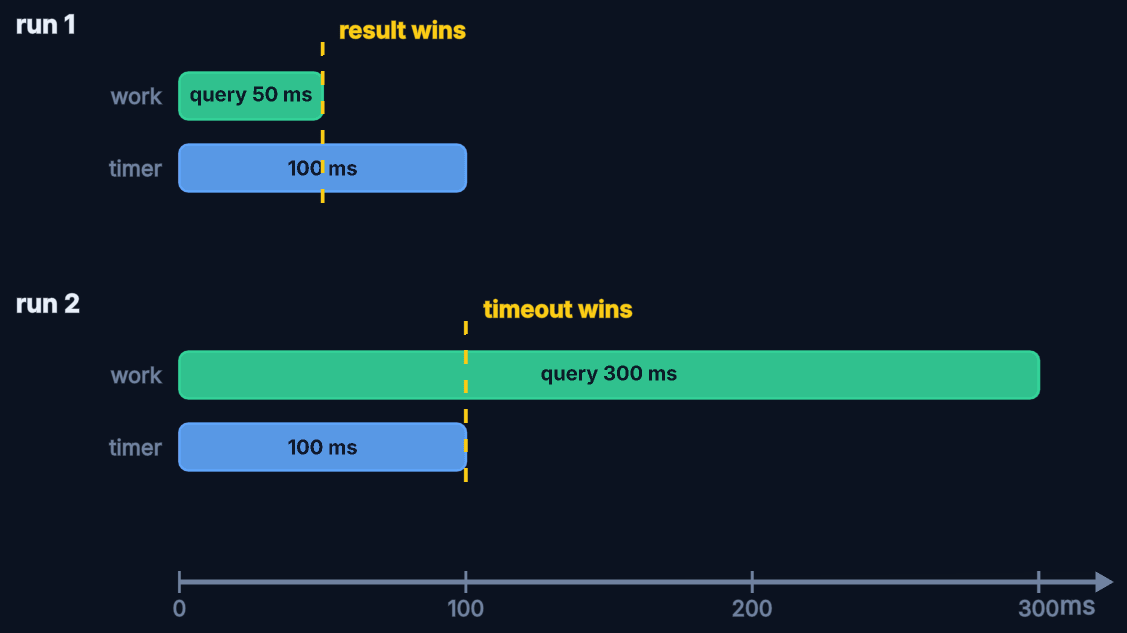
Put the result channel and a timer channel in the same select. Whichever is ready first wins.

examples/ch08_timeout/main.go — Go

```go
func slowQuery(d time.Duration) <-chan string {
    out := make(chan string, 1) // cap 1: the send never blocks
    go func() {
        time.Sleep(d)
        out <- fmt.Sprintf("result after %v", d)
    }()
    return out
}

func main() {
    for _, ms := range []int{50, 300} {
        d := time.Duration(ms) * time.Millisecond
        select {
        case r := <-slowQuery(d):
            fmt.Println("ok:", r)
        case <-time.After(100 * time.Millisecond):
            fmt.Println("timeout: gave up after 100ms")
        }
    }
}
```

output — terminal

```
$ go run .
ok: result after 50ms
timeout: gave up after 100ms
```



1. capacity **1**: if the timeout wins, the late send still succeeds and the goroutine exits (no leak, see 8.17)
2. `slowQuery(d)` runs when the select is evaluated (rule 1)
3. `time.After` returns a channel that receives once, after the duration

PRO TIP In servers use `context.WithTimeout` and `case <-ctx.Done():` instead: the deadline then flows to everything the request calls (8.14).

8.8 Non-blocking operations: select with default

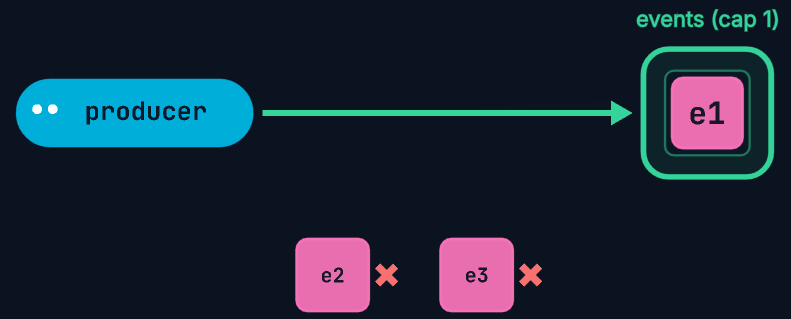
With a `default` case, select never waits. That gives you try-send and try-receive.

examples/ch08_nonblocking/main.go — Go

```go
func main() {
    events := make(chan string, 1)

    for i := 1; i <= 3; i++ {
        select {
        case events <- fmt.Sprintf("event %d", i): // try to send
            fmt.Println("queued  event", i)
        default: // channel full: do not wait
            fmt.Println("dropped event", i)
        }
    }

    select {
    case e := <-events: // try to receive
        fmt.Println("handled", e)
    default:
        fmt.Println("nothing to do")
    }
}
```

output — terminal

```
$ go run .
queued  event 1
dropped event 2
dropped event 3
handled event 1
```



dropped: the channel was full, and default ran

WHEN TO USE IT

- **Try-send**: publish a metric or event, but never stall the hot path.
- **Try-receive**: poll for work while doing something else.
- **Coalesce**: a capacity-1 "something changed" channel; extra signals merge into one.

> **PITFALL**
> A `for` loop around a select with `default` and no blocking case is a **busy loop**: it burns a whole core doing nothing. If you must wait, drop the `default`.

8.8 Switching cases off with nil channels

A receive from a closed channel is always ready, which would spin. Setting the variable to nil disables that case for good.

examples/ch08_merge_nil/main.go — Go

```go
func main() {
    a, b := gen(1, 2, 3), gen(10, 20)
    for a != nil || b != nil { // until BOTH are finished
        select {
        case v, ok := <-a:
            if !ok {
                a = nil // a nil channel is never ready: case disabled
                fmt.Println("a closed")
                continue
            }
            fmt.Println("a:", v)
        case v, ok := <-b:
            if !ok {
                b = nil
                fmt.Println("b closed")
                continue
            }
            fmt.Println("b:", v)
        }
    }
    fmt.Println("merged everything")
}
```

(1) (2)

one possible output — terminal

```
$ go run .
b: 10
b: 20
b closed
a: 1
a: 2
a: 3
a closed
merged everything
```

both open	case <-a	case <-b
b closed → b = nil	case <-a	case <-b
a closed → a = nil: loop ends	case <-a	case <-b

1. loop until **both** inputs are finished
2. a closed channel is always ready; a nil one never is

PRO TIP

The same trick turns a send off: keep `var out chan<- T` nil until you have something to send (the crawler coordinator in 8.19 does exactly this).

8.8 Is select fair? We measured it

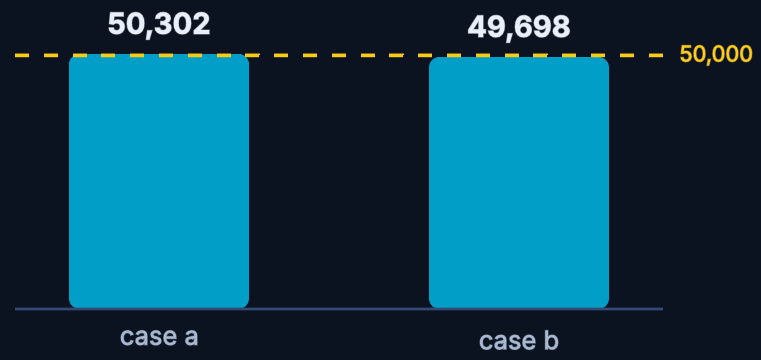
When several cases are ready, the runtime shuffles them before looking. Over 100,000 rounds with two always-ready channels:

examples/ch08_fairness/main.go — Go

```go
func main() {
    a := make(chan int, 1)
    b := make(chan int, 1)
    counts := map[string]int{}
    for range 100_000 {
        a <- 1 // both cases are ready every time
        b <- 1
        select {
        case <-a:
            counts["a"]++
            <-b
        case <-b:
            counts["b"]++
            <-a
        }
    }
    fmt.Println("case a chosen:", counts["a"])
    fmt.Println("case b chosen:", counts["b"])
}
```

output — terminal

```
$ go run .
case a chosen: 50302
case b chosen: 49698
```



difference: 0.60%

there is no priority; build one — Go

```go
// priority: drain "urgent" before anything else
for {
    select {
    case u := <-urgent:
        handle(u)
        continue
    default:
    }
    select {
    case u := <-urgent:
        handle(u)
    case n := <-normal:
        handle(n)
    }
}
```

8.8 Under the hood: selectgo in three passes

The compiler rewrites simple selects; the general case calls runtime.selectgo (runtime/select.go), which works like this.

compiler: 0 cases → block forever; 1 case → a plain send/receive; 1 case + default → selectnbsend / selectnbrecv

Shuffle	build `pollorder`: a random permutation of the non-nil cases (cheaprandn)
Sort	build `lockorder`: cases sorted by channel **address** (heap sort)
Lock	lock every channel once, in lockorder
Pass 1	walk pollorder; the first case that can proceed runs now; unlock all, return
Default?	if the select has a default: unlock all and run it
Pass 2	enqueue a sudog (isSelect = true) on **every** channel, then gopark (unlocks all)
Woken	some channel's partner picked one of our sudogs and readied us
Pass 3	lock all again and remove our sudogs from the other channels' queues

PASS 2, DRAWN

- The first channel to fire dequeues **its** sudog and wins; it marks the select as done so the others cannot also fire.
- Address order makes two selects on the same channels lock them in the same order: no lock-ordering deadlock.
- Cost grows with the number of cases: a select with 2 cases is cheap, one with 50 is not.

8.8 Timers and tickers

Timers deliver one value after a delay; tickers deliver one per period. Since Go 1.23 both are simpler to use correctly.

Call	Gives you	Use for
`time.After(d)`	`<-chan Time`, fires once	one-off timeouts in a select
`time.NewTimer(d)`	`*Timer` with `.C`, `Stop`, `Reset`	timeouts you reuse or cancel
`time.NewTicker(p)`	`*Ticker` with `.C`, `Stop`, `Reset`	periodic work, heartbeats
`time.AfterFunc(d, f)`	runs `f` in its own goroutine	callbacks, expiry (8.18 test)

GO VERSION NOTE

Go 1.23 changed timers: (1) a Timer or Ticker that is no longer referenced is garbage collected even if you never called `Stop`; (2) timer channels are now **unbuffered** (`cap(t.C) == 0`), so after `Stop` or `Reset` no stale value can be received. Modules declaring `go 1.22` or older keep the old behaviour.

examples/ch08_timer_reset/main.go — Go

```go
func main() {
    t := time.NewTimer(10 * time.Millisecond)
    fmt.Println("cap(t.C) =", cap(t.C))
    time.Sleep(50 * time.Millisecond) // the timer has expired by now
    t.Reset(200 * time.Millisecond)   // no need to drain t.C first

    start := time.Now()
    <-t.C
    fmt.Println("fired after", time.Since(start).Round(50*time.Millisecond))
}
```

ch08_timer_reset output — terminal

```
$ go run .
cap(t.C) = 0
fired after 200ms
```

examples/ch08_ticker/main.go — Go

```go
    for {
        select {
        case t := <-tick.C:
            ms := t.Sub(start).Round(10 * time.Millisecond)
            fmt.Printf("tick at +%v\n", ms)
        case <-stop:
            fmt.Println("stop")
            return
        }
    }
```

ch08_ticker output — terminal

```
$ go run .
tick at +100ms
tick at +200ms
tick at +300ms
tick at +400ms
stop
```

8.8 The for-select loop: a goroutine's main loop

Most long-lived goroutines are a `for` around a `select`: one case for work, one for periodic chores, one for being told to stop.

examples/ch08_forselect/main.go — Go

```go
func worker(jobs <-chan int, done <-chan struct{}) {
    beat := time.NewTicker(40 * time.Millisecond)
    defer beat.Stop()
    for {
        select {
        case j, ok := <-jobs:                          // 1
            if !ok {
                fmt.Println("jobs closed: exit")
                return
            }
            time.Sleep(25 * time.Millisecond) // do the job
            fmt.Println("did job", j)
        case <-beat.C:                                 // 2
            fmt.Println("heartbeat")
        case <-done:                                   // 3
            fmt.Println("told to stop: exit")
            return
        }
    }
}
```

output — terminal

```
$ go run .
did job 1
did job 2
did job 3
heartbeat
heartbeat
heartbeat
told to stop: exit
```

1. **work**: a job, or `ok == false` when the producer is done
2. **chores**: heartbeats, flushing, metrics, on a ticker
3. **stop**: a done channel or `ctx.Done()`, so the goroutine never leaks

§ **RULE**
Every goroutine you start needs a way to **end**. In a for-select loop that is a `return` in the stop case (and in the closed-input case).

i **NOTE**
A job in progress is not interrupted: `select` only chooses between operations that are ready **when it runs**. Long jobs should check ctx themselves.

8.8 Recap and check yourself

select is Go's multiplexer: the place where timeouts, cancellation and back-pressure meet.

REMEMBER

- ✓ All case expressions are evaluated first; then one ready case runs, chosen at random.
- ✓ `default` makes it non-blocking; no ready case and no default means wait.
- ✓ nil channels never fire: set a variable to nil to switch its case off.
- ✓ Fairness is statistical, not ordered: 50,302 vs 49,698 in our run. Build priorities explicitly.
- ✓ selectgo: shuffle, sort by address, lock all, pass 1, park on all (pass 2), clean up (pass 3).
- ✓ Go 1.23 timers: collected when unreferenced, unbuffered channels, no stale values after Reset.

CHECK YOURSELF

Q1. A select has cases on two closed channels inside a for loop. What goes wrong?

Answer. Both are always ready: the loop spins at full CPU. Set each to nil once it is drained.

Q2. Does case order inside a select give priority?

Answer. No. Ready cases are chosen uniformly at random.

Q3. Why does the timeout example use `make(chan string, 1)`?

Answer. So the late sender can complete its send and exit after the timeout wins; otherwise it leaks.

Q4. What does `select {}` do, and when is it useful?

Answer. Blocks forever; e.g. at the end of main in a program whose work happens in other goroutines.

CHAPTER

8.9

The Scheduler I: G, M and P

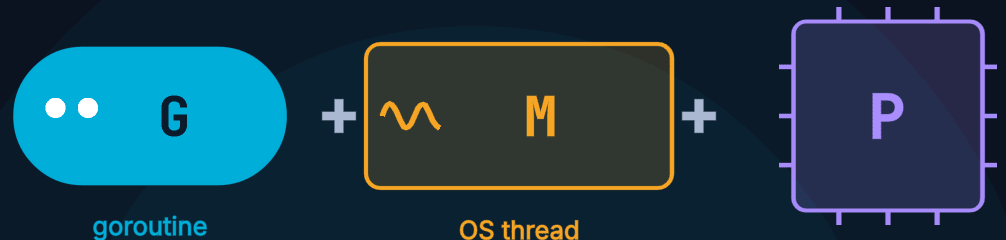
The Go runtime runs its own scheduler inside your process. Three letters explain it: G for goroutine, M for machine (an OS thread), P for processor.

IN THIS CHAPTER YOU WILL

- ✓ Explain why Go schedules goroutines itself
- ✓ Describe G, M and P and the rule that binds them
- ✓ Read the big picture: run queues, idle lists, sysmon
- ✓ Set and reason about GOMAXPROCS, container-aware since Go 1.25
- ✓ Walk one turn of `schedule()` and `findRunnable()`

? THE BIG QUESTION

Your service runs 10,000 goroutines on 4 cores. How many OS threads does it use, and what decides which goroutine runs next?



an M holding a P runs a G

thousands of Gs → a few Ms → GOMAXPROCS Ps

8.9 Why Go schedules goroutines itself

The kernel schedules threads and knows nothing about goroutines. A scheduler inside the runtime can switch in user space and cooperate with channels, the GC and the network.

Cheap switches
no system call, three registers (8.2)

Knows why a G waits
channels park and ready Gs directly (8.7)

Cooperates with the GC
stops Gs at safe points, schedules GC workers

Integrates the network
the netpoller turns I/O readiness into runnable Gs (8.10)

Keeps work local
per-P queues and caches stay warm on one core

A SHORT HISTORY OF THE SCHEDULER

2012 — Go 1.0
G-M model: one global run queue behind one lock. Contention grows with cores.

2013 — Go 1.1
Dmitry Vyukov's design adds **P**: per-processor run queues and work stealing.

2013 — Go 1.2
Preemption at function entry: a long-running G can be stopped at a call.

2015 — Go 1.5
GOMAXPROCS defaults to the number of CPUs (it used to be 1).

2020 — Go 1.14
Asynchronous preemption with signals: even tight loops can be stopped (8.11).

2025 — Go 1.25
Container-aware GOMAXPROCS, re-checked while the program runs.

8.9 Meet G, M and P

Three structs in runtime/runtime2.go. The whole scheduler is the dance between them.

G — goroutine

A function running concurrently: its stack, saved registers and state.

```
stack         [lo, hi)
sched         gobuf (sp, pc, bp)
atomicstatus  _Grunnable ...
waitreason    "chan receive"
m             running on
```

How many? As many as you start: 100,000 is ordinary.

M — machine = OS thread

A real kernel thread. It executes Gs, but only while it holds a P.

```
g0        scheduling stack
curg      the G it runs now
p         the P it holds
spinning  looking for work
lockedg   LockOSThread
```

How many? Usually a few more than GOMAXPROCS; capped at 10,000 by default.

P — processor (a scheduling context)

The right to run Go code, plus the resources for it.

```
runq     [256]guintptr
runnext  run this G next
mcache   allocation cache
timers   this P's timers
gFree    dead Gs to reuse
```

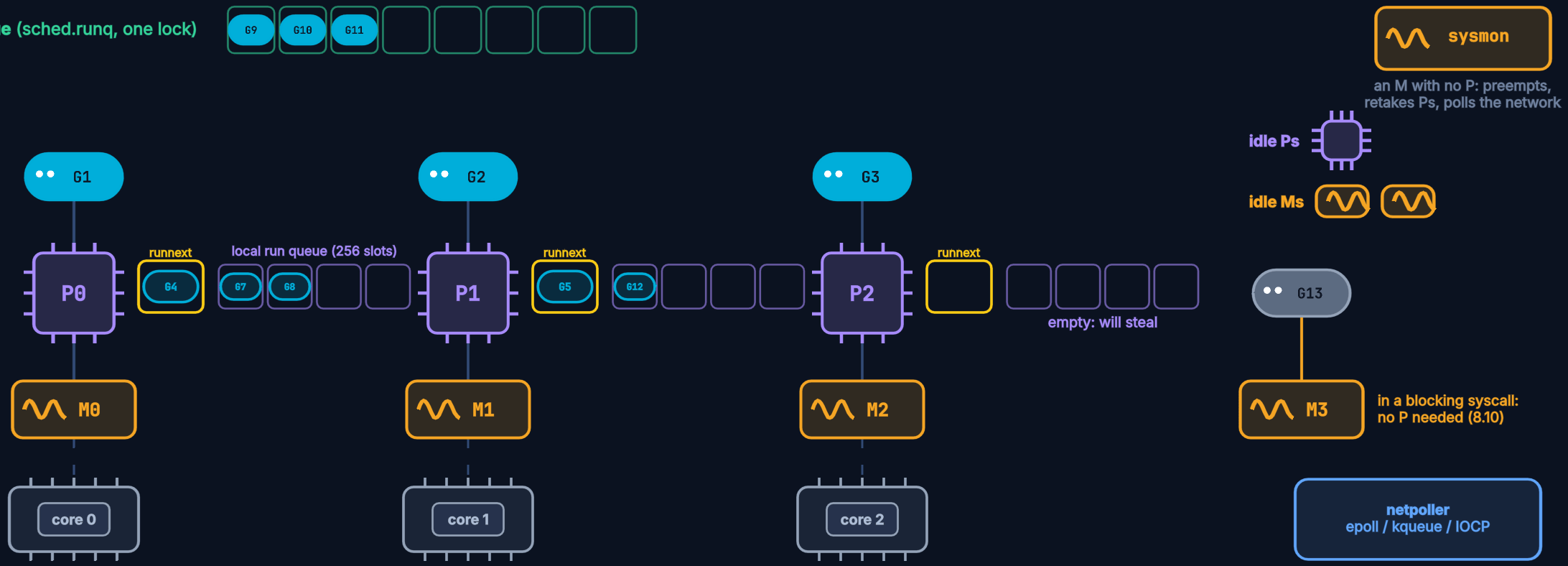
How many? Exactly GOMAXPROCS.

§ **RULE**

To run Go code, an M must hold a P. The M runs one G at a time on that P. An M in a blocking system call can give its P away, so GOMAXPROCS bounds Go code, not threads.

8.9 The big picture

GOMAXPROCS = 3. Every P has a local run queue and a runnext slot; a global queue catches overflow. Ms come and go; sysmon watches over all of it.



OS SCHEDULER: puts Ms (threads) on cores

8.9 GOMAXPROCS: how many Ps

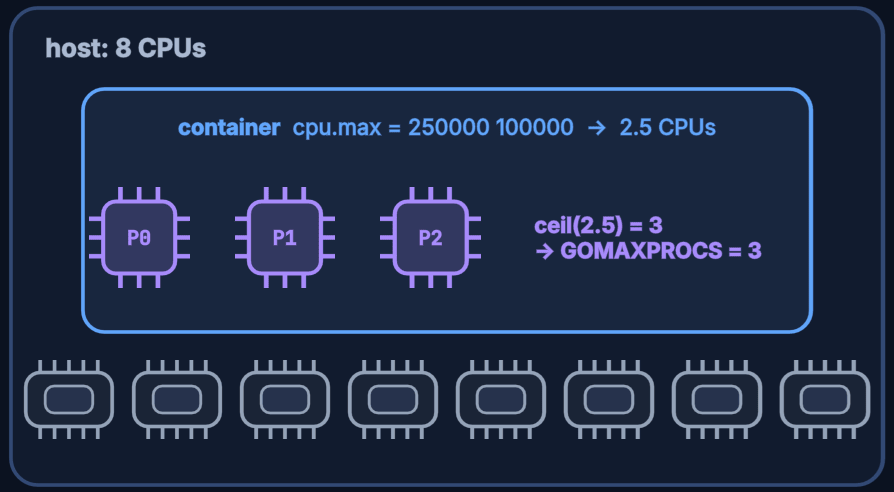
GOMAXPROCS is the number of Ps, so the number of goroutines that can execute Go code at the same instant.

examples/ch09_gomaxprocs/main.go — Go

```go
func main() {
    fmt.Println("logical CPUs (NumCPU):", runtime.NumCPU())
    fmt.Println("Ps (GOMAXPROCS):      ", runtime.GOMAXPROCS(0)) // 0 = query only

    prev := runtime.GOMAXPROCS(2) // set; returns the previous value
    fmt.Println("changed from", prev, "to", runtime.GOMAXPROCS(0))

    runtime.SetDefaultGOMAXPROCS() // Go 1.25+: back to the runtime default
    fmt.Println("default again:", runtime.GOMAXPROCS(0))
}
```

output (4 vCPUs, no CPU limit) — terminal

```
$ go run .
logical CPUs (NumCPU): 4
Ps (GOMAXPROCS):       4
changed from 4 to 2
default again: 4
```



- Default (Go 1.25+, Linux): the **smaller** of the CPUs this process may use (affinity mask) and the container's **CPU limit** (cgroup quota ÷ period, rounded **up**, at least 2).
- The runtime **re-checks** about once a second and adjusts if the limit changes.
- Setting `GOMAXPROCS=n` in the environment, or calling `runtime.GOMAXPROCS(n)`, fixes the value and stops the updates.
- `runtime.SetDefaultGOMAXPROCS()` returns to the automatic default.
- Opt out with `GODEBUG=containermaxprocs=0` or `updatemaxprocs=0`.

Setting	Effect
too high (> real CPU)	Ps compete for CPU time; the OS throttles the container; latency spikes
too low	idle cores while goroutines wait in run queues
just right	one P per CPU you are actually allowed to use

NOTE
GOMAXPROCS does **not** limit threads. Goroutines blocked in system calls, or locked to threads with `LockOSThread`, have Ms of their own (8.10).

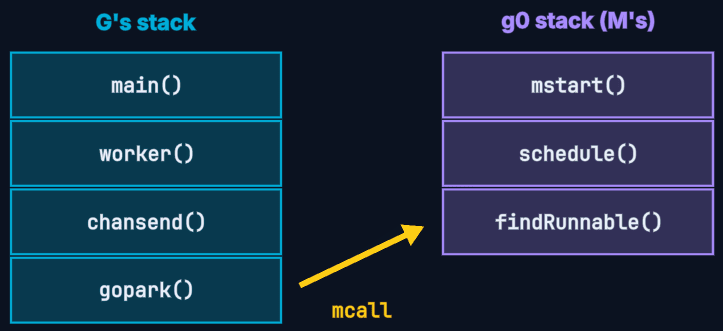
8.9 One turn of the scheduler: schedule() and findRunnable()

Whenever a G stops running (it parks, yields, exits or is preempted) its M switches to its g0 stack and calls schedule(), which looks for the next G in this order.

1. **every 61st tick** — take 1 G from the **global** queue first, so it is never starved
2. **local** — `runnext`, then the P's local run queue
3. **global** — take a batch from the global queue (up to 128, half a local queue)
4. **network** — non-blocking netpoll: Gs whose sockets became ready
5. **steal** — 4 rounds over the other Ps in random order: take **half** of a victim's queue
6. **GC** — if a GC is running, do background mark work
7. **sleep** — nothing anywhere: release the P; one M may block in netpoll, the rest park (`stopm`)

found G: `execute(gp)` → status _Grunning → `gogo(&gp.sched)` jumps to its saved PC and SP

THE g0 SWITCH



- Each M has a special goroutine **g0** whose stack runs scheduler code.
- `mcall(park_m)` saves the G's registers into `g.sched` and switches to g0.
- On g0, `schedule()` picks the next G and `gogo` switches to its stack.
- So a switch is G → g0 → G': two cheap stack swaps, no kernel.

UNDER THE HOOD

The 61 is from `pp.schedtick%61 == 0` in findRunnable: a prime, so it doesn't line up with periodic patterns in your program.

8.9 Recap and check yourself

G is the work, M is the worker thread, P is the permission (and the queue) to run Go code.

REMEMBER

- The runtime schedules Gs in user space: cheap switches that cooperate with channels, the GC and I/O.
- An M must hold a P to run Go code. #P = GOMAXPROCS; #M is whatever is needed (cap 10,000).
- Each P has a 256-slot run queue plus `runnext`; a global queue catches overflow.
- Go 1.25+: GOMAXPROCS respects the container CPU limit and is updated about once a second.
- findRunnable: global every 61 ticks, local, global batch, netpoll, steal half, GC work, sleep.
- A switch goes through g0: G → mcall → schedule() on g0 → gogo → G'.

CHECK YOURSELF

Q1. 10,000 goroutines on 4 cores, all doing CPU work. How many run at any instant?
Answer. At most GOMAXPROCS (4 by default here); the rest wait in run queues.

Q2. A container has a 1.2-CPU limit on a 64-core host. Default GOMAXPROCS in Go 1.27?
Answer. ceil(1.2) = 2, and the minimum is 2 anyway: GOMAXPROCS = 2 (it was 64 before Go 1.25).

Q3. Why does a P, not an M, own the run queue?
Answer. Ms block in syscalls and come and go; a P stays, so its queue and caches survive the hand-off.

Q4. What stops two goroutines that keep re-spawning each other from starving the global queue?
Answer. The every-61st-tick check of the global queue in findRunnable.

CHAPTER

8.10

The Scheduler II: Queues, Syscalls and I/O

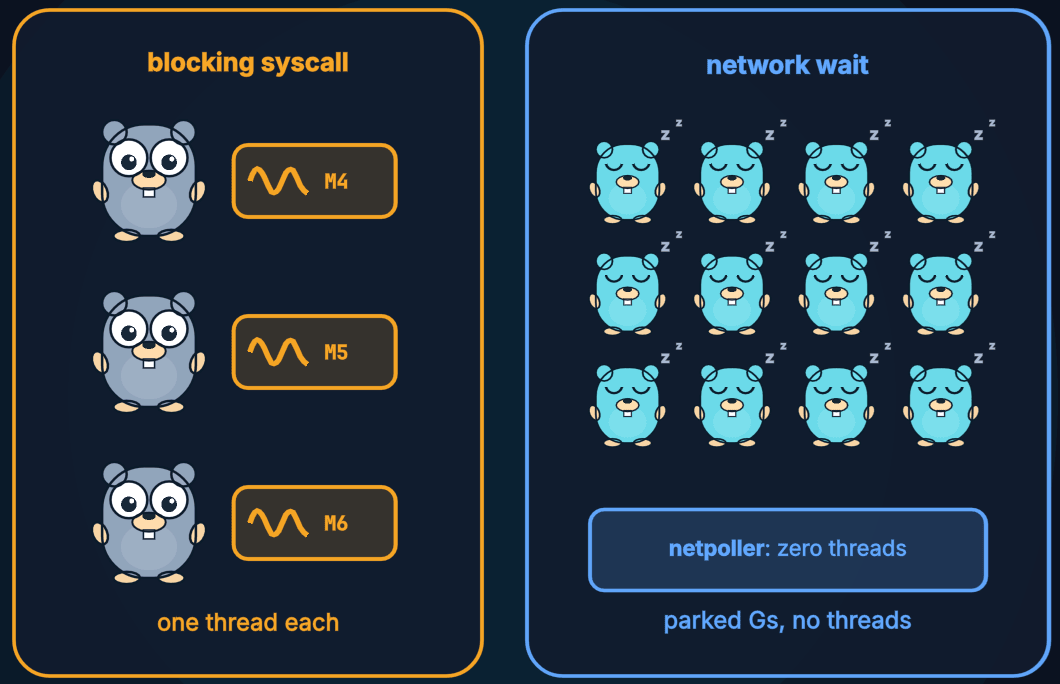
Where does a new goroutine go? What happens when one blocks in the kernel, or waits for a socket? The answers explain why Go servers need so few threads.

IN THIS CHAPTER YOU WILL

- ✓ Trace a new goroutine through runnext, the local queue and the global queue
- ✓ Explain work stealing and spinning threads
- ✓ Follow a blocking system call: hand-off, retake, return
- ✓ Explain how the netpoller parks thousands of connections
- ✓ Read sysmon's duties and the `schedtrace` output

? THE BIG QUESTION

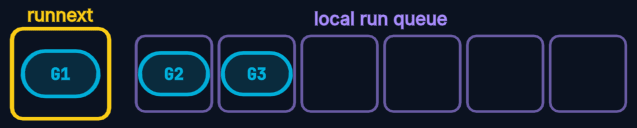
50 goroutines block in a system call, and 2,000 wait on idle network connections. Which of them need an OS thread of their own?



8.10 Where a new goroutine goes

`go f()` puts the new G in the current P's runnext slot. Everything else in this page follows from that one decision (runqput in proc.go).

1. before



P has G1 in runnext and G2, G3 queued.

2. go f() creates G9

G9 takes runnext; the old runnext G1 moves to the tail of the queue.

3. queue full (256)

The next `go` finds no room: runqputslow moves **half** (128) plus the new G to the global queue, in one locked batch.

WHY runnext?

When G1 wakes G2 (by sending on a channel, say) and then blocks, G2 should run **immediately** on the same P, with warm caches. runnext gives exactly that: the readied G runs next and **inherits the rest of the current time slice**, so a ping-pong pair cannot hog the P forever.

runtime/proc.go (Go 1.27, excerpts) — Go

```go
// runqput tries to put g on the local runnable queue.
// If next is true, runqput puts g in the pp.runnext slot.
// If the run queue is full, runnext puts g on the
// global queue.
func runqput(pp *p, gp *g, next bool)

// newproc: a new goroutine always goes to runnext
runqput(pp, newg, true)
```

KEY IDEA

Local queues are lock-free rings (only the owner adds; others may steal). The global queue needs `sched.lock`, which is why the runtime batches moves into and out of it.

8.10 Work stealing: idle Ps take half

A P with nothing to run does not wait for work to be assigned. Its M spins, picks victims at random, and takes half of one's queue.

before

after P2 steals from P0

P2 now runs G2 and has G3, G4 queued

THE RULES (findRunnable, stealWork, runqgrab)

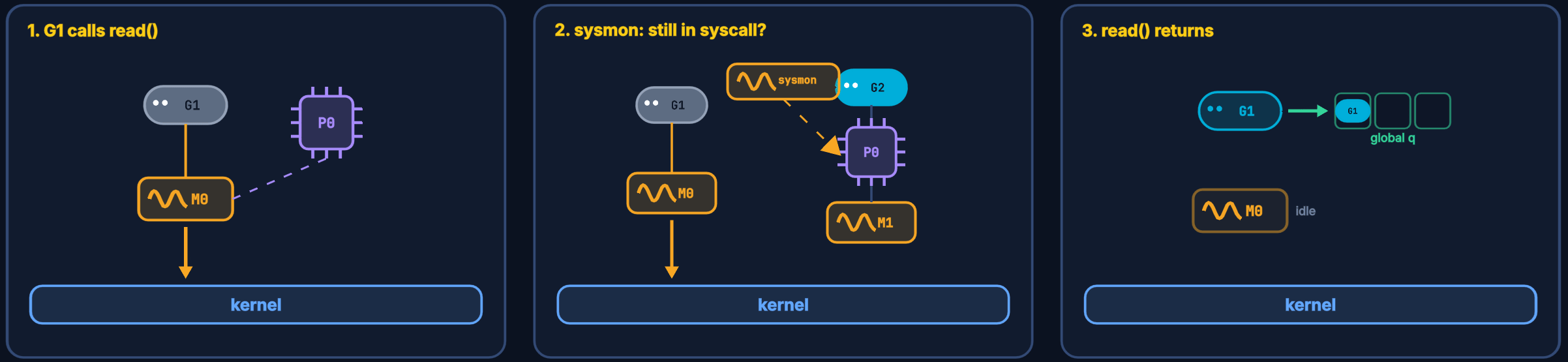
- **Victims in random order**, up to 4 passes over all Ps.
- Take **half** of the victim's queue (`n - n/2`), oldest first.
- Only on the last pass may a thief also take the victim's `runnext` (and its due timers).
- At most **half the busy Ps** may have a spinning M: idle machines don't burn CPU.
- A spinning M that finds work wakes another M, so work spreads quickly.

? **WHY HALF?**

Why half? Taking one G means stealing again soon; taking all moves the problem to the victim. Half balances both queues in one step, and each G is moved at most a few times.

8.10 Blocking system calls: hand the P to someone else

An M stuck in the kernel cannot run Go code, but its P (and the Gs queued on it) should not wait too. The runtime moves the P to another M.



`entersyscall`: G1 → _Gsyscall. M0 keeps P0 loosely, for a fast return.

After ≥ 1 sysmon tick (20 µs) with work waiting, `retake` + `handoffp`: P0 goes to M1, which runs G2.

`exitsyscall`: P0 is taken. Grab any idle P; if none, put G1 on the global queue and park M0.

examples/ch10_syscall_threads (main) — Go

```go
func main() {
    fmt.Printf("%-28s OS threads = %s\n", "at start", threads())
    run("50 x time.Sleep (parked)", func() { time.Sleep(300 * time.Millisecond) })
    ts := syscall.Timespec{Nsec: 300_000_000}
    run("50 x blocking syscall", func() { syscall.Nanosleep(&ts, nil) })
}
```

measured — terminal

```
$ go run .
at start                     OS threads = 5
50 x time.Sleep (parked)     OS threads = 5
50 x blocking syscall        OS threads = 53
```

8.10 The netpoller: thousands of connections, few threads

Go puts every socket in non-blocking mode. A read that would block parks the goroutine instead of the thread, and the OS tells the runtime when the socket is ready.

1. **Read** `conn.Read` calls `read()`; the socket has no data: the kernel returns EAGAIN at once.
2. **Park** the G parks with wait reason "IO wait"; the fd is already registered with epoll (kqueue, IOCP). The M is free.
3. **Ready** data arrives; the kernel marks the fd ready.
4. **Poll** `netpoll()` (called by findRunnable, by sysmon every 10 ms, or by an M blocked in epoll_wait) returns the ready Gs.
5. **Run** `injectglist` makes them runnable; the read is retried and succeeds.

2,000 goroutines in `conn.Read`, status IO wait

netpoller (one epoll instance per process)

examples/ch10_netpoll: the server side — Go

```
27    go func() { // server: one goroutine per connection
28        for {
29            c, err := ln.Accept()
30            if err != nil {
31                return
32            }
33            go func() {
34                buf := make([]byte, 1)
35                c.Read(buf) // blocks in the netpoller, not on a thread
36            }()
37        }
38    }()
```

measured — terminal

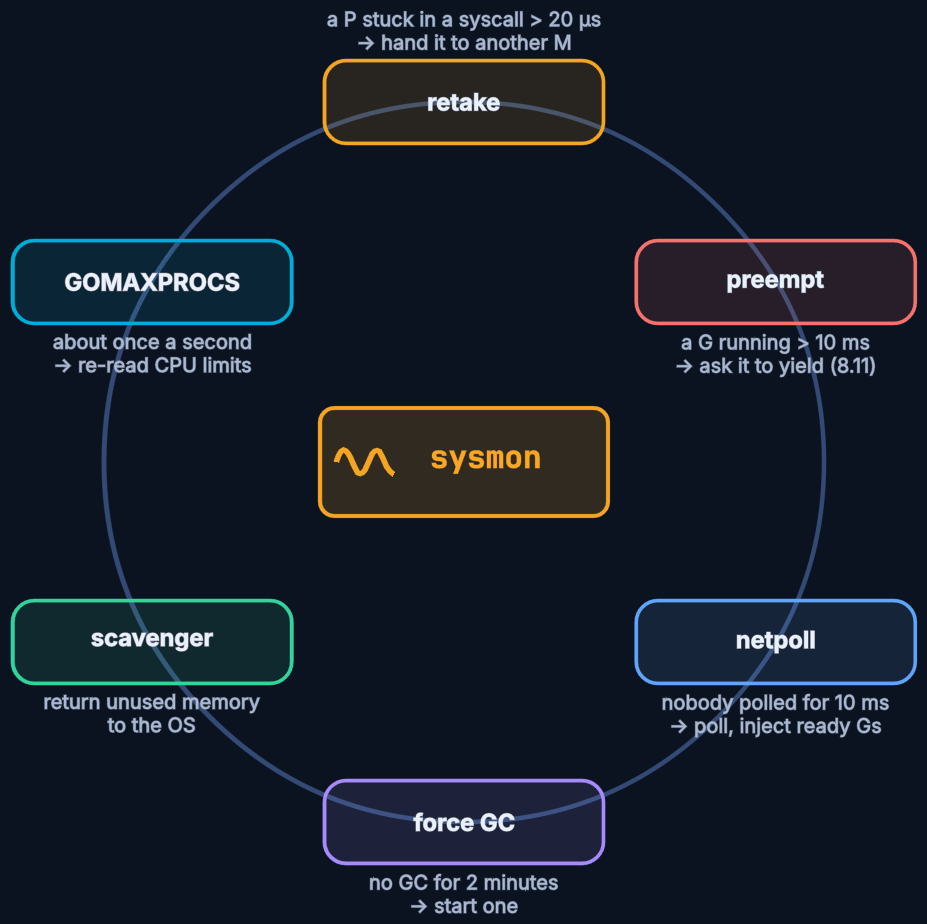
```
$ go run .
open connections: 2000
goroutines:       2002
OS threads:       8
```

KEY IDEA

2,000 open connections and 2,002 goroutines needed only **8 OS threads.** Compare 50 blocking syscalls → 53 threads on the previous page.

8.10 sysmon: the runtime's watchdog

One dedicated M runs `sysmon` without a P, so it keeps working even when every P is busy. It wakes every 20 µs, backing off to 10 ms when the program is idle.



Situation	sysmon sleeps
busy: it just did something	20 µs
idle for 1 ms (50 rounds)	doubles each round
idle for long	at most 10 ms

runtime/proc.go (Go 1.27, excerpts) — Go

```go
const forcePreemptNS = 10 * 1000 * 1000 // 10ms
var forcegcperiod int64 = 2 * 60 * 1e9   // 2 minutes

// in sysmon's loop:
if idle == 0 {
    delay = 20 // start with 20us sleep...
} else if idle > 50 { // ...double after 1ms
    delay *= 2
}
if delay > 10*1000 { // up to 10ms
    delay = 10 * 1000
}
```

NOTE
sysmon is also why a goroutine can't monopolise a P for long: it is the one that notices the 10 ms and triggers preemption (8.11).

8.10 Watching the scheduler: GODEBUG=schedtrace

The runtime prints a one-line summary every N milliseconds. Here 12 CPU-bound goroutines run on 4 Ps for 250 ms, between two idle phases.

real output — terminal

```
$ go build -o app .
$ GODEBUG=schedtrace=100 ./app
SCHED 0ms: gomaxprocs=4 idleprocs=2 threads=5 spinningthreads=1 needspinning=0 idlethreads=2 runqueue=0 [ 0 0 0 0 ] schedticks=[ 1 0 0 2 ]
SCHED 100ms: gomaxprocs=4 idleprocs=0 threads=5 spinningthreads=0 needspinning=1 idlethreads=0 runqueue=8 [ 0 0 0 0 ] schedticks=[ 4 3 3 5 ]
SCHED 201ms: gomaxprocs=4 idleprocs=0 threads=5 spinningthreads=0 needspinning=1 idlethreads=0 runqueue=5 [ 0 2 1 0 ] schedticks=[ 9 8 8 10 ]
SCHED 302ms: gomaxprocs=4 idleprocs=0 threads=5 spinningthreads=0 needspinning=1 idlethreads=0 runqueue=2 [ 1 1 0 1 ] schedticks=[ 17 13 13 15 ]
SCHED 403ms: gomaxprocs=4 idleprocs=4 threads=5 spinningthreads=0 needspinning=0 idlethreads=3 runqueue=0 [ 0 0 0 0 ] schedticks=[ 19 15 17 19 ]
```

READING ONE LINE

```
SCHED 100ms: gomaxprocs=4 idleprocs=0 threads=5 spinningthreads=0 needspinning=1 idlethreads=0 runqueue=8 [ 0 0 0 0 ]
```

Field	Value	Meaning
gomaxprocs	4	number of Ps
idleprocs	0	Ps with nothing to run
threads	5	Ms that exist
spinningthreads	0	Ms looking for work

Field	Value	Meaning
needspinning	1	a spinning M was requested
idlethreads	0	Ms parked, ready for reuse
runqueue	8	Gs in the global queue
[...]	[0 0 0 0]	Gs in each P's local queue
schedticks	[4 3 3 5]	scheduler calls per P

PRO TIP Add `scheddetail=1` for one line per P, M and G. For timelines of individual goroutines, use `runtime/trace` and `go tool trace` (8.17).

8.10 Recap and check yourself

Queues keep work local; stealing spreads it; hand-offs and the netpoller keep threads few.

REMEMBER

- A new G goes to runnext; the old runnext moves to the queue; a full queue sends half to the global queue.
- Idle Ps steal half of a random victim's queue; spinning Ms are limited to half the busy Ps.
- A syscall longer than ~20 µs loses its P to another M; on return the G may go to the global queue.
- Blocking syscalls cost threads (5 → 53 for 50 calls); network waits don't (8 threads for 2,000 conns).
- sysmon: retake Ps, preempt after 10 ms, poll the network, force GC every 2 min, update GOMAXPROCS.
- `GODEBUG=schedtrace=1000` prints Ps, threads and queue lengths every second.

CHECK YOURSELF

Q1. A goroutine wakes another via a channel send and then blocks. Where does the woken one run?
Answer. On the same P, from runnext, with the rest of the time slice.

Q2. Why does reading a file (not a socket) still create threads?
Answer. Regular files are not pollable: read() blocks in the kernel, so the M is lost for the duration.

Q3. schedtrace shows runqueue=8 and idleprocs=0. What does it mean?
Answer. All Ps are busy and 8 Gs wait in the global queue: CPU-bound saturation.

Q4. Which component would notice a P stuck in a 5-second syscall?
Answer. sysmon, via retake.

CHAPTER

8.11 Preemption

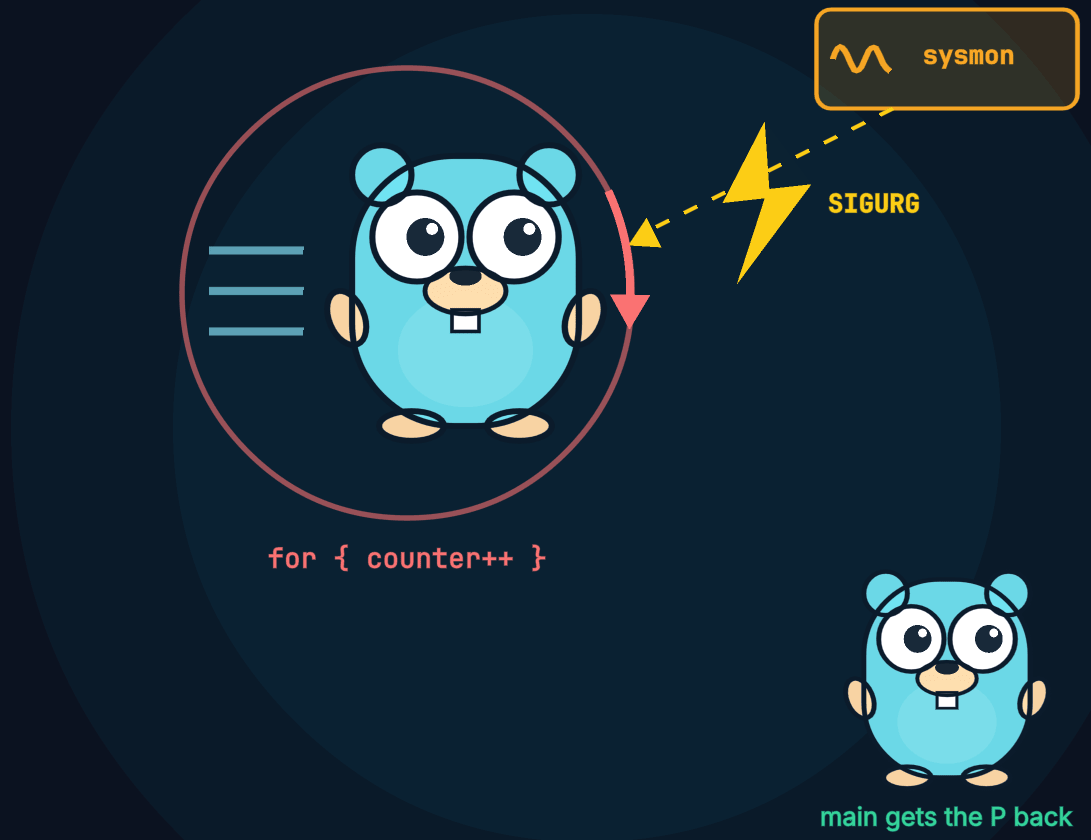
A goroutine that never blocks would keep its P forever. Preemption is how the runtime takes the CPU back, and since Go 1.14 it works even inside tight loops.

IN THIS CHAPTER YOU WILL

- ✓ List every point where a goroutine gives up its P
- ✓ Explain cooperative preemption through the stack guard
- ✓ Follow asynchronous preemption: sysmon, SIGURG, safe points
- ✓ Reproduce a pre-1.14 hang with `asyncpreemptoff=1`
- ✓ Use `Gosched` and `LockOSThread` for the right reasons

? THE BIG QUESTION

With GOMAXPROCS=1, one goroutine runs `for { counter++ }`. Will main ever run again?



8.11 When does a goroutine give up its P?

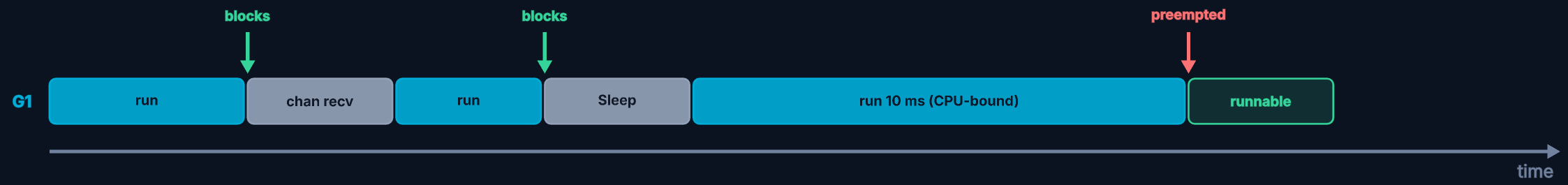
Most of the time goroutines yield on their own, because they block. Preemption is the safety net for the ones that never do.

It blocks or yields (voluntary)

- ✓ channel send / receive / `select` that must wait
- ✓ `sync.Mutex`, `RWMutex`, `WaitGroup.Wait`, `Cond.Wait`
- ✓ `time.Sleep`, timers, tickers
- ✓ network I/O through the netpoller (8.10)
- ✓ a blocking system call (the P is handed off)
- ✓ `runtime.Gosched()`: stay runnable, let others go first
- ✓ GC assist work and waits

It is stopped (involuntary: preemption)

- it ran for more than **10 ms** (sysmon's `forcePreemptNS`)
- the GC needs to scan its stack or stop the world
- a debugger or profiler asks to suspend it
- **How:** at its next function call (cooperative, Go 1.2+), or almost anywhere with a signal (asynchronous, Go 1.14+)



8.11 Cooperative preemption: hijacking the stack check

Go 1.2 reused the prologue every function already has (chapter 8.3). To ask a G to stop, the runtime poisons its stack guard.

1. **Request** `preemptone` sets `gp.preempt = true` and `gp.stackguard0 = stackPreempt`, a huge value (0xfffffffffffffade on 64-bit).
2. **Trip** at the next function call, `CMPQ SP, 16(R14)` sees SP below the guard: every stack looks "too small".
3. **Divert** `morestack` → `newstack` notices `stackPreempt`: this is not a real overflow.
4. **Yield** `gopreempt_m` puts the G on the global run queue and calls `schedule()`.

every function prologue

```
CMPQ  SP, 16(R14)   // 16(R14) = g.stackguard0
JLS   morestack     // poisoned guard: always taken
```

THE HOLE IN THE NET

The flaw: a loop with no function calls never runs a prologue, so it can never be stopped. Before Go 1.14 one such loop could freeze the whole program: the GC waited for it to stop, and so did everyone else.

unstoppable before Go 1.14 — Go

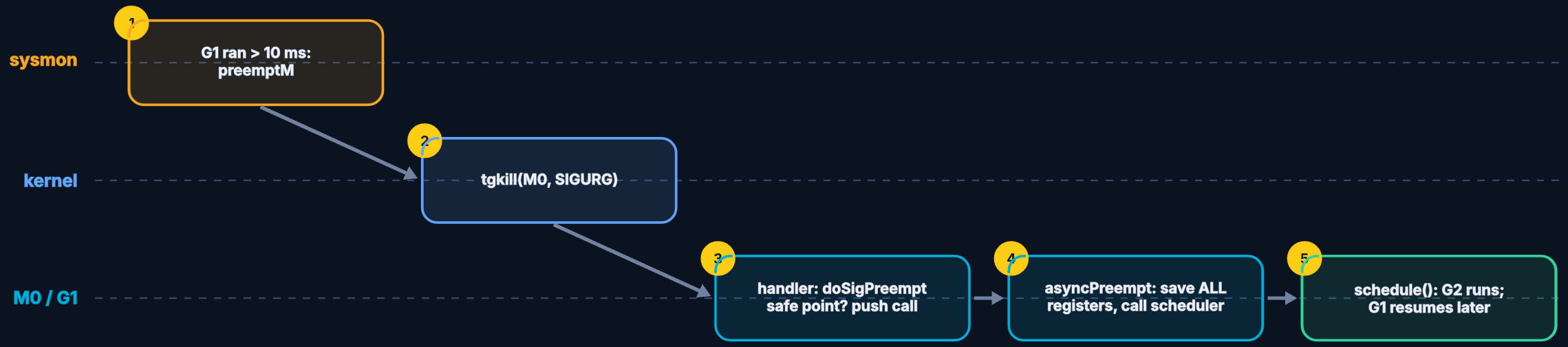
```go
// no call inside: no prologue, no check
for i := 0; i < 1<<40; i++ {
    sum += i * i
}
```

UNDER THE HOOD

The same `stackguard0` slot serves two masters: stack growth and preemption. Overloading a check that already runs everywhere made cooperative preemption almost free.

8.11 Asynchronous preemption (Go 1.14+): a signal

To stop a goroutine that makes no calls, the runtime interrupts its thread with a signal and makes the goroutine call the scheduler from wherever it is.



SAFE POINT

An **async safe point** is an instruction where the runtime can stop the G and still find every pointer in its frame: not inside the runtime itself, not in assembly without metadata, not in a write-barrier or atomic sequence. Elsewhere the handler does nothing and the request is retried later.

WHY SIGURG

Why SIGURG? From runtime/signal_unix.go: debuggers pass it through by default, libc does not use it internally, and a spurious one is harmless. SIGUSR1/2 belong to applications; SIGALRM cannot tell a real alarm from ours.

GO VERSION NOTE Go 1.14 (Feb 2020), after Austin Clements' proposal "non-cooperative goroutine preemption" (golang/go#24543). Turn it off with `GODEBUG=asyncpreemptoff=1`.

8.11 See it for yourself: one P, one tight loop

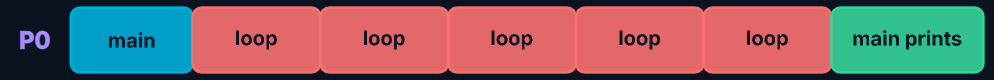
The same program, run twice. The only difference is the GODEBUG setting.

examples/ch11_preempt/main.go — Go

```go
var counter uint64

func main() {
    go func() {
        for { // a tight loop: no function calls, no channel ops
            counter++
        }
    }()
    time.Sleep(50 * time.Millisecond) // main gives up the only P
    // Async preemption (Go 1.14+) forces the loop off the CPU:
    fmt.Println("main is running again; the loop was preempted")
}
```

default: async preemption on — terminal

```
$ GOMAXPROCS=1 go run .
main is running again; the loop was preempted
```



main sleeps 50 ms; the loop holds the P but is preempted every ~10 ms, so main runs soon after its timer fires

examples/ch11_preempt_off/main.go — Go

```go
var counter uint64

func main() {
    go func() {
        for { // a tight loop: no function calls, no channel ops
            counter++
        }
    }()
    time.Sleep(50 * time.Millisecond) // main gives up the only P
    // Async preemption (Go 1.14+) forces the loop off the CPU:
    fmt.Println("main is running again; the loop was preempted")
}
```

asyncpreemptoff=1: like Go 1.13 — terminal

```
$ GOMAXPROCS=1 GODEBUG=asyncpreemptoff=1 go run .
^C   (killed after 4s: the program never finished)
```

no call, no signal: the loop never yields, main never runs, the run is killed after 4 s

★ **KEY IDEA** Async preemption bounds how long one goroutine can hold a P to about **10 ms**, whatever its code looks like.

8.11 Gosched, LockOSThread and latency

You rarely need to touch the scheduler. Here are the two knobs that exist, and what preemption means for latency.

examples/ch11_gosched/main.go — Go

```go
func main() {
    var wg sync.WaitGroup
    for _, name := range []string{"A", "B"} {
        wg.Go(func() {
            for i := range 3 {
                fmt.Print(name, i, " ")
                runtime.Gosched() // yield: go to the back of the run queue
            }
        })
    }
    wg.Wait()
    fmt.Println()
}
```

GOMAXPROCS=1 — terminal

```
$ GOMAXPROCS=1 go run .
B0 A0 B1 A1 B2 A2
```

PRO TIP

`runtime.Gosched()` moves the caller to the back of the queue and lets others run. With async preemption it is almost never needed; if you reach for it, a channel or lock is usually missing.

runtime.LockOSThread()

Wires the calling goroutine to its current thread: no other goroutine runs on that M until `UnlockOSThread`. Needed for C libraries with thread-local state (OpenGL, some GUI toolkits), for Linux namespaces and `setns`, and in `init` for code that must stay on the main thread. If the goroutine exits while locked, the thread is destroyed.

What preemption means for latency

A runnable G may wait for the running G's slice to end: up to ~10 ms per G ahead of it in the queue. Many CPU-bound goroutines on few Ps therefore raise tail latency for everyone. Fixes: fewer CPU-bound goroutines than Ps (a worker pool), or give latency-critical work its own process.

Where preemption cannot happen

Inside the runtime and assembly without safe-point metadata, while the G holds runtime locks, and during cgo calls (the C code is not Go). A long cgo call is treated like a blocking syscall instead.

8.11 Recap and check yourself

Goroutines usually yield by blocking; preemption guarantees the rest yield within ~10 ms.

REMEMBER

- Voluntary yields: channel/lock/select waits, sleeps, network I/O, syscalls, `Gosched`.
- sysmon requests preemption after 10 ms; the GC requests it to scan stacks.
- Cooperative (1.2+): poison `stackguard0`, the next call's prologue diverts to the scheduler.
- Asynchronous (1.14+): SIGURG to the thread; at a safe point the handler injects `asyncPreempt`.
- `GODEBUG=asyncpreemptoff=1` reproduces the old hang: our demo was killed after 4 s.
- `LockOSThread` is for thread-bound C state; `Gosched` is almost never the right fix.

CHECK YOURSELF

Q1. Why did pre-1.14 tight loops also block garbage collection?

Answer. Stopping the world needs every G at a safe point; the loop never reached one.

Q2. Is a goroutine preempted in the middle of `x += 1` on a shared int?

Answer. It can be, between instructions at a safe point: one more reason unsynchronised sharing is a race.

Q3. Which signal does the runtime use, and can your program also use it?

Answer. SIGURG; you can, but must tolerate spurious ones.

Q4. What happens to a locked goroutine's thread when the goroutine exits without unlocking?

Answer. The runtime terminates that thread instead of reusing it.

CHAPTER

8.12

Sharing Memory: sync and atomics

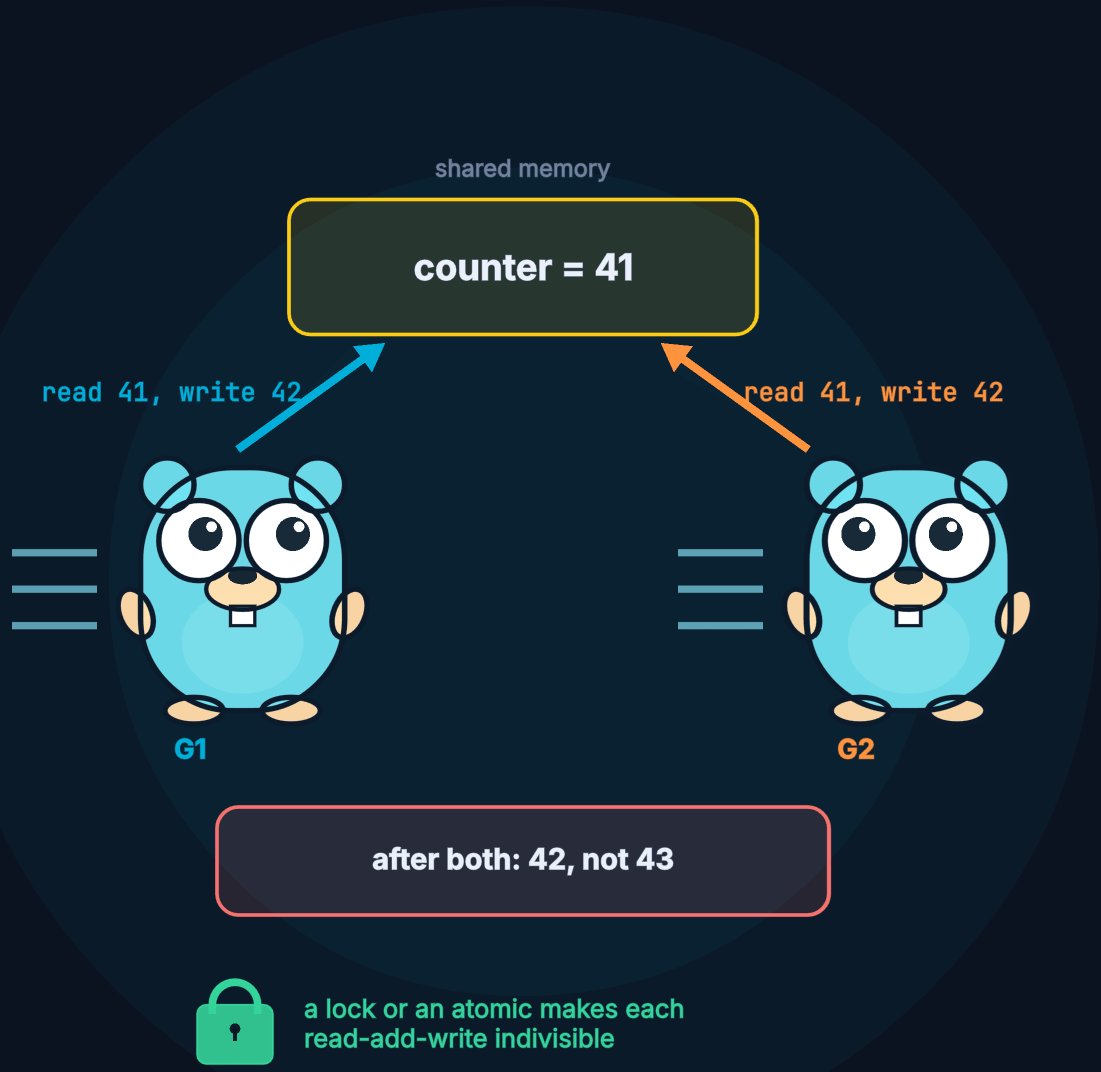
Channels move data between goroutines. Sometimes the data must stay put and many goroutines must touch it. The sync and sync/atomic packages make that safe.

IN THIS CHAPTER YOU WILL

- ✓ Explain why `counter++` loses updates under concurrency
- ✓ Guard state with `Mutex` and `RWMutex`, and know their rules
- ✓ Look inside the Mutex: fast path, spinning, starvation mode
- ✓ Use `Once`, `OnceValue`, `Cond`, `Pool` and `sync.Map` correctly
- ✓ Write lock-free code with atomics, and pick the right tool

? THE BIG QUESTION

1,000 goroutines each add 1 to a shared counter 1,000 times. We expect 1,000,000; one run printed 741,983. Where did the other 258,017 increments go?



8.12 counter++ is three steps, not one

An increment reads the value, adds one and writes it back. Two goroutines can interleave those steps, and one update silently disappears.

examples/ch12_race_counter/main.go — Go

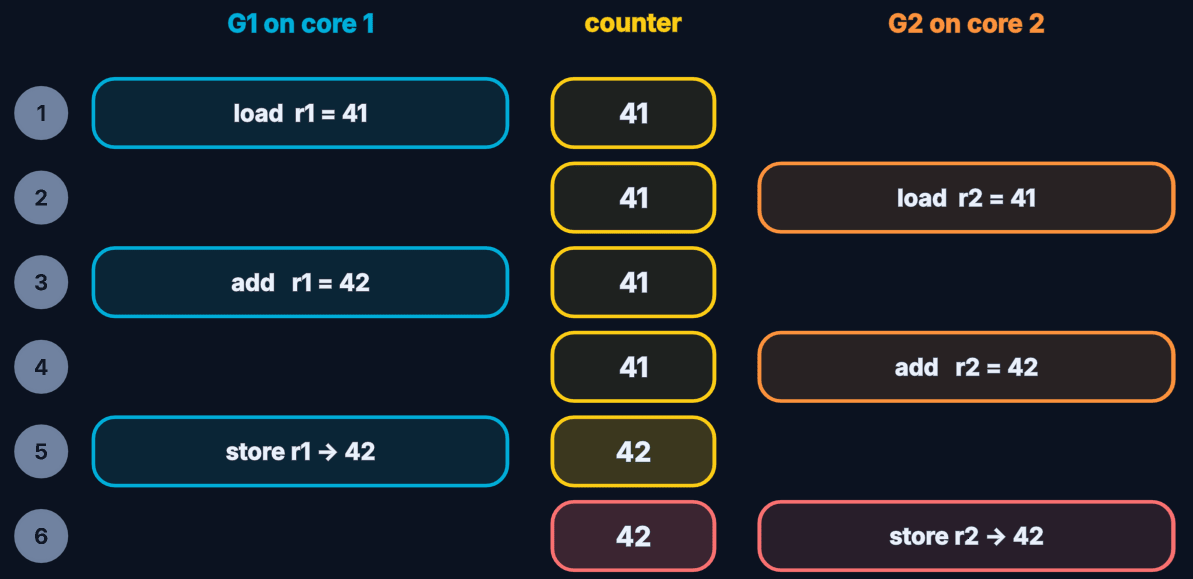
```go
func main() {
    counter := 0
    var wg sync.WaitGroup
    for range 1000 {
        wg.Go(func() {
            for range 1000 {
                counter++ // DATA RACE: load, add, store
            }
        })
    }
    wg.Wait()
    fmt.Println("expected 1000000, got", counter)
}
```

go run . (the result changes every run) — terminal

```
$ go run .
expected 1000000, got 741983
```

EVEN ONE INSTRUCTION IS NOT ATOMIC

On amd64 the compiler turns `*p++` into one instruction, `INCQ (AX)`. It is still a read, an add and a write inside the core, and without a LOCK prefix another core can use the same memory in between. To Go it is a **data race**: no result is promised at all (8.13).



LOST UPDATE

Two increments ran, but the counter moved 41 → 42 **once**. G2 overwrote G1's result with a value computed from a stale read: a **lost update**. In the run above, 258,017 of 1,000,000 updates were lost this way.

KEY IDEA

Steps 1-3 and 5 must happen as one indivisible unit. A **mutex** makes the whole block exclusive; an **atomic** makes the single operation indivisible in hardware.

8.12 sync.Mutex: one goroutine at a time

A mutex (mutual exclusion lock) guards a critical section: the code between Lock and Unlock runs in at most one goroutine at any moment.

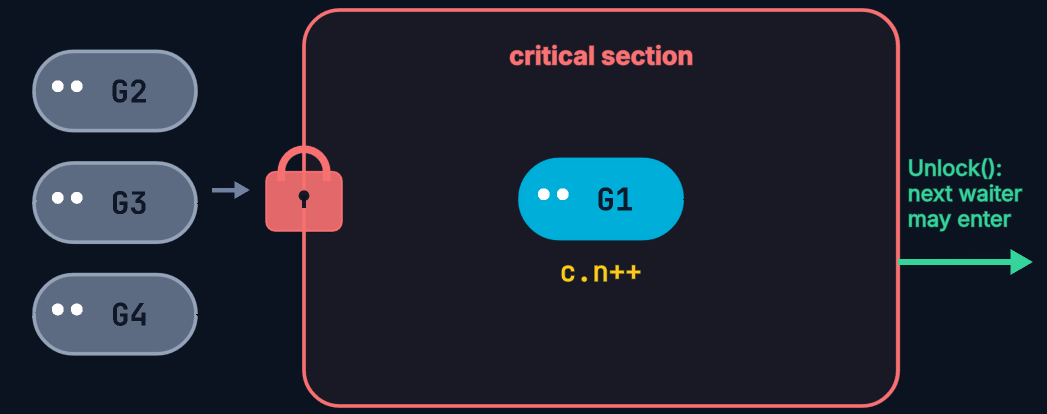
examples/ch12_mutex_counter/main.go — Go

```
 8 type Counter struct {
 9     mu sync.Mutex // guards n                    1
10     n  int
11 }
12
13 func (c *Counter) Inc() {
14     c.mu.Lock()                                  2
15     defer c.mu.Unlock()                          3
16     c.n++ // critical section: one goroutine at a time   4
17 }
```

1. **Pair the lock with its data.** The zero `sync.Mutex` is unlocked and ready; no constructor.
2. **Lock.** If another goroutine holds it, this one sleeps until it is free.
3. **defer Unlock.** Runs on every return path, even during a panic.
4. **Critical section.** Only the holder runs here, so the read-add-write cannot interleave.

same 1,000 × 1,000 loop, now calling c.Inc() — terminal

```
$ go run .
expected 1000000, got 1000000
```



KEY IDEA

Locking gives two guarantees: **exclusion** (one holder at a time) and **visibility**: whatever the previous holder wrote before Unlock is seen by the next holder after Lock. That second half is the Go memory model's happens-before (8.13).

PRO TIP

Keep critical sections short: compute outside, lock, update, unlock. Never block on I/O or a channel while holding a lock if you can avoid it.

8.12 Mutex rules that prevent real bugs

A mutex is simple to call and easy to misuse. These rules come straight from the sync docs and the vet checks.

examples/ch12_copylock/main.go — Go

```go
type Counter struct {
    mu sync.Mutex
    n  int
}

// BUG: a value receiver copies the struct, Mutex included.
func (c Counter) Inc() {
    c.mu.Lock()
    c.n++ // increments the copy, not the caller's Counter
    c.mu.Unlock()
}

func main() {
    var c Counter
    c.Inc()
    c.Inc()
    fmt.Println("n =", c.n)
}
```

go vet (copylocks analyzer) — terminal

```
$ go vet .
main.go:14:9: Inc passes lock by value:
    gozm/module08/ch12_copylock.Counter contains sync.Mutex
```

DO

- Lock, then `defer mu.Unlock()` on the very next line.
- Put the mutex right above the fields it guards and say so in a comment.
- Use pointer receivers; let `go vet` catch accidental copies.
- Hold the lock for as little work as possible.
- Always take several locks in one fixed global order.

DON'T

- Lock twice in one goroutine: Go mutexes are **not re-entrant**, the second Lock waits forever.
- Unlock an unlocked mutex: fatal error "sync: unlock of unlocked mutex". `recover` cannot catch it.
- Call unknown code (callbacks, interface methods) while holding the lock.
- Rely on `TryLock` (Go 1.18): the docs say correct uses are rare.
- Assume a lock belongs to a goroutine: any goroutine may Unlock it (legal, rarely clear).

PRO TIP

Fix: use a pointer receiver, `func (c *Counter) Inc()`. Every method on a type that contains a lock should take a pointer, and the value must never be copied after first use. The program above prints `n = 0`.

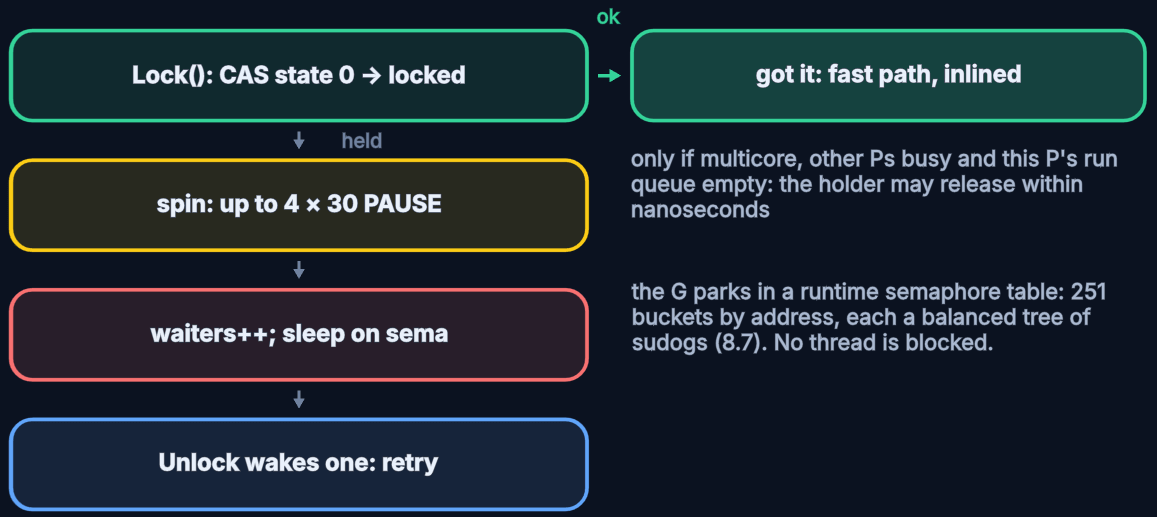
8.12 Inside sync.Mutex: one word of state, a smart slow path

The whole lock is 8 bytes. The uncontended case is a single atomic instruction; contention takes a slower, fairer path.

internal/sync/mutex.go — Go

```
type Mutex struct {
    state int32  // flags + number of waiters
    sema  uint32 // where sleeping waiters park
}
```

state (int32)

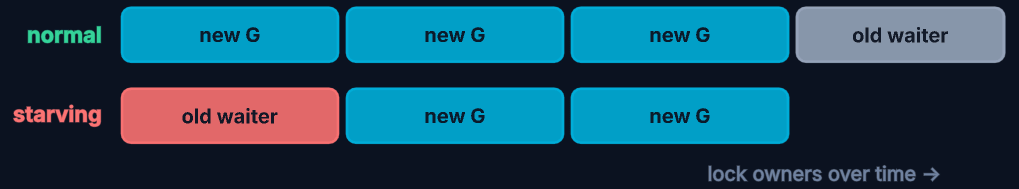


Normal mode: throughput

Waiters queue in FIFO order, but a woken waiter must **compete** with goroutines that just arrived. Newcomers are already running on a CPU, so they usually win, and the woken one goes back to the front of the queue. Lock handoff is cheap; a waiter can lose many times in a row.

Starvation mode: fairness

If a waiter fails to get the lock for more than **1 ms**, the mutex switches mode. Unlock now **hands ownership directly** to the waiter at the front; newcomers do not spin or grab it, they queue at the tail. The mutex returns to normal mode when the owner is the last waiter or waited less than 1 ms.



GO VERSION NOTE Starvation mode arrived in Go 1.9 (2017). It caps a waiter's delay; the price is slower handoffs under heavy contention.

AND UNLOCK?

Unlock clears the locked bit with one atomic add. If waiters exist and none is already awake, it releases the semaphore to wake one (normal mode) or hands the lock straight over (starvation mode).

8.12 sync.RWMutex: many readers or one writer

Reads that do not change state can safely overlap. An RWMutex lets any number of readers in together, but a writer gets the data alone.

examples/ch12_rwmutex_cache/main.go — Go

```go
type Cache struct {
    mu sync.RWMutex
    m  map[string]string
}

func (c *Cache) Get(k string) (string, bool) {
    c.mu.RLock() // many readers may hold RLock together   R
    defer c.mu.RUnlock()
    v, ok := c.m[k]
    return v, ok
}

func (c *Cache) Set(k, v string) {
    c.mu.Lock() // a writer excludes readers AND writers   W
    defer c.mu.Unlock()
    c.m[k] = v
}
```

2: W calls Lock 3: R3 calls RLock 4.6: last reader leaves, W enters

	readers	writer
RLock / RUnlock	share with other readers	waits for all of them
Lock / Unlock	wait	waits for other writers

IS IT FASTER?

4 goroutines on 4 cores, tiny critical section (one map read): Mutex 69.8 ns, RWMutex 46.6 ns per read. Readers overlap, but every RLock still does an atomic add on one shared counter, so the gain is far from 4×. It grows when reads dominate **and** take a while.

UNDER THE HOOD

`readerCount` counts readers. Lock subtracts 2^30 (`rwmutexMaxReaders`) from it, so a **negative** count tells every new RLock "a writer is waiting: queue up". That is why R3 waits even though only readers hold the lock: writers cannot be starved by a stream of readers.

PITFALL

No upgrades: RLock then Lock in one goroutine deadlocks. No recursive RLock either: if a writer arrives between the two RLocks, everyone waits forever.

8.12 sync.Once and OnceValue: initialise exactly once

Lazy initialisation from many goroutines: the first caller runs the function, everyone else waits for it and then sees its result.

examples/ch12_once/main.go — Go

```go
type Config struct{ DSN string }

// loadConfig runs at most once, on first use, even with many callers.
var loadConfig = sync.OnceValue(func() *Config {
    fmt.Println("loading config...")
    return &Config{DSN: "postgres://db"}
})

func main() {
    var wg sync.WaitGroup
    for range 5 {
        wg.Go(func() {
            _ = loadConfig() // all 5 get the same *Config
        })
    }
    wg.Wait()
    fmt.Println("DSN:", loadConfig().DSN)
}
```

1

2

go run . (five callers, one load) — terminal

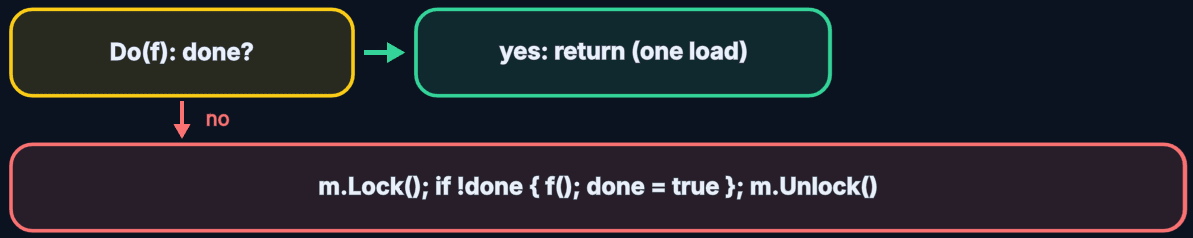
```
$ go run .
loading config...
DSN: postgres://db
```

PITFALL

If f panics, `Once.Do` counts it as done and never retries; `OnceFunc`/`OnceValue` re-panic with the same value on every call. Calling `Do` on the same Once from inside f deadlocks.

sync/once.go — Go

```go
type Once struct {
    done atomic.Bool // first: the hot path is inlined
    m    Mutex
}
```



Why a mutex and not just a CAS on done? Late callers must **wait until f has finished**, or they could use a half-built config.

API (since)	returns	use for
`Once.Do(f)` (1.0)	nothing	side effects
`OnceFunc(f)` (1.21)	`func()`	a callable once-only action
`OnceValue(f)` (1.21)	`func() T`	a lazily built value
`OnceValues(f)` (1.21)	`func() (T, error)`	value plus error

8.12 sync.Cond: wait until a condition becomes true

A condition variable lets goroutines sleep until some state guarded by a mutex changes, then wakes one (Signal) or all (Broadcast).

examples/ch12_cond/main.go — Go

```go
func main() {
    var mu sync.Mutex
    cond := sync.NewCond(&mu)
    ready := false

    var wg sync.WaitGroup
    for id := 1; id <= 3; id++ {
        wg.Go(func() {
            mu.Lock()
            for !ready { // ALWAYS wait in a loop
                cond.Wait() // atomically unlocks mu and sleeps
            }
            mu.Unlock()
            fmt.Println("runner", id, "starts")
        })
    }

    time.Sleep(50 * time.Millisecond)
    mu.Lock()
    ready = true
    mu.Unlock()
    fmt.Println("GO! (Broadcast)")
    cond.Broadcast() // wake every waiter
    wg.Wait()
}
```

go run . (order varies) — terminal

```
$ go run .
GO! (Broadcast)
runner 2 starts
runner 1 starts
runner 3 starts
```

WHAT cond.Wait() DOES

1. **Register** on the Cond's notify list (takes a ticket).
2. **Unlock** `c.L`, so others can change the state.
3. **Sleep** until Signal (one waiter) or Broadcast (all).
4. **Lock** `c.L` again before returning.
5. **Re-check** the condition: another goroutine may have changed it first.

Broadcast wakes all three; Signal would wake only the longest waiter.

PREFER CHANNELS The sync docs: "For many simple use cases, users will be better off using channels than a Cond (Broadcast corresponds to closing a channel, and Signal corresponds to sending on a channel)." Cond wins when the same state is waited on repeatedly.

8.12 sync/atomic: indivisible operations on one word

An atomic operation is done by the hardware as a single, uninterruptible step. No lock, no sleeping, no scheduler.

examples/ch12_atomic_counter/main.go — Go

```go
func main() {
    var counter atomic.Int64 // typed atomic (Go 1.19+)
    var wg sync.WaitGroup
    for range 1000 {
        wg.Go(func() {
            for range 1000 {
                counter.Add(1) // one indivisible read-modify-write   (1)
            }
        })
    }
    wg.Wait()
    fmt.Println("expected 1000000, got", counter.Load())

    var flag atomic.Bool
    fmt.Println("CAS false->true:", flag.CompareAndSwap(false, true))   (2)
    fmt.Println("CAS false->true:", flag.CompareAndSwap(false, true))
}
```

go run . — terminal

```
$ go run .
expected 1000000, got 1000000
CAS false->true: true
CAS false->true: false
```

*p++ on an int

```
INCQ (AX)
```

p.Add(1), atomic.Int64

```
MOVL  $1, CX
LOCK
XADDQ CX, (AX)
```

Compiled by Go 1.27 on amd64 (`go build -gcflags=-S`). The LOCK prefix makes the core own the cache line for the whole read-add-write.

type (Go 1.19+)	Load Store Swap CAS	Add	And Or
atomic.Bool	✓	–	–
atomic.Int32, Int64	✓	✓	1.23
atomic.Uint32, Uint64, Uintptr	✓	✓	1.23
atomic.Pointer[T]	✓	–	–
atomic.Value (any, 1.4)	✓	–	–

★ **KEY IDEA**

Go's atomics are **sequentially consistent**: all goroutines see all atomic operations in one order, and they synchronise like locks do (memory model, 2022).

! **PITFALL**

An atomic protects **one** word. Two fields that must change together (a balance and a history) need a mutex. Mixing atomic and plain access to the same variable is a race.

8.12 Compare-and-swap loops and atomic.Pointer

CAS writes a new value only if the old one is still there. Retrying it in a loop builds any lock-free update out of one primitive.

examples/ch12_cas/main.go — Go

```go
// storeMax raises m to v if v is larger: a classic CAS loop.
func storeMax(m *atomic.Int64, v int64) {
    for {
        old := m.Load()                  // 1
        if v <= old {                    // 2
            return // nothing to do
        }
        if m.CompareAndSwap(old, v) {    // 3
            return // our write won
        }
        // m changed between Load and CAS: retry
    }
}
```

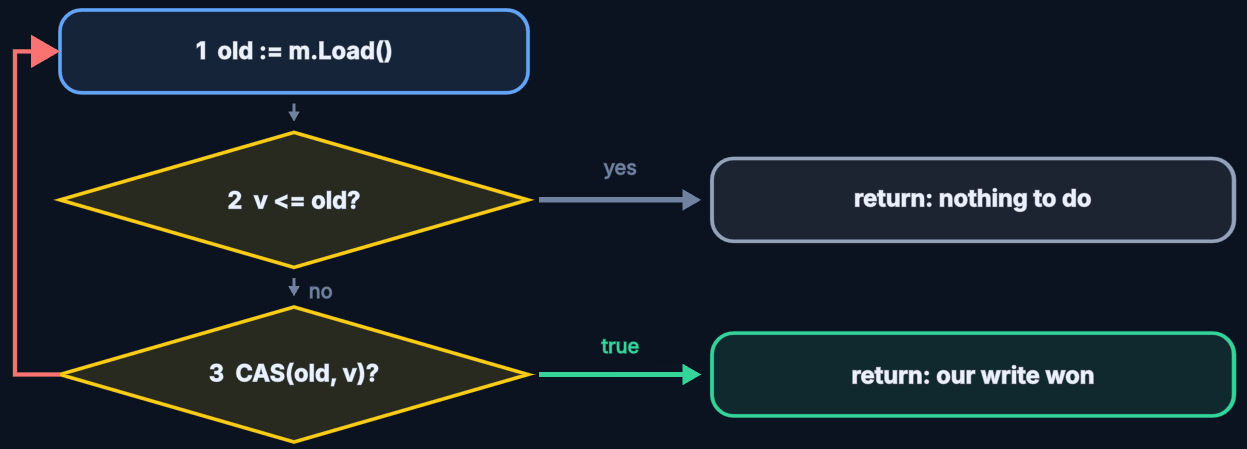
examples/ch12_cas/main.go — Go

```go
    var cfg atomic.Pointer[Config]
    cfg.Store(&Config{Limit: 10})
    old := cfg.Swap(&Config{Limit: 20}) // readers see old OR new
    fmt.Println("old:", old.Limit, "new:", cfg.Load().Limit)
```

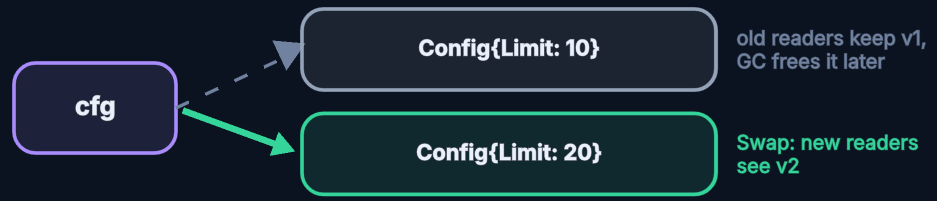
go run . — terminal

```
$ go run .
max: 100
old: 10 new: 20
```

THE CAS LOOP



PUBLISHING WITH atomic.Pointer

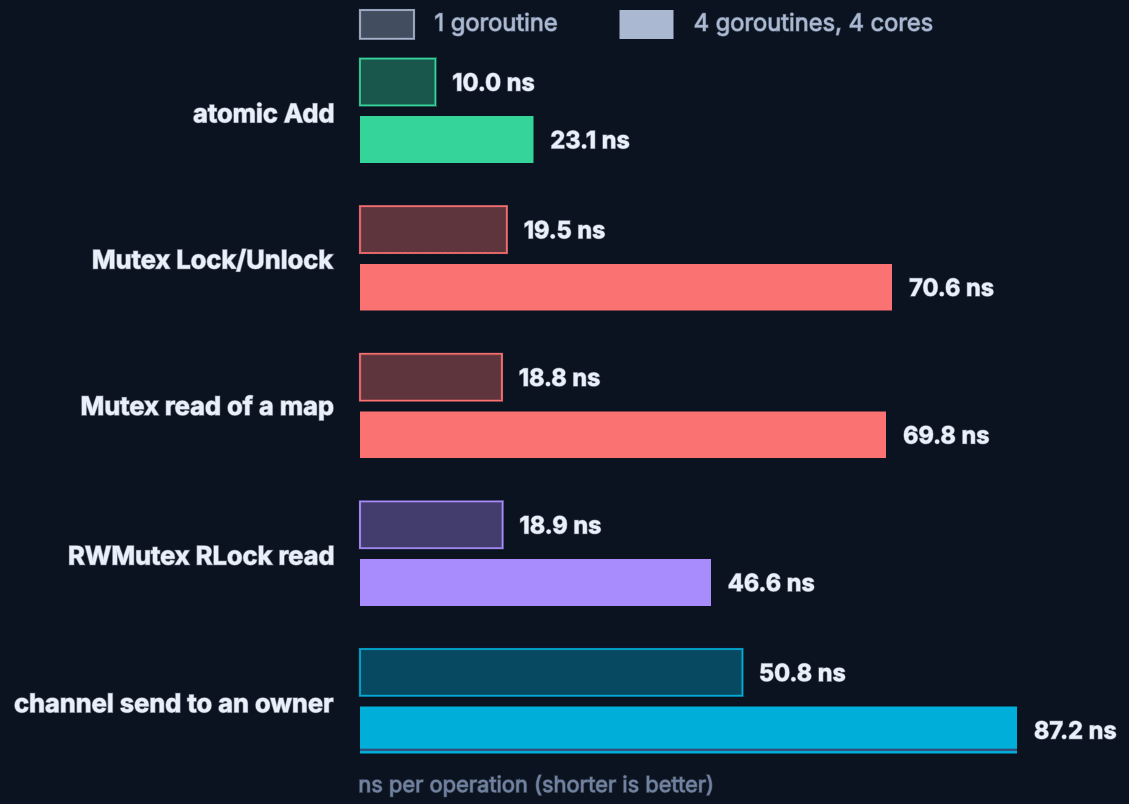


PRO TIP

Copy-on-write: never modify a published value. Build a new one and swap the pointer; readers see the old or the new value, never a mix.

8.12 What does each tool cost? Measured

Five benchmarks hammer one shared counter or map, first from 1 goroutine, then from 4 goroutines on 4 cores (-cpu=1,4).



- **Contention, not the primitive, is the cost.** Going from 1 to 4 goroutines makes a mutex 3.6× slower: the lock's cache line ping-pongs between cores and waiters spin or sleep.
- **Atomics are cheapest**: 23.1 ns contended, 3.1× faster than the mutex. One LOCK XADD, never a sleep.
- **A channel is the slowest counter** (87.2 ns): every increment is a send that the owner goroutine must receive. Channels are for moving work and data, not for bumping a number.
- **RWMutex reads overlap**: 46.6 vs 69.8 ns. Not 4× better: every RLock still writes one shared counter.
- Even the slowest here is about 87 ns. Pick the clearest tool first, measure, then optimise.

> **NOTE**
> Single runs on a 4-vCPU cloud VM (Xeon 2.1 GHz). The 1-goroutine numbers include RunParallel's own loop overhead. Your hardware will differ; the ratios are what matter.

examples/ch12_bench/counter_test.go — Go

```go
func BenchmarkAtomicInc(b *testing.B) {
    var n atomic.Int64
    b.RunParallel(func(pb *testing.PB) {
        for pb.Next() {
            n.Add(1)
        }
    })
}
```

8.12 sync.Pool: recycle temporary objects

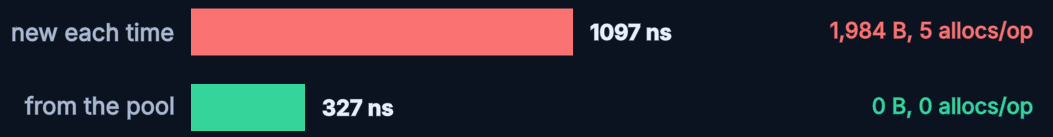
A Pool is a cache of reusable objects that the garbage collector is allowed to empty. It cuts allocations on hot paths.

examples/ch12_pool/pool_test.go

```go
var bufPool = sync.Pool{
    New: func() any { return new(bytes.Buffer) },
}

func render(buf *bytes.Buffer) int {
    for range 64 {
        buf.WriteString("hello, gopher ")
    }
    return buf.Len()
}

func BenchmarkNewBuffer(b *testing.B) {
    for b.Loop() {
        buf := new(bytes.Buffer) // fresh allocation every time
        render(buf)
    }
}

func BenchmarkPooledBuffer(b *testing.B) {
    for b.Loop() {
        buf := bufPool.Get().(*bytes.Buffer)
        buf.Reset() // pooled objects keep old contents: reset!
        render(buf)
        bufPool.Put(buf)
    }
}
```

1 2 3

BENCHMARK go test -bench . -benchmem



ONE CACHE PER P, SO GET RARELY CONTENDS

shared queue: owner pushes/pops the head, other Ps steal from the tail

Get() looks in this order

own private → own shared → steal → victim → New()

UNDER THE HOOD

Put() fills the private slot, else the own shared queue. At each GC the pool's objects move to a **victim** cache; whatever is still unused at the next GC is freed (Go 1.13).

PITFALL

Reset objects after Get; never Put something still in use; don't pool huge buffers (they stay alive); not for connections: entries vanish without notice.

8.12 sync.Map: a concurrent map for two special cases

A map safe for many goroutines without your own lock. The docs are blunt: most code should use a plain map with a mutex.

examples/ch12_syncmap/main.go — Go

```go
func main() {
    var seen sync.Map // map[string]int, safe for concurrent use
    var wg sync.WaitGroup
    for _, w := range []string{"go", "chan", "go", "select", "go"} {
        wg.Go(func() {
            // LoadOrStore: one atomic "insert if absent"
            if _, loaded := seen.LoadOrStore(w, len(w)); loaded {   // 1
                fmt.Println("duplicate:", w)
            }
        })
    }
    wg.Wait()
    n := 0
    seen.Range(func(k, v any) bool { n++; return true })   // 2
    fmt.Println("unique words:", n)
}
```

go run . — terminal

```
$ go run .
duplicate: go
duplicate: go
unique words: 3
```

USE IT WHEN (from the sync docs)

1. Write once, read many

a key is stored once and then only read, as in caches that only grow

2. Disjoint keys

goroutines read, write and overwrite entries for different sets of keys

method	does
`Load`, `Store`, `Delete`	the basics
`LoadOrStore`	insert if absent, in one atomic step
`LoadAndDelete`, `Swap`	remove or replace, returning the old value
`CompareAndSwap`, `CompareAndDelete`	change only if the value still matches
`Range`, `Clear` (1.23)	visit every entry; empty the map

GO VERSION NOTE

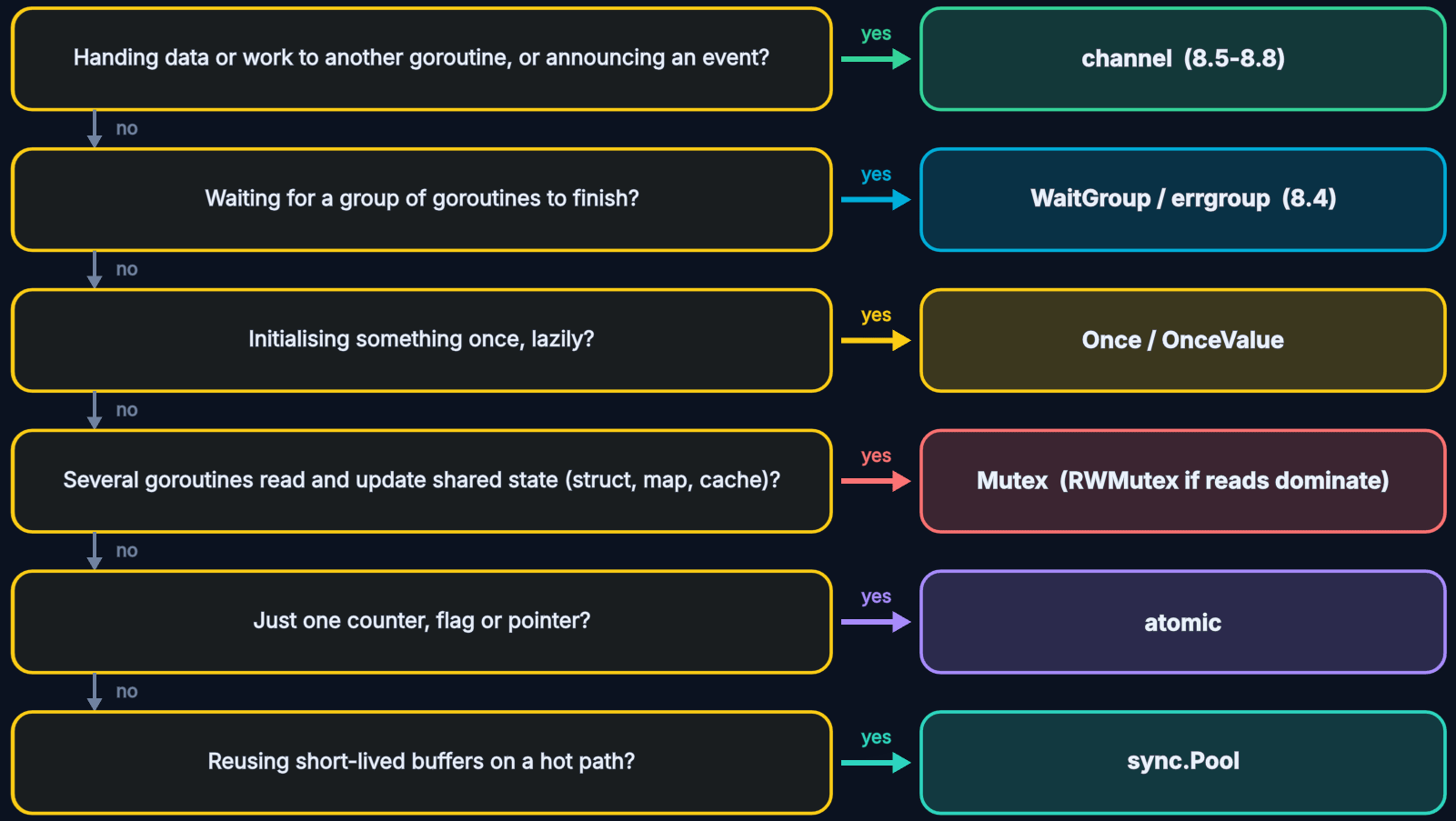
Go 1.24 rebuilt sync.Map on a concurrent hash-trie (`internal/sync.HashTrieMap`), the structure behind the `unique` package. The API did not change.

NOTE

Values come back as `any`: you need type assertions, and the compiler cannot check key or value types. Plain maps are not safe for concurrent writes: the runtime may stop the program with "fatal error: concurrent map writes".

8.12 Choosing: channel, mutex, atomic or something else?

Ask what the goroutines are doing with the data. The answer usually picks the tool.



> *Don't communicate by sharing memory; share memory by communicating.*
>
> — Go Proverbs (Rob Pike, 2015)

channel	mutex
passing ownership of data	caches
distributing units of work	state inside a struct
communicating async results	a few fields updated together

after the Go wiki page "Use a sync.Mutex or a channel?"

KEY IDEA

"Use whichever is most expressive and/or most simple." A mutex inside a type with a small API is often clearer than a goroutine plus channels, and the reverse holds for pipelines.

8.12 Recap and check yourself

Shared memory is safe when every access is ordered: by a lock, by an atomic, or by not sharing at all.

REMEMBER

- `x++` on shared memory is read-add-write: concurrent increments get lost.
- Mutex: zero value ready, `Lock` + `defer Unlock`, not re-entrant, never copy it.
- Mutex inside: CAS fast path, brief spinning, sleep on a semaphore; starvation mode after 1 ms.
- RWMutex: readers share, writers exclude; a waiting writer blocks new readers.
- Once/OnceValue: run once, everyone waits for the result. Cond: wait in a loop, or use a channel.
- Atomics: one word, sequentially consistent; CAS loops for anything fancier.
- Pool recycles garbage-collectable objects; sync.Map fits two narrow cases.

CHECK YOURSELF

Q1. A method `func (c Cache) Get(k string)` locks `c.mu`. What is wrong?
Answer. The value receiver copies the mutex, so each call locks its own copy: no exclusion. Use `*Cache`.

Q2. Why can new readers be blocked while only readers hold an RWMutex?
Answer. A writer is waiting; readerCount is negative, so new readers queue behind it.

Q3. Two goroutines call `OnceValue`'s function while the first call is still running. What does the second see?
Answer. It waits until the first call returns, then gets the same value.

Q4. When is a CAS loop better than a mutex?
Answer. When the update is one word and retries are rare: no sleeping, no lock to forget.

CHAPTER

8.13

The Memory Model and the Race Detector

On real hardware one goroutine's writes reach another goroutine late, and not always in order. The Go memory model says exactly when a write is guaranteed to be seen; the race detector finds code that ignores it.

IN THIS CHAPTER YOU WILL

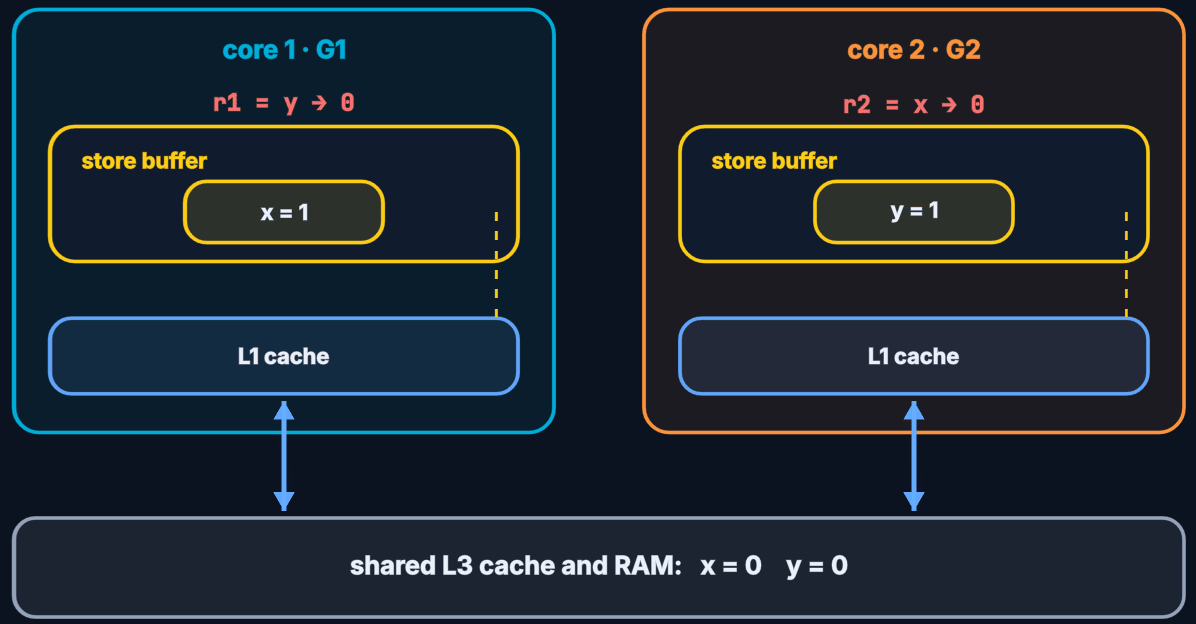
- Measure a reordering that no interleaving can explain
- Reason with happens-before: sequenced plus synchronized
- List every synchronizing operation Go provides
- Define a data race and what Go still promises after one
- Run the race detector and read its reports

THE BIG QUESTION

Goroutine A sets `msg = "hello"` and then `done = true`. Goroutine B sees `done == true`. Is B guaranteed to see `msg == "hello"`?

8.13 Memory is not one shared whiteboard

Between one goroutine's write and another's read sit a compiler, an out-of-order CPU and a cache hierarchy. Each may change the order in which others see writes.



1. **core 1:** `x = 1` goes into its store buffer, not yet into the cache.
2. **core 2:** `y = 1` goes into its store buffer.
3. **core 1:** loads `y` from the cache: still **0**.
4. **core 2:** loads `x` from the cache: still **0**.
5. **later:** both buffers drain; memory holds x = 1, y = 1. Too late for the reads.

1. The compiler

keeps values in registers, hoists loads out of loops, reorders independent loads and stores. All legal for a single goroutine.

2. The CPU

runs instructions out of order. Even x86 lets a later load pass an earlier store (the store buffer); ARM and POWER reorder far more.

3. The caches

agree on each single location eventually, but promise nothing about the order in which different locations become visible.

★ **KEY IDEA**

Synchronizing operations (channels, locks, atomics) are where the compiler and CPU are told to stop: they forbid code motion and emit memory barriers. Everything else may move.

8.13 Measured: an outcome no interleaving can explain

The store-buffer test. Each goroutine writes its own variable, then reads the other's. In every interleaving at least one read must see a 1.

examples/ch13_litmus/main.go — Go

```go
func main() {
    const n = 1_000_000
    plain := round(n,
        func(id int) {
            if id == 0 {
                x = 1
                r1 = y
            } else {
                y = 1
                r2 = x
            }
        },
        func() { x, y = 0, 0 },
        func() bool { return r1 == 0 && r2 == 0 })
```

go run . (both goroutines start each round together) — terminal

```
$ go run .
plain  ints: r1 == r2 == 0 in 2783 of 1000000 rounds
atomic ints: r1 == r2 == 0 in 0 of 1000000 rounds
```

WHAT HAPPENED

2,783 times both reads saw 0: each core's store was still in its store buffer when its load read the old value. With `atomic.Int64`: **0** times. Go's atomics are sequentially consistent; on amd64 an atomic store is an XCHG, a full barrier.

ALL 6 INTERLEAVINGS

`a: x = 1  b: r1 = y  (G1)  c: y = 1  d: r2 = x  (G2)`

order	r1	r2
a b c d	0	1
a c b d	1	1
a c d b	1	1
c a b d	1	1
c a d b	1	1
c d a b	1	0
measured, plain ints	**0**	**0**

NOTE

Sequential consistency means "as if the operations of all goroutines were interleaved in one order". The hardware does not promise that for plain memory; Go promises it only for race-free programs.

8.13 Happens before: the only order you can rely on

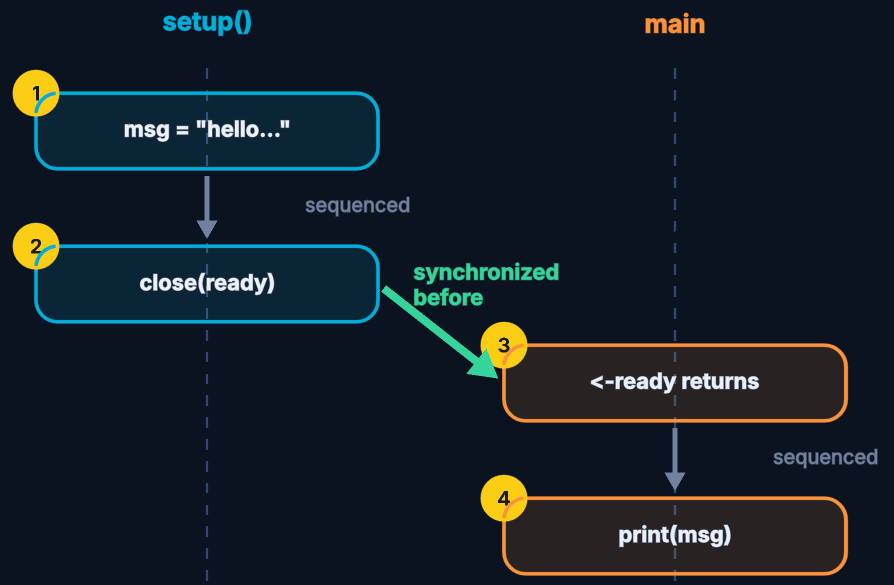
The memory model defines one relation. If a write happens before a read, the read sees it. If neither happens before the other, you have a race.

examples/ch13_publish/main.go — Go

```go
var msg string
var ready = make(chan struct{})

func setup() {
    msg = "hello, world" // (1) write the data
    close(ready)         // (2) publish: close is synchronized before...
}

func main() {
    go setup()
    <-ready          // (3) ...this receive completes
    fmt.Println(msg) // (4) guaranteed to print "hello, world"
}
```

go run . — terminal

```
$ go run .
hello, world
```



happens before: 1 → 2 → 3 → 4, so 4 sees 1

KEY IDEA

DRF-SC: a program with no data races behaves as if all goroutines ran interleaved on one processor (sequentially consistent). Race-free code lets you forget store buffers and caches.

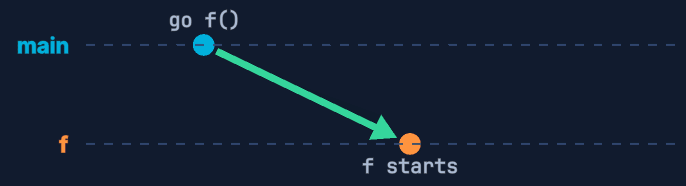
- **sequenced before**: program order inside one goroutine
- **synchronized before**: a synchronizing write observed by a synchronizing read (a close seen by a receive, an Unlock by a Lock)
- **happens before**: any chain of the two
- a plain read **must** see write w if w happens before it and no other write to that variable comes in between

8.13 Synchronizing edges (1): goroutines and channels

Quoted from the Go memory model. Each green arrow means: everything before its tail is visible after its head.

go statement

The go statement is synchronized before the new goroutine starts running.

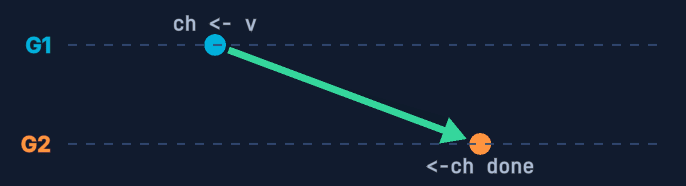


goroutine exit

A goroutine's exit is synchronized before **nothing**. Wait for it with a channel or WaitGroup.

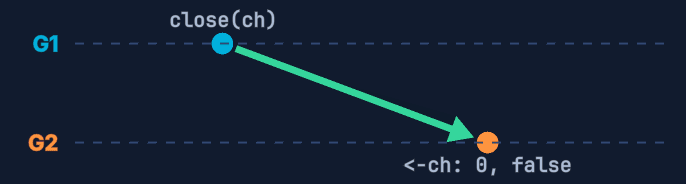
send → receive

A send is synchronized before the matching receive **completes**.



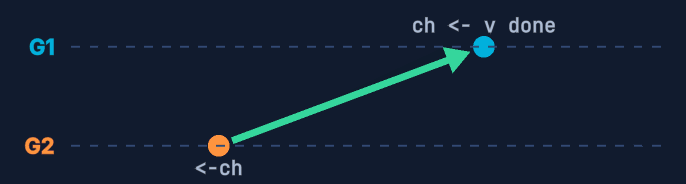
close → receive

Closing a channel is synchronized before a receive that returns a zero value because it is closed.



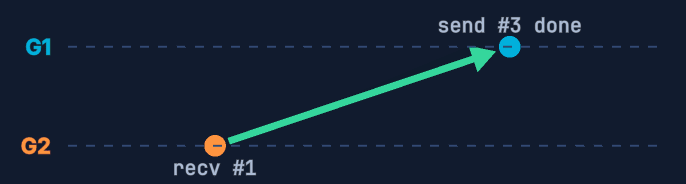
unbuffered: receive → send

A receive from an **unbuffered** channel is synchronized before the matching send completes.



buffered: receive k → send k+C

The kth receive on a channel of capacity C is synchronized before the (k+C)th send completes: a semaphore.



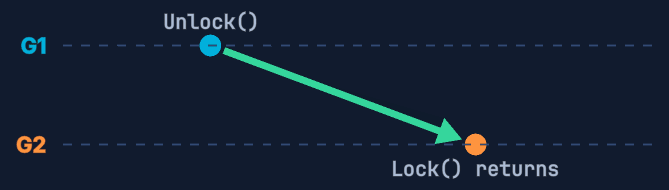
NOTE The exit card is why `go f()` followed by reading f's results is a race unless something waits for f. On a buffered channel, receive → send holds only in its k + C form.

8.13 Synchronizing edges (2): locks, Once, atomics and friends

The sync and sync/atomic packages give the rest of the edges. Every other package must document its own.

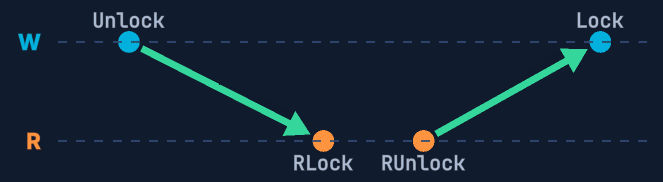
Mutex, RWMutex

Call n of `Unlock` is synchronized before call m of `Lock` returns, for every n < m.



RWMutex readers

An Unlock is synchronized before the RLock that follows it; that reader's RUnlock before the next Lock.

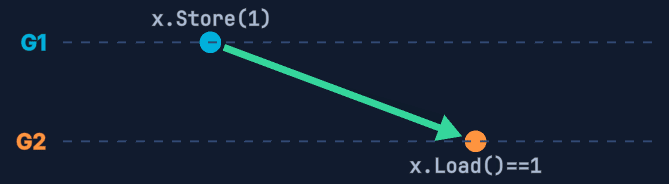


sync.Once

The single call of f completes before **any** `once.Do(f)` returns, including in other goroutines.

Atomics

If atomic B observes atomic A's effect, A is synchronized before B. All atomics act in one sequentially consistent order.

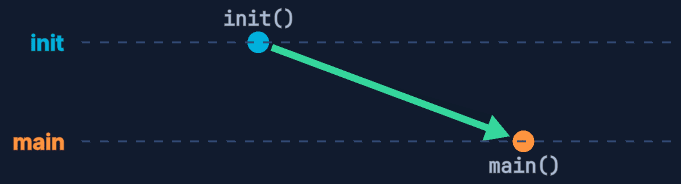


WaitGroup, Cond, Map, Pool

Documented per type. WaitGroup: each `Done` is synchronized before the `Wait` it unblocks returns.

init and finalizers

If p imports q, q's init finishes before p's starts; all init before `main.main`. `SetFinalizer(x, f)` is synchronized before `f(x)`.



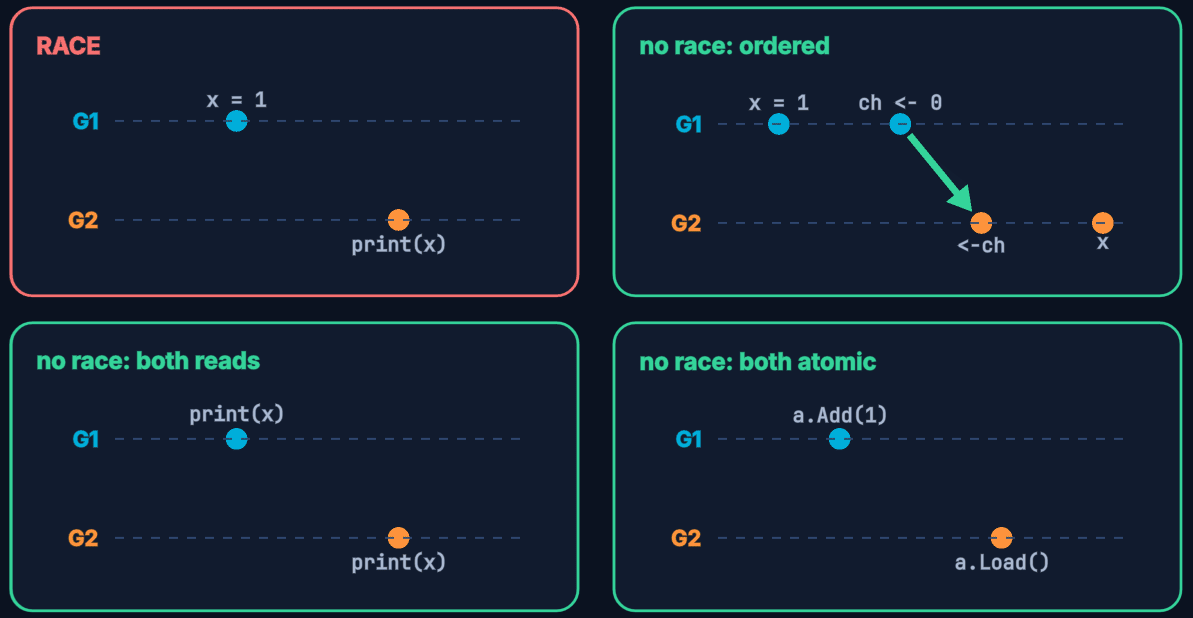
PITFALL A **failed** `TryLock` synchronizes nothing, and as far as the memory model is concerned it may fail even when the lock is free.

8.13 What a data race is, and what Go still promises

A race is defined by the happens-before relation, not by timing. A program can race even if the two accesses are seconds apart.

DEFINITION

A **data race** is a write to a memory location and another read or write of the same location, at least one of them not atomic, with **neither happening before the other**.



IF A PROGRAM RACES ANYWAY

- The implementation may **report the race and stop** the program. The race detector does exactly that.
- Otherwise, a read of a **word-sized** value sees some value that was really written to it and not yet overwritten. No values out of thin air.
- Values **larger than a word** (strings, slices, interfaces, structs) may be read half from one write and half from another.
- A torn (pointer, length) or (type, pointer) pair can cause **arbitrary memory corruption**: the one place where Go loses memory safety.

NOTE

This makes Go more like Java or JavaScript, where most races have a limited set of outcomes, and less like C and C++, where a racy program has no meaning at all. A race is still a bug.

8.13 Measured: torn reads of a slice header

A slice value is three machine words. Assigning one is three separate stores, so a racing reader can see a mix of two slices.

examples/ch13_torn/main.go — Go

```go
var s []int // a slice header is 3 words: pointer, length, capacity

//go:noinline
func load() []int { return s } // one racy read of all 3 words

func main() {
    short, long := make([]int, 1), make([]int, 1000)
    var stop atomic.Bool
    go func() { // writer: alternate between the two slices (racy!)
        for i := 0; !stop.Load(); i++ {
            if i%2 == 0 {
                s = short
            } else {
                s = long
            }
        }
    }()
    torn := 0
    for range 10_000_000 {
        v := load()
        if len(v) != cap(v) { // len from one slice, cap from the other
            torn++
        }
    }
    stop.Store(true)
    fmt.Println("torn reads:", torn, "of 10,000,000")
}
```

go run . — terminal

```
$ go run .
torn reads: 3345202 of 10,000,000
```

WHY IT MATTERS

Here **3,345,202** of 10,000,000 reads mixed the two slices. A torn header with the pointer of the 1-element array and length 1000 lets `v[999]` read (or write) far past the end of the array: memory corruption, without `unsafe`.

PRO TIP

Interfaces are two words (type, data), strings two (ptr, len), slices three. Share them through a channel, a mutex or `atomic.Pointer`, never through a plain variable.

8.13 Broken idiom 1: spinning on a plain flag

Waiting in a loop for another goroutine to set a bool looks harmless. The memory model says the loop is not even guaranteed to end.

examples/ch13_busywait/main.go — Go

```go
var done bool // plain variable: NOT synchronized

func main() {
    go func() {
        time.Sleep(10 * time.Millisecond)
        done = true // write
    }()
    for !done { // read in a loop: the compiler may load it only once
    }
    fmt.Println("finished")
}
```

go run -race . (first lines) — terminal

```
$ go run -race .
finished
==================
WARNING: DATA RACE
Write at 0x00000060a048 by goroutine 8:
  main.main.func1()
      ./main.go:13 +0x30

Previous read at 0x00000060a048 by main goroutine:
  main.main()
      ./main.go:15 +0x33
```

you wrote — Go

```go
for !done {
}
```

→

compiler may emit — Go

```go
if !done {
    for {}
}
```

Nothing in the loop synchronizes, so the compiler may read `done` once and keep it in a register. The memory model's own example allows hoisting a load out of a loop like this.

examples/ch13_busywait_atomic/main.go (fixed) — Go

```go
var done atomic.Bool // synchronized

func main() {
    go func() {
        time.Sleep(10 * time.Millisecond)
        done.Store(true) // synchronized before the Load that sees it
    }()
    for !done.Load() {
    }
    fmt.Println("finished")
}
```

NOTE

With Go 1.27 the racy loop happened to finish on this machine; the race detector still flags it. Better than any spin: `close` a channel or call `wg.Wait()`, which also sleep instead of burning a core.

8.13 Broken idiom 2: double-checked locking

Checking a pointer without a lock to skip the lock "when it's already set" was a classic in C++ and Java. In Go it is a race, twice.

examples/ch13_dcl/main.go — Go

```go
var (
    mu       sync.Mutex
    instance *Config
)

// BROKEN "double-checked locking": the first check reads
// instance without any synchronization.
func get() *Config {
    if instance == nil { // racy read                          1
        mu.Lock()
        if instance == nil {
            instance = &Config{Name: "prod"} // write under the lock   W
        }
        mu.Unlock()
    }
    return instance
}

func main() {
    var wg sync.WaitGroup
    for range 4 {
        wg.Go(func() { _ = get().Name })                       2
    }
    wg.Wait()
    fmt.Println(get().Name)
```

go run -race . (abridged) — terminal

```
$ go run -race .
prod
==================
WARNING: DATA RACE
Read at 0x0000005ebc00 by goroutine 11:
  main.get()
      ./main.go:18 +0x27
Previous write at 0x0000005ebc00 by goroutine 10:
  main.get()
      ./main.go:21 +0xa7
  ...
Found 2 data race(s)
```

1 **Race 1:** line 18 reads `instance` with no lock while another goroutine writes it at line 21.

2 **Race 2:** line 31 reads `.Name`. Seeing the pointer does **not** mean seeing the fields written before it was stored (the memory model's own example).

fixed: let sync.OnceValue do it — Go

```go
var get = sync.OnceValue(func() *Config {
    return &Config{Name: "prod"}
})
```

The detector found **2** races in one short run. `OnceValue` does the fast check correctly: an atomic load first, the lock only on the slow path.

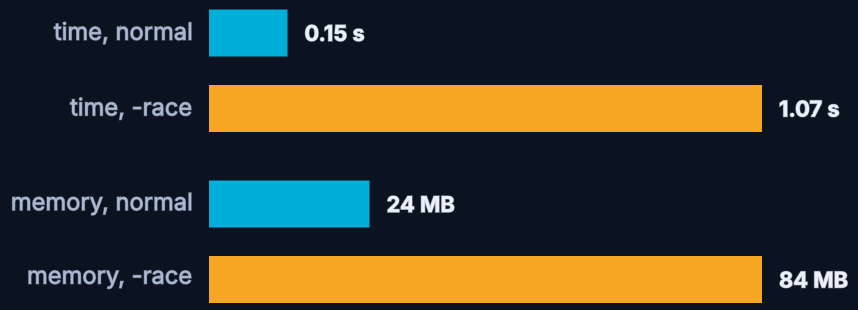
8.13 The race detector: -race

Built on ThreadSanitizer from the LLVM project. It watches a real run and reports every pair of accesses that happened without a happens-before edge.

command	use it for
`go test -race ./...`	every test run in CI
`go run -race .`	trying a program
`go build -race`	a canary or staging binary under real load

1. **Instrument.** The compiler inserts a call before every memory access and around every channel op, lock, atomic and `go` statement.
2. **Track.** Shadow memory records recent accesses to each 8-byte word: which goroutine, read or write, at what logical time.
3. **Order.** Vector clocks follow the synchronizing operations: that is the happens-before graph.
4. **Report.** An access that conflicts with a recorded one not ordered by happens-before is a race: print both stacks, keep going, exit with status 66.

MEASURED COST (examples/ch13_cost)



7.1× slower, 3.4× the memory for this workload (goroutines, a mutex, sorting)

LIMITS

It finds only races that **happen** in the run it watches: code your tests never execute is never checked. Reports are real (no false positives), but silence proves nothing.

OPTIONS AND PLATFORMS

`GORACE="halt_on_error=1"` stops at the first race; `log_path=race` writes reports to files; `exitcode=N` changes 66. Supported on linux (amd64, arm64, loong64, ppc64le, riscv64, s390x), darwin (amd64, arm64), and freebsd, netbsd, windows on amd64.

8.13 Reading a race report

Every report has the same four parts. Read them in order and the bug is usually obvious.

examples/ch12_race_detect/main.go — Go

```go
func main() {
    balance := 100
    var wg sync.WaitGroup
    wg.Go(func() { balance += 50 }) // write
    wg.Go(func() { balance -= 20 }) // concurrent write
    wg.Wait()
    fmt.Println("balance:", balance)
}
```

go run -race . — terminal

```
$ go run -race .
balance: 130
==================
WARNING: DATA RACE
Read at 0x00c00009c028 by goroutine 9:
  main.main.func2()
      ./main.go:12 +0x2e

Previous write at 0x00c00009c028 by goroutine 8:
  main.main.func1()
      ./main.go:11 +0x44

Goroutine 9 (running) created at:
  sync.(*WaitGroup).Go()
      $GOROOT/src/sync/waitgroup.go:238 +0x72

Goroutine 8 (finished) created at:
  sync.(*WaitGroup).Go()
      $GOROOT/src/sync/waitgroup.go:238 +0x72
==================
Found 1 data race(s)
exit status 66
```

1 — 2 — 3 — 4

1. **The access that tripped it:** a read (or write), its address, the goroutine, and the stack down to `file:line`.
2. **The earlier conflicting access:** "Previous write at" the same address, from another goroutine, with no happens-before edge between the two.
3. **Where each goroutine was born:** the `go` statement (here inside `WaitGroup.Go`), and whether it is still running.
4. **Summary and exit status 66.** The program kept running: the result 130 was luck. The order of the two goroutines changes from run to run.

PRO TIP

Fix the **pair**: find the line in part 1 and the line in part 2 and add one happens-before edge between them (a lock around both, a channel, or an atomic).

8.13 Race-free by habit: common races and their fixes

Nearly every race in real code is one of a handful of shapes. Each has a standard fix.

the race	the fix
many goroutines do `count++`	`atomic.Int64`, or a Mutex (8.12)
a goroutine polls a plain `done` flag	`close(ch)`, `wg.Wait()`, or `atomic.Bool`
lazy init behind an unlocked nil check	`sync.OnceValue`
a config struct replaced while being read	`atomic.Pointer[T]`, copy-on-write
several goroutines write one map	Mutex around the map, or `sync.Map`
main reads results a goroutine wrote	receive them from a channel, or `wg.Wait()` first
a slice appended to from many goroutines	one slot per goroutine, or a Mutex
`go func(){ use(i) }()` in a loop before Go 1.22	Go 1.22+ gives each iteration its own `i`

> *If you must read the rest of this document to understand the behavior of your program, you are being too clever. Don't be clever.*
>
> — The Go Memory Model

MAPS CHECK THEMSELVES

The runtime also catches some map races by itself, cheaply and always on, by crashing:

examples/ch13_map_race — terminal

```
$ go run .
fatal error: concurrent map writes
```

- ✓ Run `go test -race ./...` in CI on every change.
- ✓ Write tests that really run things concurrently: `t.Parallel()`, many goroutines, `-count=20`.
- ✓ Run load tests against a `-race` build.

8.13 Recap and check yourself

Synchronize every shared access, and let the race detector check that you did.

REMEMBER

- Compilers, CPUs and caches reorder memory; we measured a "0, 0" no interleaving allows.
- Happens-before = sequenced-before (one goroutine) + synchronized-before (channels, locks, atomics, `go`, Once, WaitGroup).
- A goroutine's exit synchronizes nothing; buffered channels only order send → receive.
- Race-free programs are sequentially consistent (DRF-SC).
- Racy multiword values can tear: we measured mixed slice headers.
- `-race` finds races in the code you run, at several times the time and memory.

CHECK YOURSELF

Q1. A goroutine sets `result = 42`, and `main` sleeps 1 s before reading it. Race?
Answer. Yes. Sleeping creates no happens-before edge, however long it is.

Q2. Unbuffered `ch`: G1 writes x then does `<-ch`; G2 does `ch <- 0` then reads x. Safe?
Answer. Yes: the receive is synchronized before the send completes.

Q3. Same with `make(chan int, 1)`?
Answer. No. With a buffer the send can finish before the receive, so x is racy.

Q4. Your tests pass under `-race`. Is the program race-free?
Answer. Only the paths the tests ran were checked. Silence proves nothing about the rest.

CHAPTER

8.14

Cancellation with context

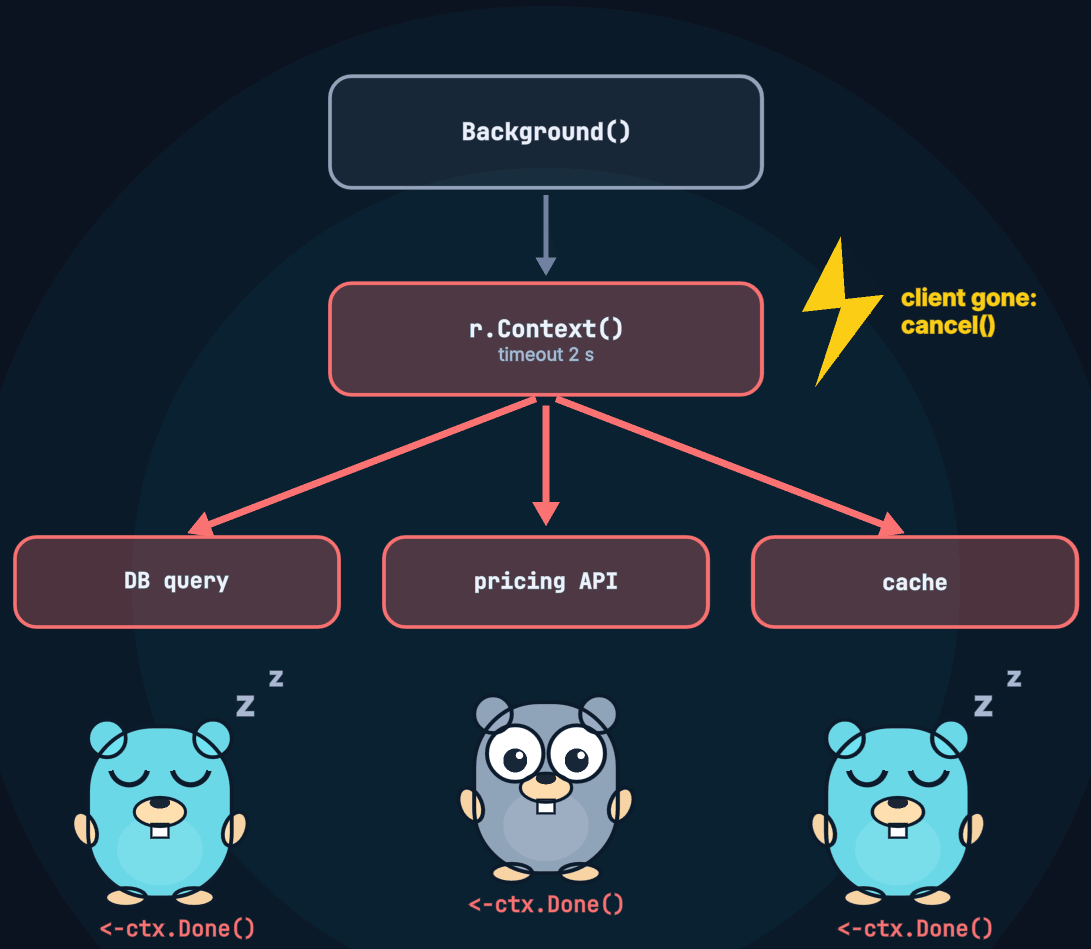
Goroutines are cheap to start and easy to forget. A context.Context carries one message to every goroutine working on a request: stop now, and here is why.

IN THIS CHAPTER YOU WILL

- ✓ Give work a deadline with `WithTimeout`, and always `cancel`
- ✓ Cancel whole trees of goroutines from one place
- ✓ Look inside: `cancelCtx`, `timerCtx`, `valueCtx`
- ✓ Make loops, channel code and library calls cancellable
- ✓ Use causes, `AfterFunc`, `WithoutCancel` and values well

? THE BIG QUESTION

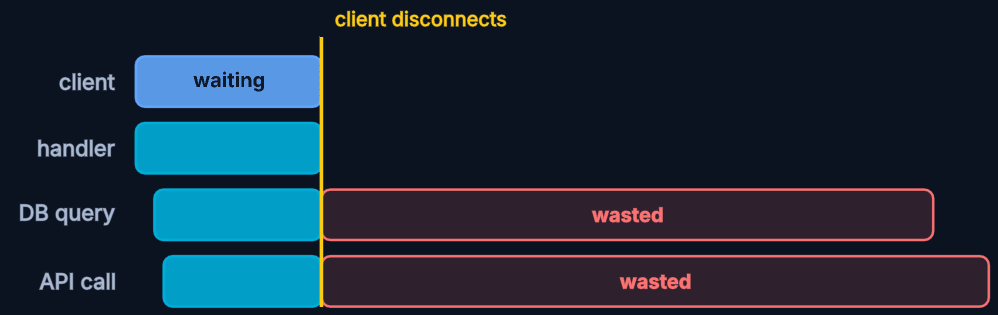
A user closes the browser tab. The handler has three goroutines running: a database query, a call to a pricing service and a cache lookup. How do all three find out, at once?



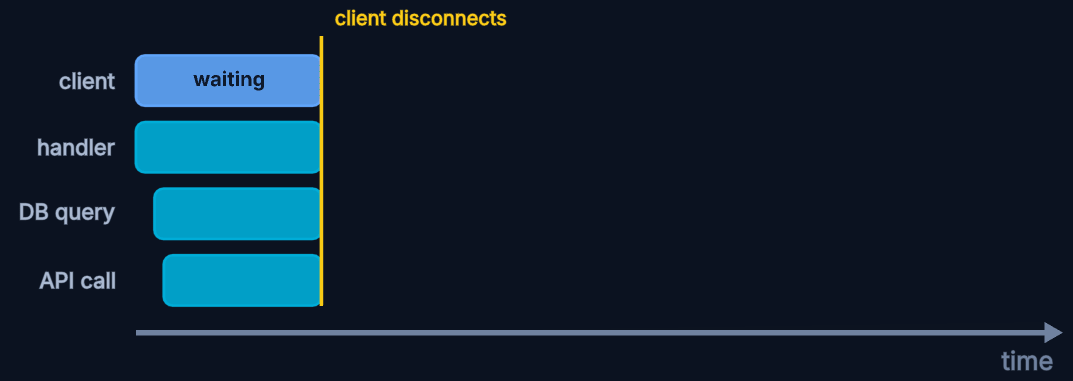
8.14 The problem: work that outlives its purpose

A request spawns goroutines. If the client leaves or a deadline passes, nothing in Go stops them unless you pass a signal down.

Without a context: goroutines keep running



With a context: one cancel stops them all



KEY IDEA

Go has **no way to kill a goroutine** from outside. Stopping is cooperative: you pass a signal down, each goroutine checks it and returns. `context` is that signal.

context.Context (package context) — Go

```go
type Context interface {
    Deadline() (deadline time.Time, ok bool)
    Done() <-chan struct{}
    Err() error
    Value(key any) any
}
```

method	answers
`Done()`	a channel that is **closed** when the work should stop
`Err()`	nil, then `Canceled` or `DeadlineExceeded`
`Deadline()`	when it will be cancelled automatically, if ever
`Value(key)`	request-scoped data: request ID, user, trace span

GO VERSION NOTE

Added to the standard library in Go 1.7 (2016), from golang.org/x/net/context. Now it is the first parameter of almost every blocking API.

8.14 WithTimeout: a time budget for everything below

A deadline is a budget for the whole operation, not for one call. Every function that receives the context shares what is left.

examples/ch14_timeout/main.go — Go

```go
// query honours cancellation: it returns as soon as ctx is done.
func query(ctx context.Context, d time.Duration) (string, error) {
    select {
    case <-time.After(d): // pretend work
        return "rows", nil
    case <-ctx.Done():
        return "", ctx.Err()
    }
}

func main() {
    ctx, cancel := context.WithTimeout(context.Background(), 100*time.Millisecond)
    defer cancel() // always release the timer and the child links

    r, err := query(ctx, 30*time.Millisecond)
    fmt.Printf("fast query: %q err=%v\n", r, err)

    r, err = query(ctx, 500*time.Millisecond)
    fmt.Printf("slow query: %q err=%v\n", r, err)
    deadline, _ := ctx.Deadline()
    fmt.Println("deadline was set:", !deadline.IsZero())
}
```

go run . — terminal

```
$ go run .
fast query: "rows" err=<nil>
slow query: "" err=context deadline exceeded
deadline was set: true
```

1. **select** on the work and on `ctx.Done()`: whichever happens first wins.
2. **WithTimeout** = WithDeadline(now + d). It starts a timer that cancels the context.
3. **defer cancel()** releases the timer and the parent's reference **now**, not at the deadline. Always call it, even on success.

go vet catches a discarded cancel (lostcancel) — terminal

```
$ go vet .
main.go:12:7: the cancel function returned by context.WithTimeout should be
    called, not discarded, to avoid a context leak
```

8.14 The context tree: cancellation flows down

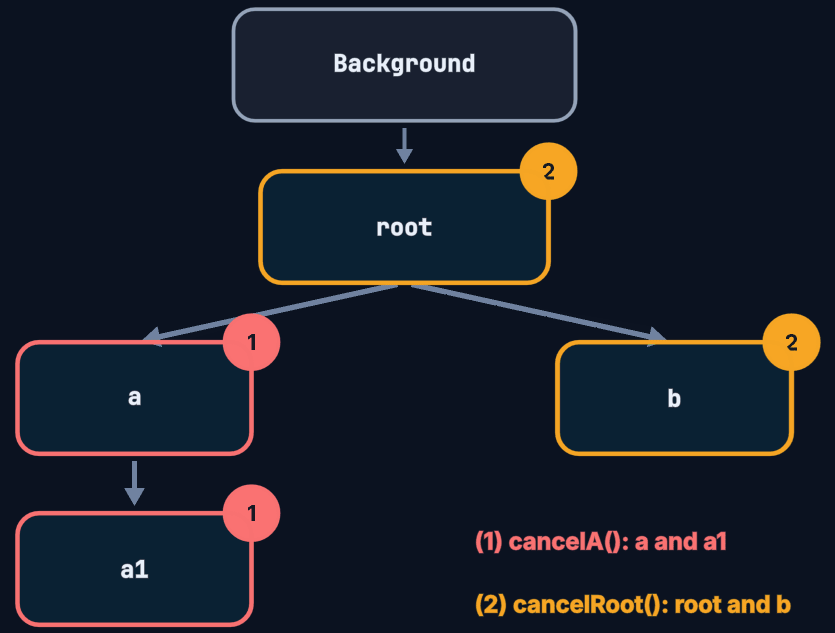
Every derived context is a child of its parent. Cancelling a node cancels its whole subtree, and nothing above it.

examples/ch14_tree/main.go — Go

```go
func worker(ctx context.Context, name string, wg *sync.WaitGroup) {
    defer wg.Done()
    <-ctx.Done() // block until this context is cancelled
    fmt.Printf("%-12s stopped: %v\n", name, ctx.Err())
}

func main() {
    root, cancelRoot := context.WithCancel(context.Background())
    a, cancelA := context.WithCancel(root)
    b, cancelB := context.WithCancel(root)
    a1, cancelA1 := context.WithCancel(a)
    defer cancelB()
    defer cancelA1()

    var wg sync.WaitGroup
    for name, ctx := range map[string]context.Context{"a": a, "b": b, "a1 (child)": a1} {
        wg.Add(1)
        go worker(ctx, name, &wg)
    }

    fmt.Println("cancel a  -> a and its child a1 stop; b keeps running")
    cancelA()                                                          (1)
    time.Sleep(50 * time.Millisecond)
    fmt.Println("cancel root -> everything left stops")
    cancelRoot()                                                       (2)
    wg.Wait()
```

go run . — terminal

```
$ go run .
cancel a  -> a and its child a1 stop; b keeps running
a           stopped: context canceled
a1 (child)  stopped: context canceled
cancel root -> everything left stops
b           stopped: context canceled
```



- Cancel flows **down** only: cancelling a never touches root or b.
- A child's deadline can only be **earlier**. Asking for a later one than the parent's just returns a plain `WithCancel` child (context.go).
- Each `With...` call returns a new node; the parent is never modified.

8.14 Inside package context: three small structs

A context is a linked list of nodes pointing at their parents. Cancellation is a mutex, a closed channel and a set of children.

context/context.go (abridged) — Go

```go
type cancelCtx struct {
    Context                 // the parent
    mu       sync.Mutex
    done     atomic.Value   // chan struct{}, made lazily
    children map[canceler]struct{}
    err      atomic.Value   // set by the first cancel
    cause    error
}

type timerCtx struct {
    cancelCtx
    timer    *time.Timer    // fires cancel(DeadlineExceeded)
    deadline time.Time
}

type valueCtx struct {
    Context                 // the parent
    key, val any
}
```

cancel(err, cause), once

1. **Lock** `mu`; if `err` is already set, return: cancel is idempotent.
2. **Record** `err` and `cause`.
3. **Close** `done` (or store a shared, already-closed channel if nobody asked for Done yet).
4. **Recurse** into every child, then drop the children map.
5. **Detach** from the parent's children map.

How a child links to its parent

parent is...	the child...
never cancelled (`Background`)	links nothing
a `cancelCtx` (the usual case)	joins its `children` map
a custom type with `AfterFunc`	registers a callback
any other custom Context	costs a goroutine that waits on both

★ **KEY IDEA**

`Done()` is just a channel that gets **closed**: the close broadcast from 8.5. That is why one cancel wakes any number of goroutines, and why selecting on it is cheap.

8.14 Making code cancellable: three shapes

A context only asks. Your code has to look: in loops, in every blocking channel operation, and by passing ctx to library calls.

examples/ch14_loop/main.go — Go

```go
// sumSquares is CPU-bound: it polls ctx.Err() every 65,536 steps.
func sumSquares(ctx context.Context, n int) (int, error) {
    sum := 0
    for i := range n {
        if i%(1<<16) == 0 {
            if err := ctx.Err(); err != nil {
                return 0, fmt.Errorf("stopped at step %d: %w", i, err)
            }
        }
        sum += i * i
    }
    return sum, nil
```

1

examples/ch14_loop/main.go — Go

```go
// produce blocks on sends, so it selects on ctx.Done() as well.
func produce(ctx context.Context, out chan<- int) {
    defer close(out)
    for i := 0; ; i++ {
        select {
        case out <- i:
        case <-ctx.Done():
            return
        }
    }
}
```

2

go run . — terminal

```
$ go run .
stopped at step 36831232: context deadline exceeded
is DeadlineExceeded: true
got 0
got 1
got 2
producer stopped and closed its channel
```

code that...	becomes cancellable by...
① computes in a loop	checking `ctx.Err()` every N steps (cheap: one atomic load)
② blocks on a channel	a `select` with `case <-ctx.Done():`
③ calls a library	passing ctx: `http.NewRequestWithContext`, `db.QueryContext`, `exec.CommandContext`, `(*net.Dialer).DialContext`
sleeps	`select` on a timer and `ctx.Done()`, not `time.Sleep`

PRO TIP

Return `ctx.Err()` (or wrap it with `%w`) so callers can test `errors.Is(err, context.DeadlineExceeded)`.

8.14 Saying why: WithCancelCause and Cause

Err tells you that a context ended and how. Since Go 1.20 a cause can tell you why, without changing what Err returns.

examples/ch14_cause/main.go — Go

```go
var errQuota = errors.New("tenant quota exceeded")

func main() {
    ctx, cancel := context.WithCancelCause(context.Background())
    cancel(errQuota)                        // cancel AND record why
    fmt.Println("Err:  ", ctx.Err())        // always context.Canceled
    fmt.Println("Cause:", context.Cause(ctx)) // the reason we gave

    tctx, tcancel := context.WithTimeoutCause(context.Background(),
        20*time.Millisecond, errors.New("payment API too slow"))
    defer tcancel()
    <-tctx.Done()
    fmt.Println("Err:  ", tctx.Err())
    fmt.Println("Cause:", context.Cause(tctx))
}
```

go run . — terminal

```
$ go run .
Err:   context canceled
Cause: tenant quota exceeded
Err:   context deadline exceeded
Cause: payment API too slow
```

ctx.Err()

What happened: only `Canceled` or `DeadlineExceeded`. A fixed set, safe for control flow and `errors.Is`.

context.Cause(ctx)

Why: the error you passed to cancel. Made for logs, metrics and error messages. Without a cause it equals `Err()`.

plain (Go 1.7)	with a cause
`WithCancel`	`WithCancelCause` (1.20)
`WithDeadline`	`WithDeadlineCause` (1.21)
`WithTimeout`	`WithTimeoutCause` (1.21)

★ **KEY IDEA**

The cause travels down the tree with the cancellation: every descendant reports the same `Cause`. The **first** cancellation wins; later calls change nothing.

8.14 AfterFunc and WithoutCancel (Go 1.21)

Two tools for the edges of a request: run cleanup when a context ends, and keep working after it ends.

examples/ch14_afterfunc/main.go — Go

```go
    ctx, cancel := context.WithCancel(context.Background())

    // Run cleanup in its own goroutine when ctx is done (Go 1.21+).
    stop := context.AfterFunc(ctx, func() {
        fmt.Println("AfterFunc: closing the connection")
    })
    _ = stop // stop() would unregister it if called before cancel

    fmt.Println("working...")
    cancel()
    time.Sleep(20 * time.Millisecond)

    ctx2, cancel2 := context.WithCancel(context.Background())
    stop2 := context.AfterFunc(ctx2, func() { fmt.Println("never printed") })
    fmt.Println("stop() before cancel:", stop2()) // true: unregistered
    cancel2()
    time.Sleep(20 * time.Millisecond)
```

(1) (2)

go run . — terminal

```
$ go run .
working...
AfterFunc: closing the connection
stop() before cancel: true
```

(1) `AfterFunc(ctx, f)` runs f **in its own goroutine** once ctx is done.
(2) `stop()` unregisters f; it returns true if f had not started.

examples/ch14_withoutcancel/main.go — Go

```go
func main() {
    base := context.WithValue(context.Background(), key{}, "req-42")
    req, cancel := context.WithCancel(base)
    audit := context.WithoutCancel(req) // keeps values, drops cancellation
    cancel()                            // the request is finished

    fmt.Println("request ctx err:", req.Err())
    fmt.Println("audit ctx err:  ", audit.Err())
    fmt.Println("audit sees value:", audit.Value(key{}))
    select {
    case <-audit.Done():
        fmt.Println("audit cancelled")
    case <-time.After(10 * time.Millisecond):
        fmt.Println("audit log written after the request ended")
    }
}
```

go run . — terminal

```
$ go run .
request ctx err: context canceled
audit ctx err:   <nil>
audit sees value: req-42
audit log written after the request ended
```

- `WithoutCancel(ctx)` keeps the **values** but drops cancellation and deadline.
- Use it for work that must outlive the request: audit logs, async cleanup.

8.14 Context values: request-scoped data only

WithValue adds one key/value node to the chain. Value(key) walks up the chain until it finds the key.

examples/ch14_value/main.go — Go

```go
// An unexported key type: no other package can collide with it.
type ctxKey int

const requestIDKey ctxKey = 0

func WithRequestID(ctx context.Context, id string) context.Context {
    return context.WithValue(ctx, requestIDKey, id)
}

func RequestID(ctx context.Context) (string, bool) {
    id, ok := ctx.Value(requestIDKey).(string)
    return id, ok
}

func handle(ctx context.Context) {
    if id, ok := RequestID(ctx); ok {
        fmt.Println("handling request", id)
    }
}

func main() {
    ctx := WithRequestID(context.Background(), "7f3a-91c2")
    handle(ctx)
    _, ok := RequestID(context.Background())
    fmt.Println("background has an id:", ok)
}
```

1

2

Value(requestIDKey):
asks each node, child to parent,
until a key matches.
O(depth) per lookup.

✖ DON'T

```go
ctx = WithValue(ctx, "db", pool)
ctx = WithValue(ctx, "verbose", true)
```

dependencies and optional parameters; string keys collide

✓ DO

```go
ctx = WithRequestID(ctx, id)
id, ok := RequestID(ctx)
```

request-scoped data, typed accessors, unexported key type

go run . — terminal

```
$ go run .
handling request 7f3a-91c2
background has an id: false
```

8.14 Rules of the road, and where context already lives

The package documentation states the conventions; the standard library follows them everywhere.

THE CONVENTIONS (package context docs)

- Pass a Context **explicitly, as the first parameter**, named `ctx`.
- Do **not store** Contexts inside a struct type.
- Never pass a **nil** Context. If unsure, pass `context.TODO()`.
- Use Values only for request-scoped data that crosses processes and APIs.
- One Context may be used by many goroutines at once: it is safe.
- Always call the `CancelFunc`: `go vet` reports a discarded one.
- `Background()` at the top (main, init, tests); `TODO()` as a placeholder.

★ **KEY IDEA**

Context answers "should this work continue?". It does not stop anything by force: a goroutine that never checks `ctx.Done()` or `ctx.Err()` runs on.

IN THE STANDARD LIBRARY

package	what takes or gives a ctx
`net/http`	`r.Context()`: cancelled when the client disconnects or ServeHTTP returns; `NewRequestWithContext` for clients
`database/sql`	`QueryContext`, `ExecContext`, `BeginTx`
`os/exec`	`CommandContext`: kills the process on cancel
`net`	`Dialer.DialContext`, `Resolver` lookups
`os/signal`	`NotifyContext` (1.16): cancelled on Ctrl-C
`testing`	`t.Context()` (1.24): cancelled when the test ends
`x/sync/errgroup`	`WithContext`: cancelled on the first error (8.4)

8.14 Recap and check yourself

One tree of contexts per request; cancel at the top, listen at every blocking point.

REMEMBER

- ✓ `Done()` is a channel closed on cancel: one close wakes every listener.
- ✓ `WithTimeout`/`WithDeadline` set a budget for everything below; `defer cancel()` always.
- ✓ Cancellation flows down the tree; children can only shorten deadlines.
- ✓ Inside: `cancelCtx` (mutex, lazy done channel, children map), `timerCtx`, `valueCtx`.
- ✓ Listen in loops (`ctx.Err()`), in selects (`<-ctx.Done()`), and pass ctx to every call.
- ✓ `Cause` explains why; `AfterFunc` cleans up; `WithoutCancel` outlives the request.
- ✓ Values: request-scoped only, unexported key types, typed accessors.

CHECK YOURSELF

Q1. A parent has a 1 s deadline; you call `WithTimeout(parent, 5*time.Second)`. When does the child expire?

Answer. After at most 1 s: a child can never outlive its parent's deadline.

Q2. Your goroutine does `time.Sleep(10 * time.Second)` and the context is cancelled. What happens?

Answer. Nothing until the sleep ends. Select on a timer and `ctx.Done()` instead.

Q3. Why is it safe to call a `CancelFunc` twice?

Answer. cancel checks `err` under the mutex and returns if it is already set.

Q4. Where should a database handle go: in ctx, or a parameter/field?

Answer. A parameter or struct field. Context values are for request-scoped data.

CHAPTER

8.15

Concurrency Patterns I: Pipelines and Pools

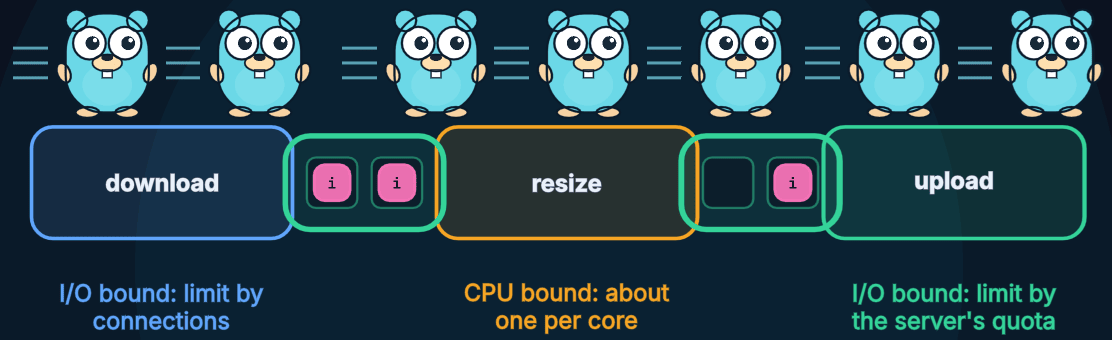
Goroutines and channels are building blocks. A handful of patterns built from them solve most real problems: generators, pipelines, fan-out and fan-in, worker pools and semaphores.

IN THIS CHAPTER YOU WILL

- ✓ Write generators, and know when an iterator is better
- ✓ Chain stages into a cancellable pipeline
- ✓ Fan work out to many goroutines and fan results back in
- ✓ Size a worker pool, and bound concurrency with a semaphore
- ✓ Keep results in order without a lock

? **THE BIG QUESTION**

10,000 images must be downloaded, resized and uploaded. How many goroutines should you start, and how do they hand work to each other?



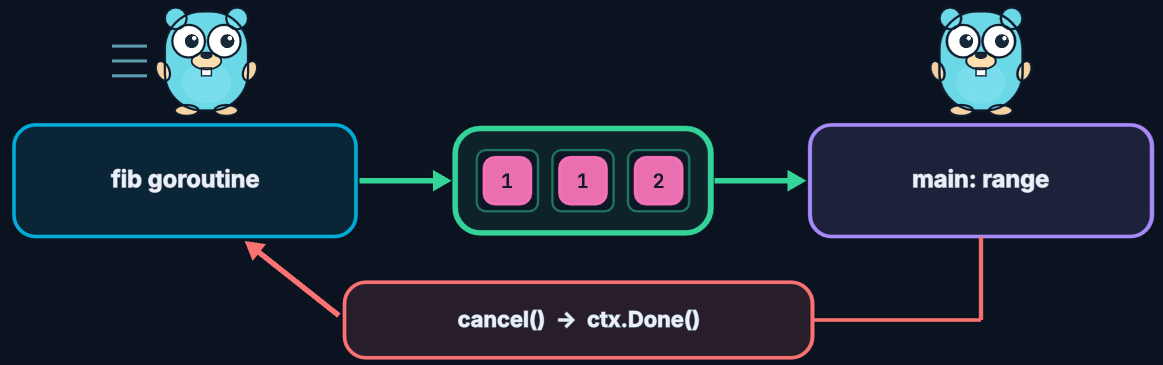
stages run concurrently; channels connect them

8.15 Generator: a goroutine that produces a stream

A function starts a goroutine and returns the receive end of a channel. The caller ranges over the channel as values arrive.

examples/ch15_generator/main.go — Go

```go
// fib returns a channel that yields Fibonacci numbers until ctx is done.
func fib(ctx context.Context) <-chan int {
    out := make(chan int)
    go func() {
        defer close(out)
        a, b := 0, 1
        for {
            select {
            case out <- a:
                a, b = b, a+b
            case <-ctx.Done(): // consumer left: stop, do not leak
                return
            }
        }
    }()
    return out
}

func main() {
    ctx, cancel := context.WithCancel(context.Background())
    defer cancel()
    for v := range fib(ctx) {
        if v > 100 {
            break // cancel() (deferred) stops the generator goroutine
        }
        fmt.Print(v, " ")
    }
    fmt.Println()
}
```



1. **Return a receive-only channel.** `<-chan int`: callers can only receive; the generator owns it.
2. **The owner closes it.** `defer close(out)`, so `range` in the caller ends cleanly.
3. **Select on the context.** If the caller stops early, the goroutine is not stuck on a send forever.
4. **Break, then cancel.** The deferred `cancel()` is the consumer's way to say "enough".

> **PITFALL**
> Without the ctx case, `break` leaves the generator blocked on `out <- a` forever: a **goroutine leak** (8.17). Every generator needs a way to be told to stop.

`go run .` prints `0 1 1 2 3 5 8 13 21 34 55 89`

8.15 Generator or iterator? Measure first

Go 1.23 range-over-func iterators produce sequences without goroutines or channels. For plain sequences they win by a mile.

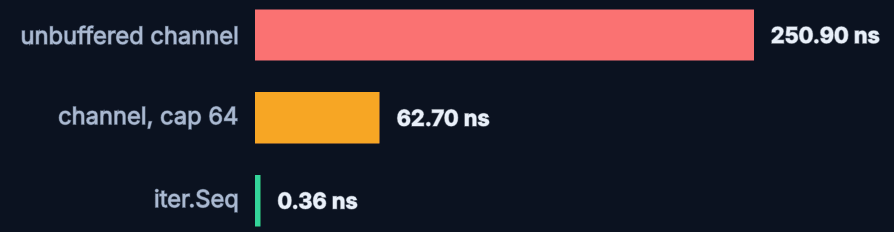
examples/ch15_iter/main.go — Go

```go
// Same sequence with a Go 1.23 iterator: no goroutine, no channel.
func fib() iter.Seq[int] {
    return func(yield func(int) bool) {
        a, b := 0, 1
        for yield(a) { // yield returns false when the loop breaks
            a, b = b, a+b
        }
    }
}

func main() {
    for v := range fib() {
        if v > 100 {
            break
        }
        fmt.Print(v, " ")
    }
    fmt.Println()
}
```

go run . — terminal

```
$ go run .
0 1 1 2 3 5 8 13 21 34 55 89
```

COST PER VALUE (examples/ch15_genbench)



the iterator is about **691×** cheaper: the compiler inlines the loop body; a channel needs two goroutines to meet for every value

	channel generator	iter.Seq
extra goroutine	yes	no
leaks if caller stops early	yes, unless ctx	no: yield returns false
producer runs in parallel	yes	no, it runs in the loop
good for	I/O, slow producers, many consumers	walking data, lazy sequences

★ **KEY IDEA** Use a channel when the producer should run **concurrently** with the consumer. For "give me the next value", use an iterator.

8.15 Pipeline: stages connected by channels

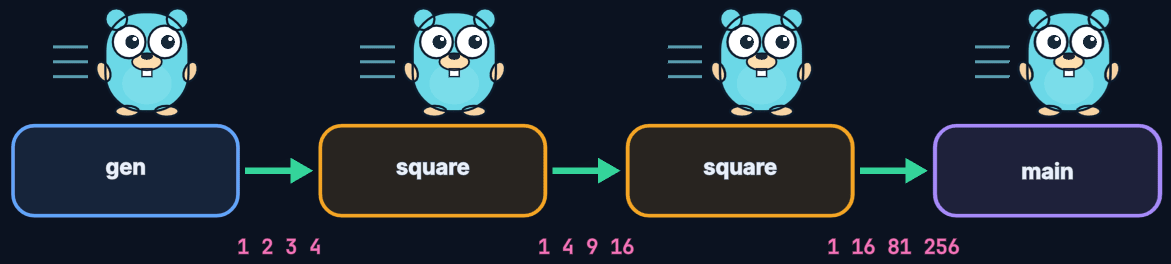
Each stage receives from an inbound channel, does one job, and sends on an outbound channel. All stages run at the same time.

examples/ch15_pipeline/main.go — Go

```
23 func square(ctx context.Context, in <-chan int) <-chan int {
24     out := make(chan int)
25     go func() {
26         defer close(out)
27         for n := range in {
28             select {
29             case out <- n * n:
30             case <-ctx.Done():
31                 return
32             }
33         }
34     }()
35     return out
36 }
37
38 func main() {
39     ctx, cancel := context.WithCancel(context.Background())
40     defer cancel()
41     // stage 1 -> stage 2 -> stage 2 again -> sink
42     for v := range square(ctx, square(ctx, gen(ctx, 1, 2, 3, 4))) {
43         fmt.Println(v)
44     }
45 }
```

1 2 3

gen (lines 8-21) has the same shape: it sends its arguments, then closes out.



1. **range over in:** the stage ends when its inbound channel is closed.
2. **defer close(out):** when a stage is done sending, it closes its outbound channel, which ends the next stage.
3. **select on ctx.Done():** if the consumer quits, every stage unblocks and returns. One cancel tears down the whole pipeline.

go run . — terminal

```
$ go run .
1
16
81
256
```

SOURCE

These rules come from the Go blog post "Go Concurrency Patterns: Pipelines and cancellation" (Sameer Ajmani, 2014): stages close their outbound channels when all sends are done, and keep receiving until inbound channels are closed or senders are unblocked.

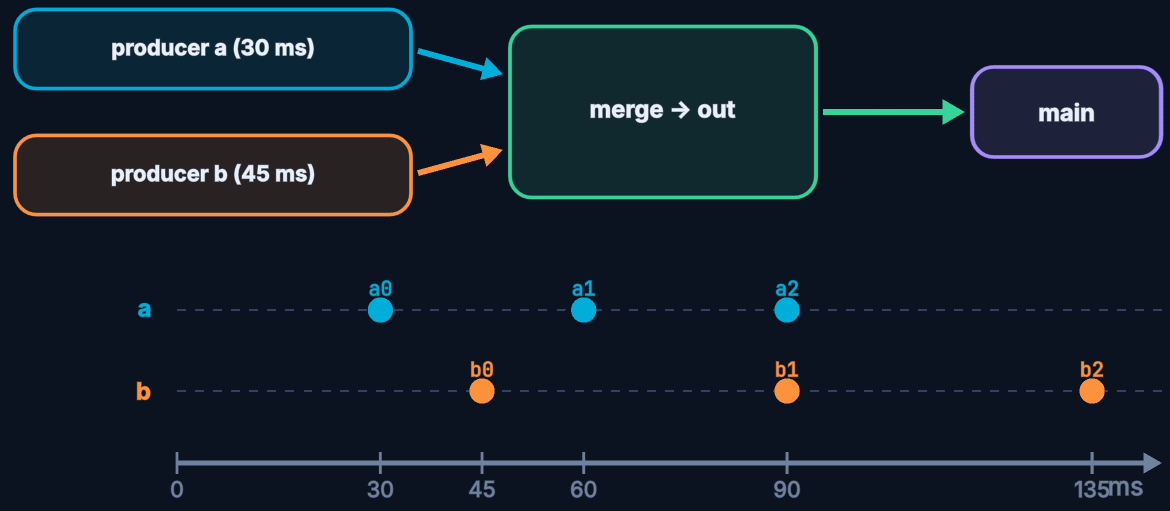
8.15 Fan-out and fan-in

Fan-out: several goroutines receive from one channel and share the work. Fan-in: several channels are merged into one.

examples/ch15_fanin/main.go — Go

```go
// merge fans in any number of channels into one.
func merge[T any](ins ...<-chan T) <-chan T {
    out := make(chan T)
    var wg sync.WaitGroup
    for _, in := range ins {
        wg.Go(func() {                          // 1
            for v := range in {
                out <- v
            }
        })
    }
    go func() { wg.Wait(); close(out) }() // close after ALL inputs end   // 2
    return out
}

func main() {
    all := merge(producer("a", 3, 30*time.Millisecond),
        producer("b", 3, 45*time.Millisecond))
    for v := range all {
        fmt.Print(v, " ")
    }
    fmt.Println()
}
```

1. one forwarding goroutine per input channel
2. a separate goroutine closes `out` after **all** forwarders finish; no forwarder may close it alone



go run . printed: a0 b0 a1 a2 b1 b2

KEY IDEA

Fan-in interleaves by **arrival time**, not by source. When order matters, carry an index with each value (see the ordered-results page).

DEFINITION

Fan-out is the mirror image: start N goroutines that all `range` over the same input channel. Each value goes to exactly one of them. The worker pool on the next page is fan-out plus fan-in.

8.15 Worker pool: a fixed team pulls from a queue

N long-lived goroutines range over a jobs channel. Whoever is free takes the next job, so fast workers simply do more.

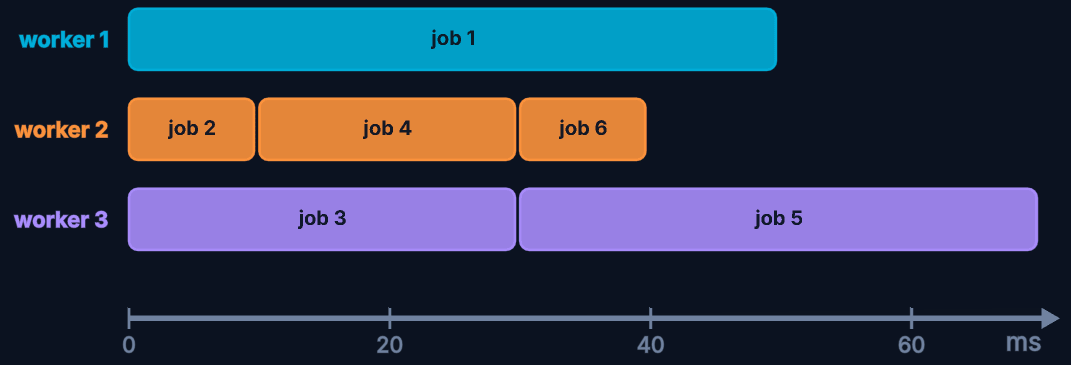
examples/ch15_workerpool/main.go — Go

```go
func worker(id int, jobs <-chan Job, results chan<- Result) {
    for j := range jobs { // each worker pulls the next free job
        time.Sleep(time.Duration(j.N) * 10 * time.Millisecond)
        results <- Result{j.ID, id, j.N * j.N}
    }
}

func main() {
    jobs := make(chan Job, 10)
    results := make(chan Result, 10)

    var wg sync.WaitGroup
    for w := 1; w <= 3; w++ { // a fixed pool of 3 workers
        wg.Go(func() { worker(w, jobs, results) })
    }
    for i, n := range []int{5, 1, 3, 2, 4, 1} {
        jobs <- Job{ID: i + 1, N: n}
    }
    close(jobs) // no more jobs: workers exit their range loops

    go func() { wg.Wait(); close(results) }()
    for r := range results {
        fmt.Printf("job %d -> %2d (worker %d)\n", r.JobID, r.Out, r.Worker)
    }
}
```

1. each worker loops until `jobs` is closed and drained
2. closing `jobs` is how main says "no more work"
3. close `results` only after every worker returned

go run . — terminal

```
$ go run .
job 2 ->  1 (worker 2)
job 3 ->  9 (worker 3)
job 4 ->  4 (worker 2)
job 6 ->  1 (worker 2)
job 1 -> 25 (worker 1)
job 5 -> 16 (worker 3)
```



job lengths 50, 10, 30, 20, 40, 10 ms; the busy-until times balance out by themselves

PRO TIP

How many workers? CPU-bound work: about `runtime.GOMAXPROCS(0)`. I/O-bound work: as many as the slowest shared resource allows (DB connections, API quota), found by measuring.

8.15 Semaphore: at most N at a time

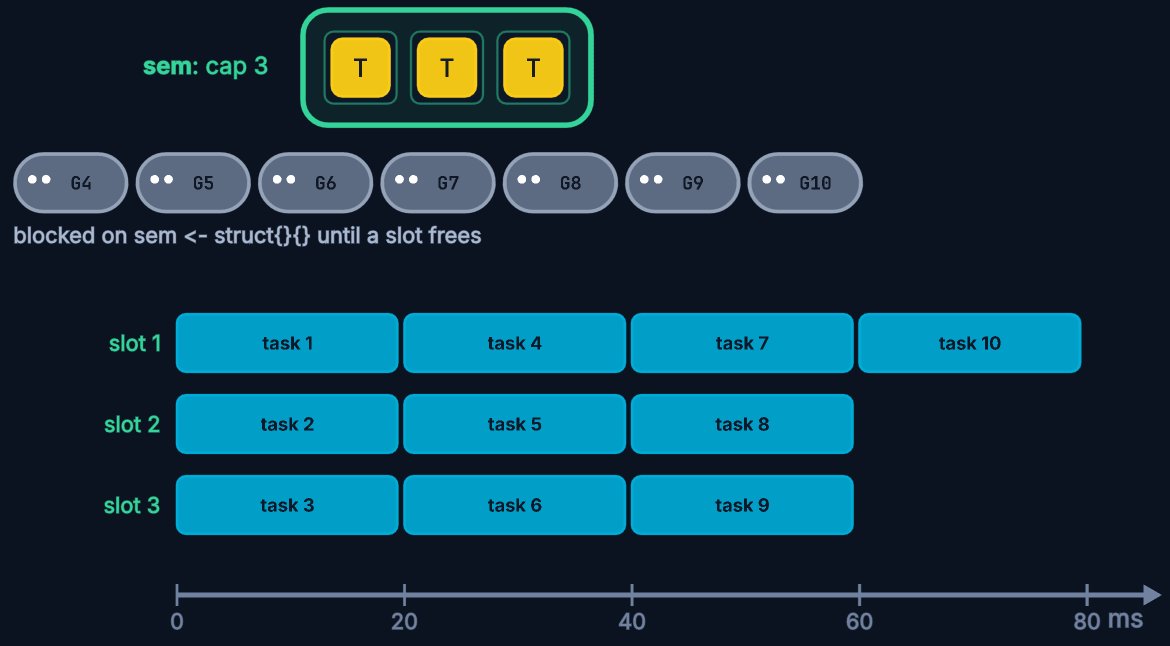
Sometimes you want one goroutine per task but a cap on how many run at once. A buffered channel of capacity N is a counting semaphore.

examples/ch15_semaphore/main.go — Go

```go
func main() {
    sem := make(chan struct{}, 3) // at most 3 at a time
    var running, peak atomic.Int32
    var wg sync.WaitGroup

    for range 10 {
        wg.Go(func() {
            sem <- struct{}{}        // acquire a slot (blocks when full)
            defer func() { <-sem }() // release it

            n := running.Add(1)
            for {
                p := peak.Load()
                if n <= p || peak.CompareAndSwap(p, n) {
                    break
                }
            }
            time.Sleep(20 * time.Millisecond) // the limited work
            running.Add(-1)
        })
    }
    wg.Wait()
    fmt.Println("tasks: 10, peak concurrency:", peak.Load())
}
```

go run . — terminal

```
$ go run .
tasks: 10, peak concurrency: 3
```



tool	when
`chan struct{}` with cap N	simplest; no extra package
`errgroup.Group.SetLimit(n)`	tasks can fail; cancel the rest on error
`semaphore.NewWeighted(n)`	tasks take different weights; `Acquire(ctx, w)`

8.15 Results in order, computed in any order

Goroutines finish in random order. Give each one its own slot in a pre-sized slice and the order takes care of itself.

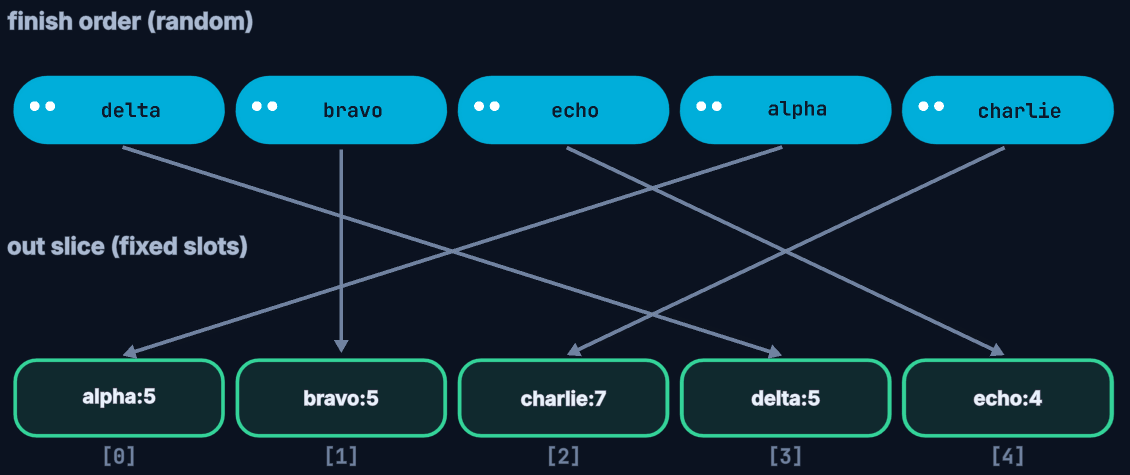
examples/ch15_ordered/main.go — Go

```go
func main() {
    inputs := []string{"alpha", "bravo", "charlie", "delta", "echo"}
    out := make([]string, len(inputs)) // result i goes to slot i   (1)

    var wg sync.WaitGroup
    for i, s := range inputs {
        wg.Go(func() {
            time.Sleep(time.Duration(rand.IntN(40)) * time.Millisecond)
            out[i] = fmt.Sprintf("%s:%d", s, len(s))   (2)
        })
    }
    wg.Wait() // finish order is random, but the output order is not   (3)
    fmt.Println(out)
}
```

go run . (same output every run) — terminal

```
$ go run .
[alpha:5 bravo:5 charlie:7 delta:5 echo:4]
```

1. **Pre-size** the result slice: one element per input.
2. **Write only out[i]:** every goroutine touches a different element, so there is no race and no lock.
3. **Wait, then read:** `wg.Wait()` is the happens-before edge that makes all writes visible.



✕ DON'T

```go
results = append(results, r) // from many goroutines
```

append reads and writes the shared slice header: a data race and lost results.

✓ DO

```go
out[i] = r // one slot per goroutine
```

distinct elements are distinct memory locations.

8.15 Patterns at a glance

Match the shape of the problem to a pattern. Each one has a rule that keeps it from leaking or racing.

pattern	use it when	shape	the rule that keeps it safe
generator	a stream produced by one goroutine	go f() → chan	owner closes; select on ctx.Done() when sending
iterator	a lazy sequence, consumed in a loop	iter.Seq	no goroutine: prefer it when no concurrency is needed
pipeline	work passes through steps	stage → stage → stage	close outbound when done; one cancel stops all stages
fan-out	spread one stream over many workers	1 chan → N goroutines	each value is received exactly once
fan-in	merge many streams into one	N chans → 1 chan	close the merged channel after all inputs end
worker pool	many jobs, a bounded team	jobs chan → N workers → results	close jobs to stop; close results after wg.Wait
semaphore	one goroutine per task, capped	chan struct{} with cap N	acquire before work, release in defer
errgroup	tasks that can fail together	g.Go(f); g.Wait()	first error cancels the shared ctx (8.4)
indexed results	output order must match input	out[i] = ...	each goroutine writes only its own slot

★ **KEY IDEA** Every pattern answers the same three questions: **who closes** each channel, **how does it stop** early, and **how many** goroutines run at once. Answer them before you write the code.

8.15 Recap and check yourself

Small, composable patterns; three questions for each: who closes, how it stops, how many run.

REMEMBER

- Generators return `<-chan T`, close it themselves, and watch `ctx.Done()`.
- Iterators (`iter.Seq`) are hundreds of times cheaper when no concurrency is needed.
- Pipelines: range over in, `defer close(out)`, select on ctx for every send.
- Fan-in needs a separate closer goroutine waiting on all forwarders.
- Worker pools: close jobs to stop; close results after `wg.Wait()`.
- A buffered `chan struct{}` of cap N is a semaphore; `errgroup.SetLimit` too.
- Write results into `out[i]`, never `append` from many goroutines.

CHECK YOURSELF

Q1. In a fan-in, why can't each forwarder `close(out)` when its input ends?

Answer. The others may still send: a send on a closed channel panics. Close once, after all finish.

Q2. A pipeline's consumer returns early. What stops the upstream stages?

Answer. The cancelled ctx: every send selects on `ctx.Done()` and returns.

Q3. Your worker pool processes CPU-heavy jobs on 8 cores. How many workers?

Answer. About 8 (`runtime.GOMAXPROCS(0)`); more only adds switching.

Q4. When is a channel generator better than an iterator?

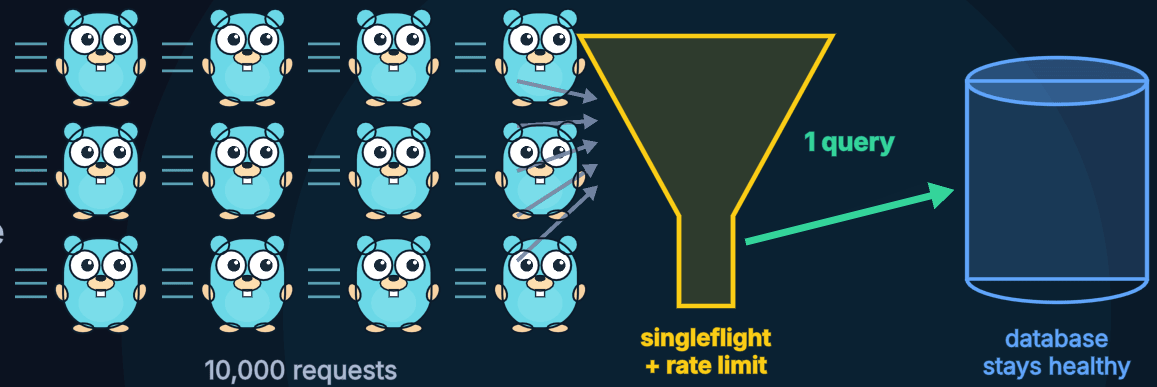
Answer. When the producer should run concurrently: I/O, slow sources, several consumers.

CHAPTER

8.16

Concurrency Patterns II: Production

Real services face bursts, slow dependencies, failures and restarts. A few more patterns keep a concurrent Go program polite to its neighbours and safe to stop.



IN THIS CHAPTER YOU WILL

- ✓ Limit request rates with a token bucket
- ✓ Fan messages out to subscribers without one slow reader blocking all
- ✓ Shut down gracefully on SIGINT and SIGTERM
- ✓ Retry with exponential backoff and jitter
- ✓ Collapse duplicate work with singleflight; bound and cancel with errgroup

? **THE BIG QUESTION**

A cache entry expires and 10,000 requests for it arrive in the same millisecond. Your database can handle 100 queries per second. What protects it?

8.16 Rate limiting with a token bucket

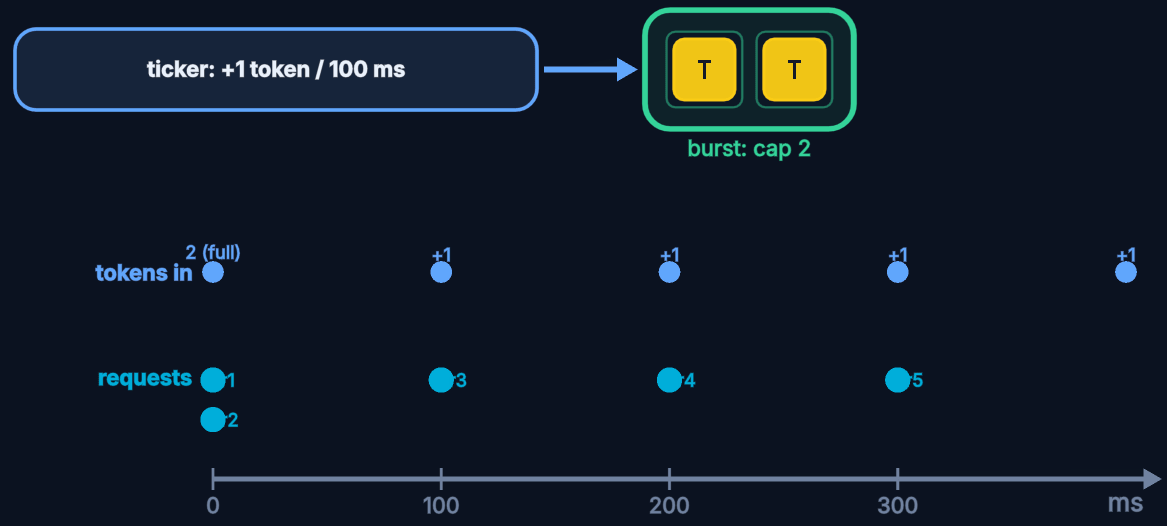
Tokens drip into a bucket at a fixed rate; each request takes one. A full bucket allows a short burst, then the steady rate applies.

examples/ch16_ratelimit/main.go

```go
func main() {
    requests := []int{1, 2, 3, 4, 5}
    limiter := time.NewTicker(100 * time.Millisecond) // 10 req/s
    defer limiter.Stop()

    // a bucket that allows a burst of 2 before the steady rate applies
    burst := make(chan struct{}, 2)
    burst <- struct{}{}
    burst <- struct{}{}
    go func() {
        for range limiter.C {
            select {
            case burst <- struct{}{}: // refill one token
            default: // bucket full: drop the token
            }
        }
    }()

    start := time.Now()
    for _, r := range requests {
        <-burst // wait for a token
        ms := time.Since(start).Round(10 * time.Millisecond).Milliseconds()
        fmt.Printf("request %d at +%3dms\n", r, ms)
    }
}
```

1 2 3 4

go run . printed: r1 at +0 ms, r2 at +0 ms, r3 at +100 ms, r4 at +200 ms, r5 at +300 ms



1. **Ticker** sets the steady rate: 10 requests per second.
2. **Bucket** a buffered channel holding up to 2 tokens: the allowed burst.
3. **Refill** one token per tick; if the bucket is full the token is dropped.
4. **Take** a request waits for a token before it proceeds.

PRO TIP

In production use `golang.org/x/time/rate`: `rate.NewLimiter(rate.Every(100*time.Millisecond), 2)`, then `lim.Wait(ctx)` to block or `lim.Allow()` to reject. Same token bucket, no goroutine, ctx-aware.

8.16 Publish/subscribe: one sender, many receivers

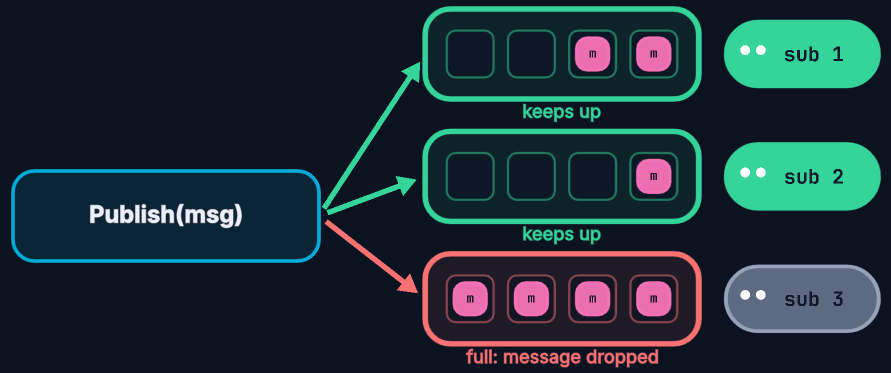
A channel gives each value to one receiver. To broadcast, give every subscriber its own channel, and decide what to do when one is slow.

examples/ch16_pubsub/main.go — Go

```go
type Broker struct {
    mu   sync.Mutex
    subs []chan string
}

func (b *Broker) Subscribe() <-chan string {
    ch := make(chan string, 4) // per-subscriber buffer
    b.mu.Lock()
    b.subs = append(b.subs, ch)
    b.mu.Unlock()
    return ch
}

func (b *Broker) Publish(msg string) {
    b.mu.Lock()
    defer b.mu.Unlock()
    for _, ch := range b.subs {
        select {
        case ch <- msg:
        default: // slow subscriber: drop instead of blocking everyone
        }
    }
}

func (b *Broker) Close() {
    b.mu.Lock()
    defer b.mu.Unlock()
    for _, ch := range b.subs {
        close(ch)
    }
}
```

1

2



go run . — terminal

```
$ go run .
subscriber 2 got "deploy started"
subscriber 2 got "deploy finished"
subscriber 1 got "deploy started"
subscriber 1 got "deploy finished"
```

when a subscriber is slow...	trade-off
block (plain send)	nobody loses data; one slow reader stalls all
drop (select + default)	publisher never waits; slow readers miss events
drop oldest / disconnect	more code; the fast path stays fast

8.16 Graceful shutdown: stop taking work, finish what's in hand

On SIGINT or SIGTERM a service should stop accepting work, let in-flight work finish within a deadline, release resources and exit 0.

examples/ch16_shutdown/main.go — Go

```go
func main() {
    // ctx is cancelled on SIGINT (Ctrl+C) or SIGTERM.
    ctx, stop := signal.NotifyContext(context.Background(),   // 1
        os.Interrupt, syscall.SIGTERM)
    defer stop()

    jobs := make(chan int)
    var wg sync.WaitGroup
    for id := 1; id <= 2; id++ {
        wg.Go(func() { worker(ctx, id, jobs) })
    }
    go func() { // simulate the operator pressing Ctrl+C
        time.Sleep(150 * time.Millisecond)
        syscall.Kill(os.Getpid(), syscall.SIGINT)
    }()
    for j := 1; ; j++ {
        select {
        case jobs <- j:
            continue
        case <-ctx.Done():   // 2
        }
        break
    }
    fmt.Println("signal received:", context.Cause(ctx))
    wg.Wait() // drain: let in-flight jobs finish   // 4
    fmt.Println("clean exit")
}
```

go run . (workers: lines 13-27) — terminal

```
$ go run .
worker 1 finished job 1
worker 2 finished job 2
worker 2 finished job 4
worker 1 finished job 3
signal received: interrupt signal received
worker 2 finished job 5
worker 2: shutting down
worker 1 finished job 6
worker 1: shutting down
clean exit
```

1. **NotifyContext** turns SIGINT/SIGTERM into a cancelled ctx.
2. **Stop dispatching:** main's select sees ctx.Done() and stops sending jobs.
3. **Finish in-flight jobs** (amber), then each worker sees ctx.Done() and returns.
4. **wg.Wait(),** then flush, close, exit. HTTP servers: `srv.Shutdown(ctx)` closes listeners and waits for active requests. Kubernetes allows 30 s after SIGTERM by default.

8.16 Retries: exponential backoff with jitter

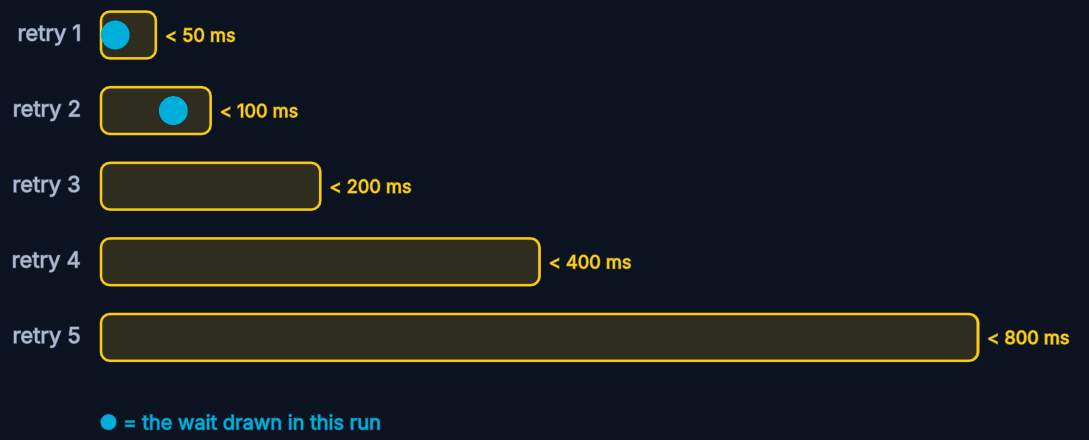
Retrying at once, or on a fixed schedule, makes every failed client hit the recovering server at the same moment. Spread them out.

examples/ch16_retry/main.go — Go

```go
func retry(ctx context.Context, attempts int, base time.Duration,
    f func() error) error {
    var err error
    for i := range attempts {
        if err = f(); err == nil {
            return nil
        }
        // exponential backoff with "full jitter"
        wait := time.Duration(rand.Int64N(int64(base << i)))
        fmt.Printf("attempt %d failed (%v); retry in %v\n",
            i+1, err, wait.Round(time.Millisecond))
        select {
        case <-time.After(wait):
        case <-ctx.Done():
            return errors.Join(err, ctx.Err())
        }
    }
    return err
}

func main() {
    ctx, cancel := context.WithTimeout(context.Background(), time.Second)
    defer cancel()
    err := retry(ctx, 5, 50*time.Millisecond, flaky)
    fmt.Println("result:", err, "after", calls, "calls")
}
```

1
2

WAIT BEFORE RETRY i: RANDOM IN [0, base·2^i)



- **Exponential**: each failure doubles the window, so load falls off fast.
- **Full jitter**: a random wait inside the window spreads clients apart.
- **Cap** the window and the number of attempts; respect `ctx` while sleeping.
- Retry only **idempotent** operations, and only errors that can heal (timeouts, 503).
- `errors.Join` (Go 1.20) keeps both the last error and the ctx error.

Full jitter: Marc Brooker, "Exponential Backoff And Jitter", AWS Architecture Blog, 2015.

go run . — terminal

```
$ go run .
attempt 1 failed (503 unavailable); retry in 13ms
attempt 2 failed (503 unavailable); retry in 66ms
result: <nil> after 3 calls
```

8.16 singleflight: one call for many identical requests

When many goroutines ask for the same missing thing at once, let one of them fetch it and hand the result to all the others.

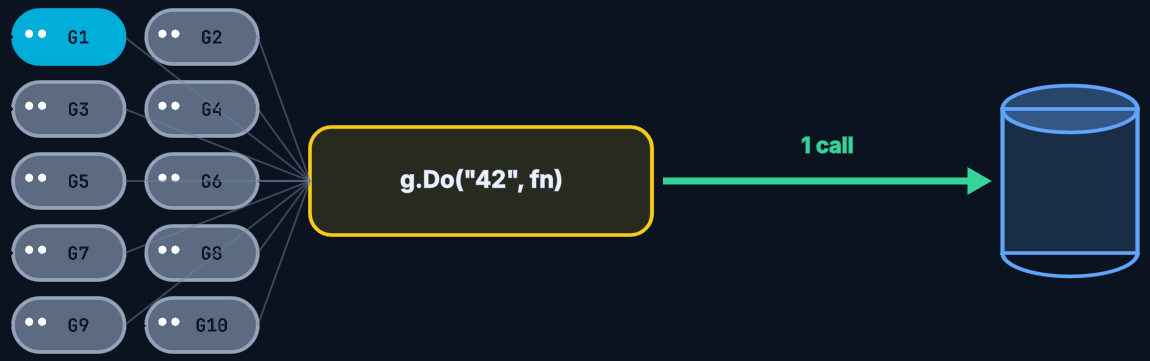
examples/ch16_singleflight/main.go — Go

```go
var backendCalls atomic.Int32

func loadUser(id string) (any, error) {
    backendCalls.Add(1)
    time.Sleep(50 * time.Millisecond) // a slow database query
    return "user " + id, nil
}

func main() {
    var g singleflight.Group
    var wg sync.WaitGroup
    var shared atomic.Int32
    for range 10 { // 10 requests for the same key arrive at once
        wg.Go(func() {
            // Only one loadUser runs per key; the others wait for it.
            _, _, sh := g.Do("42", func() (any, error) { return loadUser("42") })
            if sh {
                shared.Add(1)
            }
        })
    }
    wg.Wait()
    fmt.Println("requests:      10")
    fmt.Println("backend calls:", backendCalls.Load())
    fmt.Println("shared result:", shared.Load())
}
```

1

go run . — terminal

```
$ go run .
requests:      10
backend calls: 1
shared result: 10
```



G1 runs fn; G2-G10 wait and get the same value and error

method	what it does
`Do(key, fn)`	run fn once per key at a time; return `(v, err, shared)`
`DoChan(key, fn)`	the same, as a channel: select it against `ctx.Done()`
`Forget(key)`	the next call runs fn again even if one is in flight

★ **KEY IDEA**

Classic use: a **cache stampede**. The entry expires, thousands of requests miss at once, and singleflight turns them into one database query. It lives in `golang.org/x/sync`.

8.16 errgroup with a limit: bounded, cancellable fan-out

SetLimit caps how many goroutines run; WithContext cancels the rest at the first error; Wait returns that error.

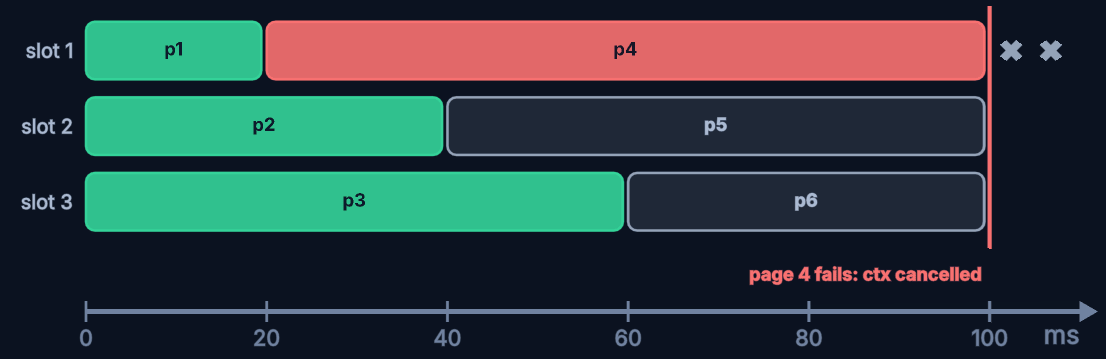
examples/ch16_errgroup/main.go — Go

```go
func fetch(ctx context.Context, page int) error {
    select {
    case <-time.After(time.Duration(20*page) * time.Millisecond):
    case <-ctx.Done():
        return ctx.Err() // a sibling failed: give up early
    }
    if page == 4 {
        return errors.New("500 internal server error")
    }
    return nil
}

func main() {
    g, ctx := errgroup.WithContext(context.Background())
    g.SetLimit(3) // at most 3 fetches in flight
    start := time.Now()
    for page := 1; page <= 8; page++ {
        g.Go(func() error { // blocks while 3 are running
            err := fetch(ctx, page)
            ms := time.Since(start).Round(10 * time.Millisecond).Milliseconds()
            fmt.Printf("+%3dms page %d: %v\n", ms, page, err)
            return err
        })
    }
    fmt.Println("g.Wait():", g.Wait())
}
```

(1) (2)

go run . — terminal

```
$ go run .
+ 20ms page 1: <nil>
+ 40ms page 2: <nil>
+ 60ms page 3: <nil>
+100ms page 4: 500 internal server error
+100ms page 7: context canceled
+100ms page 8: context canceled
+100ms page 5: context canceled
+100ms page 6: context canceled
g.Wait(): 500 internal server error
```



(1) `g.Go` blocks while 3 are running: the loop itself is throttled

(2) `Wait` returns the **first** error; later ones are discarded

- Pages 5-8 return `context.Canceled` right away instead of doing work nobody will use.

8.16 Recap and check yourself

Be polite to dependencies, be ready to stop, and never do the same expensive work twice at once.

REMEMBER

- ✓ Token bucket = ticker (rate) + buffered channel (burst). In production: `x/time/rate`.
- ✓ Pub/sub: one channel per subscriber, and an explicit policy for slow ones.
- ✓ Graceful shutdown: `signal.NotifyContext`, stop intake, drain, `wg.Wait`, exit.
- ✓ Retries: exponential backoff, full jitter, a cap, ctx-aware sleeps, idempotent ops only.
- ✓ `singleflight` collapses identical concurrent calls into one.
- ✓ `errgroup` + `SetLimit` + `WithContext`: bounded fan-out that stops at the first error.

CHECK YOURSELF

Q1. Why add random jitter to exponential backoff?
Answer. Without it, clients that failed together retry together, and overload the server again.

Q2. A publisher uses a plain send to each subscriber. What can one stuck subscriber do?
Answer. Block the publisher, and with it every other subscriber.

Q3. Is singleflight a cache?
Answer. No. It only merges calls that overlap in time; once fn returns, the next call runs fn again.

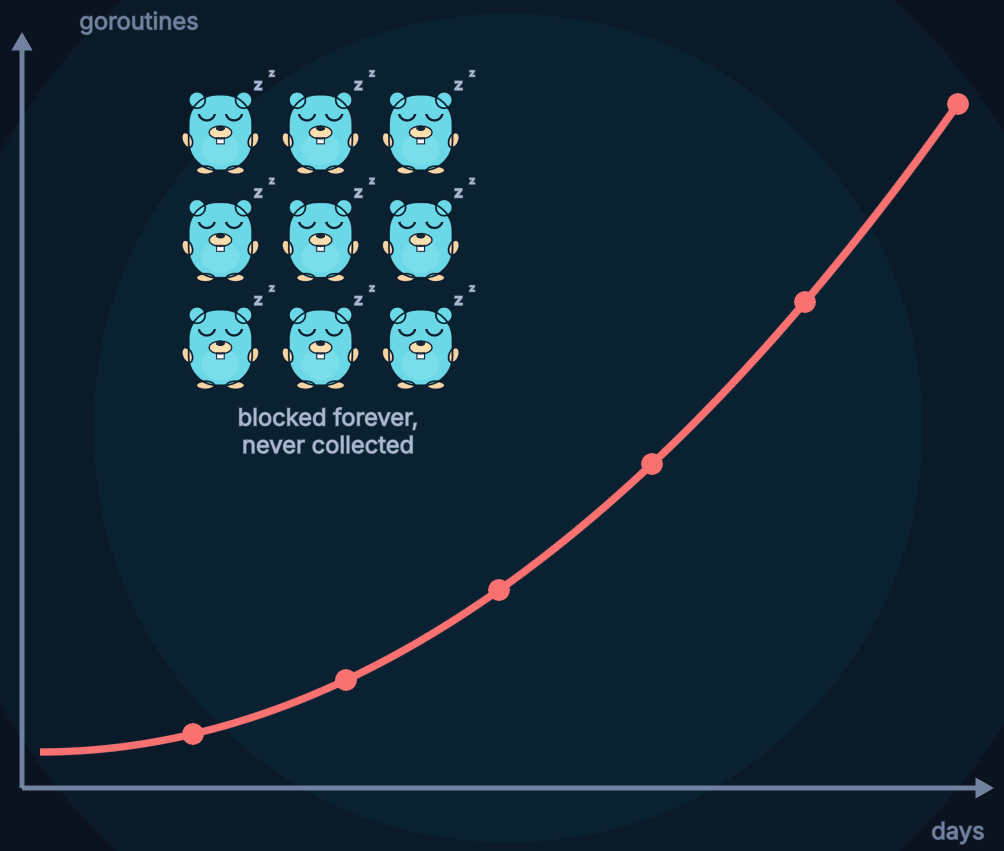
Q4. What does `g.Go` do when `SetLimit(3)` and 3 tasks are running?
Answer. It blocks the caller until one finishes. `TryGo` returns false instead.

CHAPTER

8.17

Leaks, Deadlocks and Debugging

Concurrency bugs rarely crash on the first try. They leak, stall or slow down. This chapter shows how they happen and the tools that make them visible.



IN THIS CHAPTER YOU WILL

- ✓ Recognise the five ways a goroutine leaks
- ✓ Find leaks with the goroutineleak profile (Go 1.27)
- ✓ Read deadlock reports and goroutine dumps
- ✓ Choose the right profile: goroutine, block, mutex, trace
- ✓ Map common panics and symptoms to their causes

? **THE BIG QUESTION**

A service's goroutine count climbs from 200 to 2 million over a week and memory grows with it. Nothing crashes. What is happening, and how do you find the line responsible?

8.17 A goroutine leak in 20 lines

A leaked goroutine is blocked forever. It is never collected, and neither is anything its stack points to.

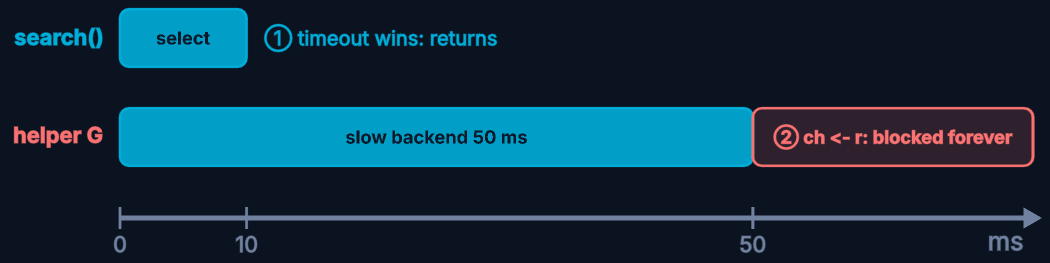
examples/ch17_leak/main.go — Go

```go
// search leaks a goroutine every time the timeout wins.
func search(q string) (string, error) {
    ch := make(chan string) // unbuffered: the bug
    go func() {
        time.Sleep(50 * time.Millisecond) // slow backend
        ch <- "result for " + q           // blocks forever if nobody receives
    }()
    select {
    case r := <-ch:
        return r, nil
    case <-time.After(10 * time.Millisecond):
        return "", fmt.Errorf("timeout")
    }
}

func main() {
    fmt.Println("goroutines before:", runtime.NumGoroutine())
    for i := range 100 {
        search(fmt.Sprint("q", i))
    }
    time.Sleep(100 * time.Millisecond) // every backend has answered by now
    fmt.Println("goroutines after: ", runtime.NumGoroutine())
    // Fix: make(chan string, 1) so the late send never blocks.
}
```

! 2 1

go run . — terminal

```
$ go run .
goroutines before: 1
goroutines after:  101
```



1 → 101

goroutines after 100 calls: every timeout left one behind

✖ DON'T

```go
ch := make(chan string)
```

the late send has no receiver

✓ DO

```go
ch := make(chan string, 1)
```

the late send fits in the buffer; G exits

Also fine: select on `ctx.Done()` in the helper, so it can give up too.

8.17 Five ways goroutines leak

Every leak is a goroutine waiting for something that will never happen. These are the usual somethings.

1. A send nobody receives

The receiver returned early (timeout, error, break).

```
ch <- result // receiver is gone
```

fix: buffer of 1, or select on ctx.Done()

2. A receive nobody satisfies

The sender forgot to close, or died; `range` waits forever.

```
for v := range jobs { … } // never closed
```

fix: the owner closes; defer close(ch)

3. A nil channel

Send and receive on nil block forever (handy in select, fatal elsewhere).

```
var ch chan int; <-ch
```

fix: make the channel; check the init path

4. A lock or WaitGroup never released

A missing Unlock on an error path, a missing Done.

```
mu.Lock(); if err != nil { return }
```

fix: defer mu.Unlock(); defer wg.Done()

5. A loop with no way out

Workers, tickers and pollers that never check for shutdown.

```
for { work(); time.Sleep(d) }
```

fix: select on ctx.Done() each round

★ **KEY IDEA**

Ask of every `go` statement: **how does this goroutine end?** If the answer depends on someone else doing something, make sure that someone always does it, or give the goroutine a ctx to give up.

Since Go 1.23 unreferenced timers and tickers are garbage-collected even if never stopped; the goroutine waiting on them is not.

8.17 Finding leaks: the goroutineleak profile

Go 1.27 can prove that some goroutines are stuck forever: the garbage collector finds the ones blocked on channels or locks nobody else can reach.

examples/ch17_leakprofile/main.go — Go

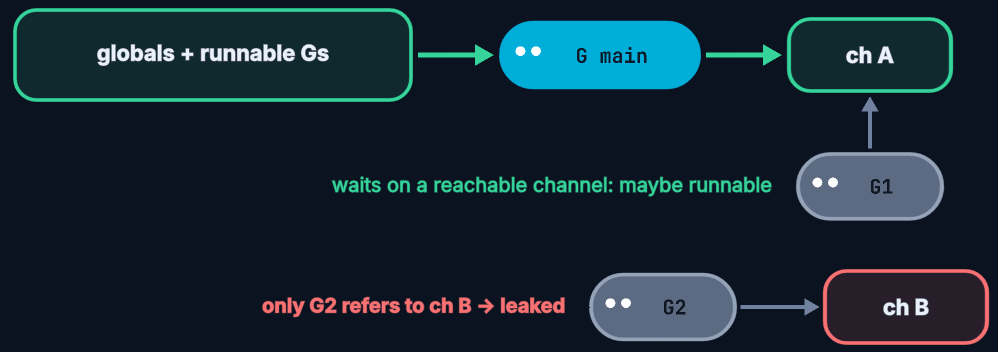
```go
func search() {
    ch := make(chan string) // unbuffered: the late send blocks forever
    go func() {
        time.Sleep(20 * time.Millisecond)
        ch <- "late result"
    }()
    select {
    case <-ch:
    case <-time.After(5 * time.Millisecond):
    }
}

func main() {
    for range 3 {
        search()
    }
    time.Sleep(50 * time.Millisecond)
    // Go 1.27: the GC finds goroutines blocked on channels that no
    // other goroutine can reach, and reports them as leaked.
    p := pprof.Lookup("goroutineleak")
    p.WriteTo(os.Stdout, 1)
    fmt.Println("leaked goroutines:", p.Count())
}
```

1

go run . (address line removed) — terminal

```
$ go run .
goroutineleak profile: total 3
#   0x4e0815    main.search.func1+0x35  ./main.go:14
leaked goroutines: 3
```

HOW THE GC DECIDES



1. **Mark** from globals and goroutines that can still run.
2. **Grow** a blocked G whose channel or lock was reached may be woken: mark it too, repeat.
3. **Report** Gs never reached are blocked on primitives nobody can touch: state `_Gleaked`, listed by `pprof.Lookup("goroutineleak")`.

GO VERSION NOTE

Experimental in Go 1.26 (`GOEXPERIMENT=goroutineleakprofile`), on by default in Go 1.27. It has no false positives, but misses leaks on channels that are still reachable, for example from a global variable.

8.17 Deadlocks: a cycle of waiting

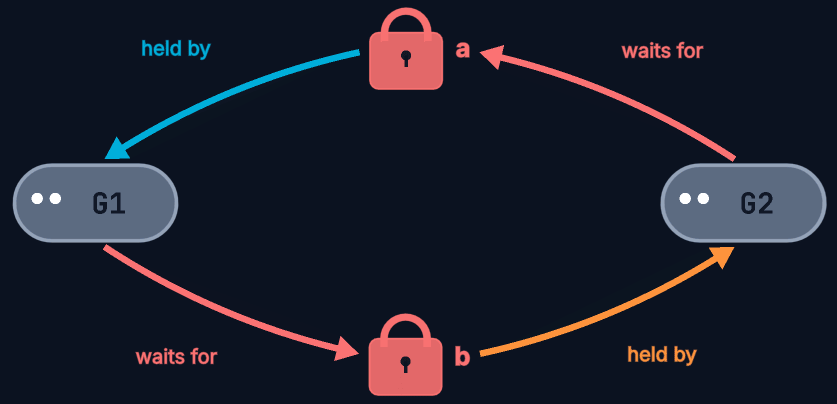
Two goroutines each hold one lock and wait for the other's. The runtime notices only when every goroutine is stuck.

examples/ch17_abba/main.go — Go

```go
func main() {
    var a, b sync.Mutex
    var wg sync.WaitGroup
    wg.Go(func() {
        a.Lock()
        time.Sleep(10 * time.Millisecond)
        b.Lock() // waits for b ... held by the other goroutine
        fmt.Println("never")
    })
    wg.Go(func() {
        b.Lock()
        time.Sleep(10 * time.Millisecond)
        a.Lock() // waits for a ... held by the first goroutine
        fmt.Println("never")
    })
    wg.Wait()
}
```

go run . (abridged: headers and main.go lines) — terminal

```
$ go run .
fatal error: all goroutines are asleep - deadlock!
goroutine 1 [sync.WaitGroup.Wait]:
    ./main.go:24 +0x115
goroutine 7 [sync.Mutex.Lock]:
    ./main.go:15 +0x65
goroutine 8 [sync.Mutex.Lock]:
    ./main.go:21 +0x65
```



- **Detection is global only.** "all goroutines are asleep" fires when nothing at all can run. In a server, an HTTP listener is always alive, so the same cycle just hangs those two requests forever: a *partial* deadlock.
- **Fix: one global lock order.** Every goroutine takes `a` before `b`. Or restructure so one goroutine owns both pieces of state.
- **Find them:** a goroutine dump shows goroutines in `[sync.Mutex.Lock]` for many minutes.
- Channels deadlock the same way: G1 sends to G2 while G2 sends to G1, both unbuffered.

8.17 Reading a goroutine dump

A dump lists every goroutine, what it is waiting for, and the line it is stuck on. It is the first thing to look at when a program hangs.

examples/ch17_dump/main.go — Go

```go
func main() {
    var mu sync.Mutex
    never := make(chan int)
    mu.Lock()
    for range 2 {
        go func() { <-never }() // stuck on a receive
    }
    go func() { mu.Lock() }() // waiting for a mutex
    go time.Sleep(time.Hour)  // sleeping
    time.Sleep(20 * time.Millisecond)

    // The same dump you get from SIGQUIT (Ctrl+\) or /debug/pprof/goroutine?debug=2
    pprof.Lookup("goroutine").WriteTo(os.Stdout, 2)
}
```

how to get one	
`kill -QUIT <pid>` or Ctrl+\	dump all goroutines to stderr, then exit
`GOTRACEBACK=all`	on a crash, print every user goroutine (default: only the crashing one)
`/debug/pprof/goroutine?debug=2`	live, over HTTP, with `import _ "net/http/pprof"`
`pprof.Lookup("goroutine")`	from code, as above; `debug=1` groups identical stacks

go run . (abridged: runtime frames removed) — terminal

```
goroutine 1 [running]:
main.main()
    ./main.go:22 +0x15d

goroutine 7 [chan receive]:
main.main.func1()
    ./main.go:15 +0x19
created by main.main in goroutine 1
    ./main.go:15 +0x70

goroutine 8 [chan receive]:
main.main.func1()
    ./main.go:15 +0x19
created by main.main in goroutine 1
    ./main.go:15 +0x70

goroutine 9 [sync.Mutex.Lock]:
sync.(*Mutex).Lock(...)
main.main.func2()
    ./main.go:17 +0x2c
created by main.main in goroutine 1
    ./main.go:17 +0x116

goroutine 10 [sleep]:
time.Sleep(0x34630b8a000)
created by main.main in goroutine 1
    ./main.go:18 +0x125
```

- `[chan receive]`, `[sync.Mutex.Lock]`, `[sleep]`: the wait reason, the same strings as the scheduler's G states (8.9).
- Long waits show minutes: `[chan receive, 42 minutes]` is a strong leak hint.
- "created by ... in goroutine 1" tells you **who started** it: often the real culprit.

8.17 Profiles and traces for concurrent programs

Each tool answers a different question. Import net/http/pprof once and most of them are a URL away.

tool	answers	turn it on / read it
goroutine profile	what is every goroutine doing right now?	`/debug/pprof/goroutine`
goroutineleak (1.27)	which goroutines can never wake up?	`/debug/pprof/goroutineleak`, `pprof.Lookup`
block profile	where do goroutines wait on channels, select, locks?	`runtime.SetBlockProfileRate(n)`
mutex profile	which locks are contended, and who holds them too long?	`runtime.SetMutexProfileFraction(n)`
CPU profile	where is CPU time spent (spinning, contention)?	`/debug/pprof/profile?seconds=30`
execution trace	when did each G run, block, migrate; GC; syscalls?	`go test -trace=t.out`, `go tool trace t.out`
flight recorder (1.25)	the last few seconds of trace, saved when something goes wrong	`trace.NewFlightRecorder(cfg)`, `WriteTo`
`-race`	which accesses are unsynchronized?	`go test -race ./...` (8.13)

read any profile in a browser — terminal

```
go tool pprof -http=:8080 http://localhost:6060/debug/pprof/block
```

PRO TIP

Block and mutex profiling cost something; sample (rate > 1) in production. The execution trace is the only tool that shows **time**: use it for latency puzzles.

8.17 Field guide: symptom → cause → fix

The messages and symptoms you will meet, and where in this module each one is explained.

you see	usual cause	fix (chapter)
fatal error: all goroutines are asleep - deadlock!	everyone waits: a send with no receiver, a missing Done, a lock cycle	find the cycle in the dump (8.5, 8.17)
panic: send on closed channel	a receiver or a second sender closed it	only the sole sender closes (8.6)
panic: close of closed channel	two owners	one owner, or sync.Once (8.6)
panic: sync: negative WaitGroup counter	more Done than Add	Add before go, or wg.Go (8.4)
fatal error: concurrent map writes	a plain map shared between goroutines	Mutex or sync.Map (8.12)
WARNING: DATA RACE	unsynchronized shared memory	add a happens-before edge (8.13)
goroutine count keeps growing	a leak	goroutineleak profile, ctx (8.17)
hangs, CPU idle	a partial deadlock or a missing close	goroutine dump (8.17)
CPU busy, little progress	spinning or heavy lock contention	CPU and mutex profiles (8.17)
tail latency spikes	too many CPU-bound Gs, GC, contention	execution trace, worker pool (8.11, 8.15)

★ **KEY IDEA** Most of these are found by three habits: run tests with `-race`, give every goroutine an exit path (ctx, close), and look at a goroutine dump before guessing.

8.17 Recap and check yourself

Every goroutine needs an exit; every hang leaves evidence in a dump or a profile.

REMEMBER

- A leaked goroutine waits forever and pins all the memory its stack references.
- Leaks: sends without receivers, unclosed ranges, nil channels, unreleased locks, endless loops.
- Go 1.27's goroutineleak profile proves leaks with the GC; it misses reachable channels.
- The runtime detects only total deadlock; partial deadlocks need a goroutine dump.
- Dumps: SIGQUIT, `GOTRACEBACK=all`, `/debug/pprof/goroutine?debug=2`.
- Block, mutex and CPU profiles show waiting and contention; traces show time.

CHECK YOURSELF

Q1. Why doesn't the garbage collector simply free a goroutine blocked forever?

Answer. In general it cannot know nobody will wake it. The leak profile proves it only when the primitive is unreachable.

Q2. A server hangs but never prints "all goroutines are asleep". Why?

Answer. Some goroutine (a listener, a ticker) can still run, so the deadlock is only partial.

Q3. Where would you look first for lock contention?

Answer. The mutex profile (who holds) and the block profile (who waits).

Q4. What one-line change fixes the leak in ch17_leak?

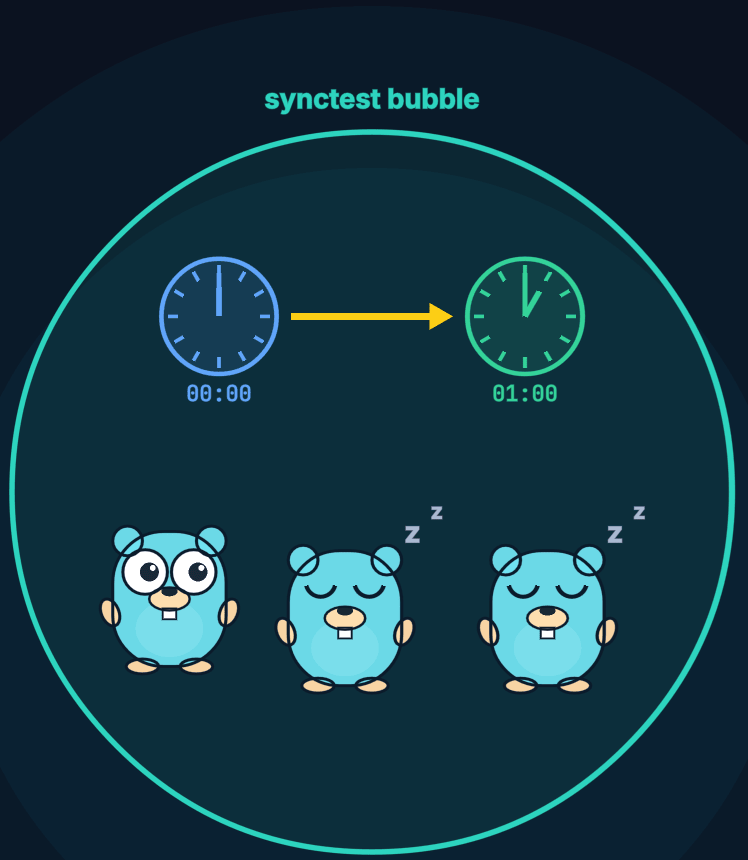
Answer. `make(chan string, 1)`: the late send completes into the buffer and the goroutine exits.

CHAPTER

8.18

Testing Concurrent Code

Concurrent code is hard to test because timing changes from run to run. Go gives you a fake clock, a way to wait until goroutines settle, and a race detector that turns silent bugs into failures.



IN THIS CHAPTER YOU WILL

- ✓ See why sleep-based tests are slow and flaky
- ✓ Test timeouts and expiry instantly with `testing/synctest`
- ✓ Use `synctest.Wait` to check state at exactly the right moment
- ✓ Let the bubble catch deadlocks and leaked goroutines
- ✓ Run `-race`, `-count` and `-cpu` to shake out rare bugs

? **THE BIG QUESTION**

You must test a cache entry that expires after one hour. Do you make the test wait an hour, sprinkle `time.Sleep`, or is there something better?

8.18 Why concurrent tests flake

A test that sleeps and hopes the background work is done passes on a fast laptop and fails on a busy CI machine.

- ✖ **Slow**: every sleep is real time, multiplied by every test, every run.
- ✖ **Flaky**: no sleep is long enough under load, and longer sleeps make it slower.
- ✖ **Blind**: you cannot test "nothing happened yet" without waiting.

BETTER, IN ORDER OF PREFERENCE

1. **Synchronize, don't sleep.** Have the code under test signal completion: a channel, a `WaitGroup`, a callback. Then the test waits for exactly that.
2. **Fake the clock.** `testing/synctest` runs the test in a bubble whose `time` package is fake: an hour passes instantly and deterministically.
3. **Wait for quiescence.** `synctest.Wait()` returns when every other goroutine in the bubble is blocked: the perfect moment to assert.
4. **Shake it.** Run with `-race`, `-count=100` and `-cpu=1,2,4` to expose orderings a single run never hits.

8.18 testing/synctest: a bubble with a fake clock

Inside synctest.Test, time starts at midnight UTC on 2000-01-01 and jumps forward whenever every goroutine in the bubble is blocked.

examples/ch18_synctest/timeout.go (code under test) — Go

```go
16 func (c *Cache) Set(k, v string, ttl time.Duration) {
17     c.mu.Lock()
18     c.m[k] = v
19     c.mu.Unlock()
20     time.AfterFunc(ttl, func() {
21         c.mu.Lock()
22         delete(c.m, k)
23         c.mu.Unlock()
24     })
25 }
```

examples/ch18_synctest/timeout_test.go — Go

```go
9  func TestExpiry(t *testing.T) {
10     synctest.Test(t, func(t *testing.T) { // a "bubble" with a fake clock
11         c := New()
12         c.Set("token", "abc", time.Hour)
13
14         synctest.Sleep(59 * time.Minute) // instant: the fake clock jumps
15         if _, ok := c.Get("token"); !ok {
16             t.Fatal("expired too early")
17         }
18         synctest.Sleep(2 * time.Minute) // passes the TTL; AfterFunc has run
19         if _, ok := c.Get("token"); ok {
20             t.Fatal("not expired after the TTL")
21         }
22         t.Log("fake time now:", time.Now().Format("15:04"))
23     })
24 }
```

go test -v . (real time: 0.00 s) — terminal

```
$ go test -v .
=== RUN   TestExpiry
    timeout_test.go:22: fake time now: 01:01
--- PASS: TestExpiry (0.00s)
PASS
ok      gozm/module08/ch18_synctest 0.003s
```

fake clock (each jump is instant)

1. **Test** creates the bubble; goroutines and timers started inside belong to it.
2. **Sleep** the fake clock jumps 59 min the moment nothing else can run, then waits for the bubble to settle.
3. **Past the TTL,** the AfterFunc timer has fired and finished before `Sleep` returns.

GO VERSION NOTE Go 1.24: experiment (`synctest.Run`). Go 1.25: `synctest.Test` and `Wait`. Go 1.27: `synctest.Sleep` = `time.Sleep` + `Wait`.

8.18 synctest.Wait: assert when everything has settled

Wait returns once every other goroutine in the bubble is durably blocked. Then the test can look at state without races and without guessing.

examples/ch18_wait/afterfunc_test.go — Go

```go
func TestAfterFunc(t *testing.T) {
    synctest.Test(t, func(t *testing.T) {
        ctx, cancel := context.WithCancel(t.Context())
        called := false
        context.AfterFunc(ctx, func() { called = true })

        synctest.Wait() // every other goroutine in the bubble is blocked
        if called {
            t.Fatal("called before cancel")
        }

        cancel()
        synctest.Wait() // the AfterFunc goroutine has run and exited
        if !called {
            t.Fatal("not called after cancel")
        }
        t.Log("called exactly when expected; no sleeps, no flakes")
    })
}
```

1

2

go test -v . — terminal

```
$ go test -v .
=== RUN   TestAfterFunc
    afterfunc_test.go:25: called exactly when expected; no sleeps, no flakes
--- PASS: TestAfterFunc (0.00s)
PASS
ok      gozm/module08/ch18_wait 0.002s
```

durably blocking (time can advance)	NOT durably blocking
send/receive on a channel made in the bubble	locking a sync.Mutex or RWMutex
select whose cases are all bubble channels	network or file I/O
sync.Cond.Wait	system calls
WaitGroup.Wait (Add called in the bubble)	channels made outside the bubble
time.Sleep	

DEFINITION

Durably blocked = blocked on something only another goroutine **in the same bubble** can release. A mutex could be unlocked from outside, so it does not count.

- When all are durably blocked: `Wait` returns if called; otherwise time jumps to the next timer; otherwise the test fails with a deadlock.
- `t.Context()` inside the bubble is cancelled when the test ends.
- Avoid real network and external processes inside a bubble: use fakes.

8.18 The bubble catches leaks and deadlocks

synctest.Test waits for every goroutine it started. A goroutine stuck forever turns into a test failure with its exact line.

examples/ch18_bubble_leak/leak_test.go — Go

```
 8 func TestLeakyHelper(t *testing.T) {
 9     synctest.Test(t, func(t *testing.T) {
10         results := make(chan int)
11         go func() {
12             results <- 42 // nobody ever receives: this goroutine leaks
13         }()
14         // the test body returns without reading results
15     })
16 }
```

KEY IDEA

Without the bubble this test passes and the goroutine leaks silently into the rest of the test binary. With it, the leak is a red test pointing at line 12.

1. **The test body returns.** The root goroutine of the bubble exits.
2. **One goroutine is left,** durably blocked on a bubble channel: nothing can ever release it.
3. **Test panics:** "deadlock: main bubble goroutine has exited but blocked goroutines remain".

go test -v . (abridged) — terminal

```
$ go test -v .
=== RUN   TestLeakyHelper
--- FAIL: TestLeakyHelper (0.00s)
panic: deadlock: main bubble goroutine has exited but blocked goroutines remain
    [recovered, repanicked]
    ./leak_test.go:9 +0x1a
goroutine 22 [chan send (durable), synctest bubble 1]:
    ./leak_test.go:12 +0x1e
    ./leak_test.go:11 +0x67
FAIL    gozm/module08/ch18_bubble_leak  0.004s
FAIL
```

The dump even marks the wait as **(durable)** and names the bubble.

8.18 Run tests with -race, many times, on many Ps

A race detector only sees races that happen. Make them happen: real concurrency in the test, repeated runs, different GOMAXPROCS.

examples/ch18_race_test/counter.go — Go

```go
// Hits counts page views. BUG: no synchronization.
type Hits struct{ n int }

func (h *Hits) Inc()     { h.n++ }
func (h *Hits) Get() int { return h.n }
```

examples/ch18_race_test/counter_test.go — Go

```go
func TestHitsConcurrent(t *testing.T) {
    var h Hits
    var wg sync.WaitGroup
    for range 8 { // a test must really run things concurrently
        wg.Go(func() {
            for range 100 {
                h.Inc()
            }
        })
    }
    wg.Wait()
    t.Log("hits:", h.Get())
}
```

`t.Parallel()` runs tests concurrently with each other; it does not make one test concurrent.

go test -race -v . (abridged) — terminal

```
$ go test -v -race .
=== RUN   TestHitsConcurrent
==================
WARNING: DATA RACE
Read at 0x00c0000aa178 by goroutine 9:
  ...  (stacks and a second report)
    counter_test.go:19: hits: 800
    testing.go:1867: race detected during execution of test
--- FAIL: TestHitsConcurrent (0.00s)
FAIL
FAIL    gozm/module08/ch18_race_test    0.017s
FAIL
```

PITFALL

The count printed **800**, the right answer, and the test still **failed**: with `-race` any detected race fails the test. Without `-race` this test is green forever.

flag	why
`-race`	fail on any data race
`-count=100`	repeat (and skip the test cache)
`-cpu=1,2,4`	run with different GOMAXPROCS
`-shuffle=on`	random test order: finds shared state
`-timeout=30s`	turn a hang into a failure with a dump

8.18 Designing concurrent code to be testable

The easiest concurrent code to test is code whose goroutines can be started, observed and stopped by the caller.

Return a way to stop

Functions that start goroutines take a `ctx`, or return a stop/Close method. Tests call it in `t.Cleanup`.

Signal completion

Give callers something to wait on: a `done` channel, a `WaitGroup`, a result channel. Tests wait on it instead of sleeping.

Keep goroutines local

No package-level workers started in `init`. Each test builds its own instance, so tests cannot interfere.

Use the real clock, in a bubble

With `synctest` you rarely need a fake clock interface: `time.Now`, `time.After` and timers are already fake inside the bubble.

Make the core synchronous

Put the logic in plain functions; keep the goroutine shell thin. Test the logic without any concurrency at all.

Measure with RunParallel

Benchmarks with `b.RunParallel` (8.12) show contention that a single-goroutine benchmark hides.

★ **KEY IDEA** If a test needs `time.Sleep`, the code is missing a signal. If a test hangs, run it with `-timeout` and read the goroutine dump it prints.

8.18 Recap and check yourself

Deterministic tests: synchronize instead of sleeping, fake the clock, and let -race and the bubble fail loudly.

REMEMBER

- Sleep-based tests are slow and flaky; wait on a signal instead.
- `synctest.Test`: a bubble with a fake clock starting 2000-01-01 00:00 UTC.
- Time advances only when every bubble goroutine is durably blocked.
- `synctest.Wait` returns when everything else has settled; `Sleep` (1.27) = sleep + wait.
- Leftover blocked goroutines make the test fail: leaks become visible.
- `-race`, `-count`, `-cpu`, `-shuffle`, `-timeout`: make rare orderings happen.

CHECK YOURSELF

Q1. Inside a bubble, a goroutine waits on `mu.Lock()`. Can the fake clock advance?
Answer. No. A mutex is not durably blocking: something outside could unlock it.

Q2. What is `time.Now()` at the start of every synctest bubble?
Answer. Midnight UTC, January 1, 2000.

Q3. Your test passes 99 times in 100. Which flags help you find out why?
Answer. `-race -count=1000 -cpu=1,2,4`, plus `-timeout` to get a dump if it hangs.

Q4. Why did `TestHitsConcurrent` fail although it printed 800?
Answer. `-race` fails any test in which it detects a data race, whatever the result.

CHAPTER

8.19

Capstone: a Concurrent Web Crawler

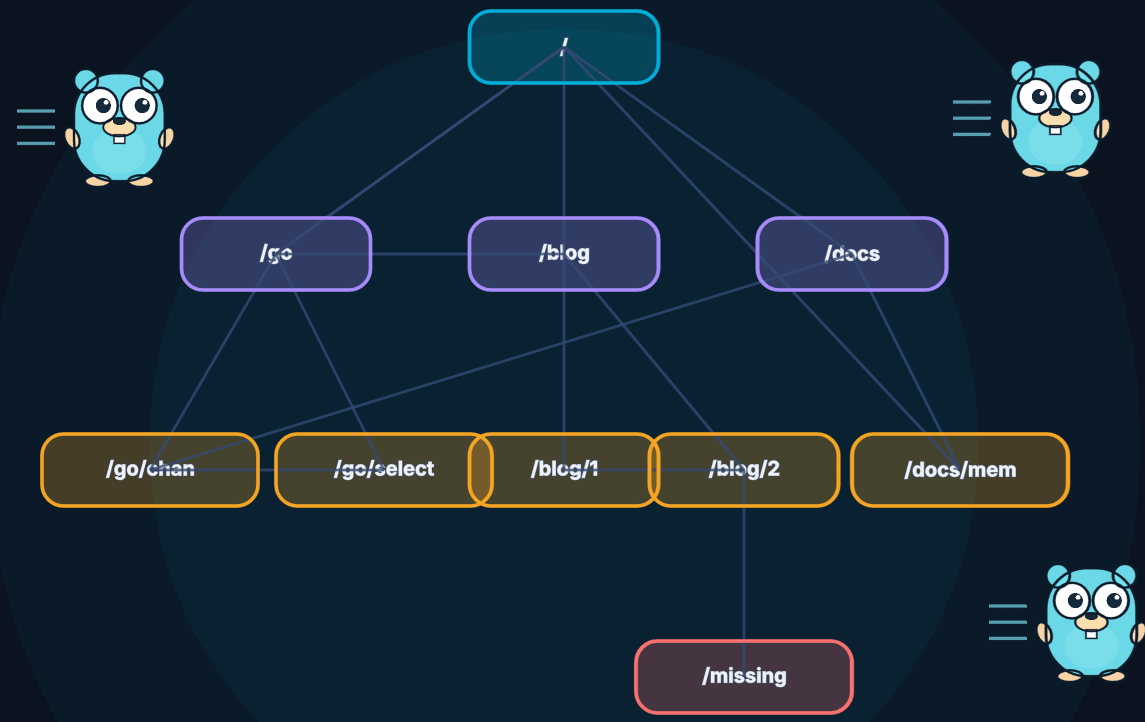
One program that uses the whole module: a worker pool, channels as the only shared state, a select with a nil channel, a rate limit, a deadline, a clean shutdown, a race-free test with a fake clock.

IN THIS CHAPTER YOU WILL

- ✓ Split a problem into an owner goroutine and a worker pool
- ✓ Coordinate with a select whose cases switch on and off
- ✓ Add a rate limit and a deadline without new locks
- ✓ Shut down without leaks, and prove it in a test
- ✓ Review concurrent code with a checklist

? THE BIG QUESTION

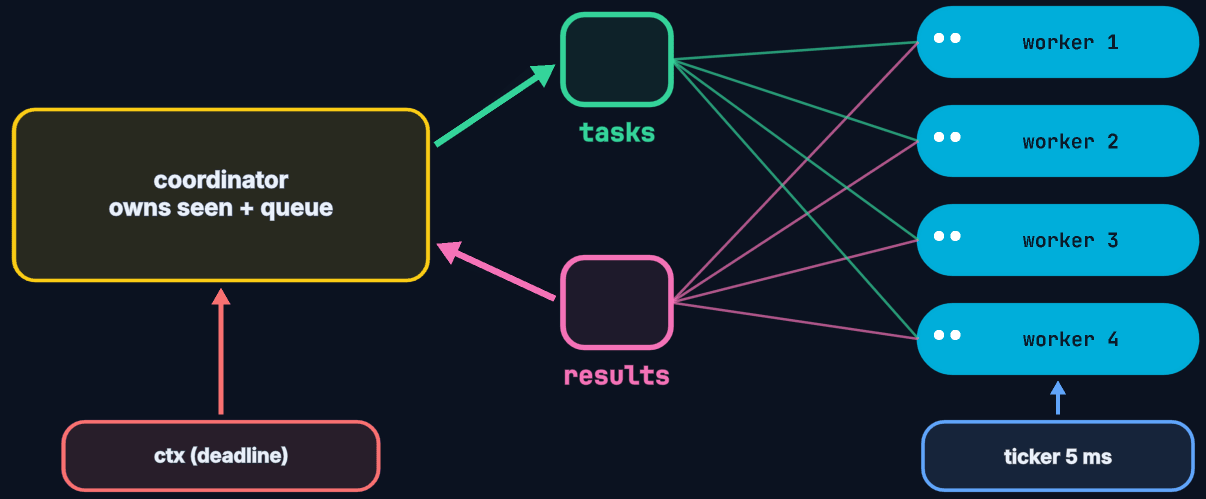
Fetch every page of a site in parallel, never the same page twice, at most 200 fetches per second, and stop at a deadline. Which goroutine owns which data?



10 pages · 4 workers · 1 coordinator

8.19 The design: one owner, many workers

Shared state (seen pages, the queue) lives in exactly one goroutine. Workers own nothing; they get a URL and send back a page.



thing	owner	how others reach it
`seen` map, `queue`	coordinator	nobody else: no lock needed
`tasks` channel	coordinator (sends, closes)	workers range over it
`results` channel	workers (send)	coordinator receives
`inflight` count	coordinator	"how many tasks are out?"
rate limit	the ticker	each worker takes a tick before fetching
deadline	`ctx`	fetch and the coordinator select on Done

- `tasks` and `results` are unbuffered: every hand-off is a synchronisation point.
- The coordinator never blocks on one channel only; it always selects (next pages).

KEY IDEA

"Share memory by communicating" in practice: the map that would need a mutex is touched by one goroutine only, and every other goroutine talks to it through channels.

8.19 The fake web and the fetcher

To keep the example self-contained, the "web" is a map and every fetch takes 20 ms. The crawler cannot tell the difference.

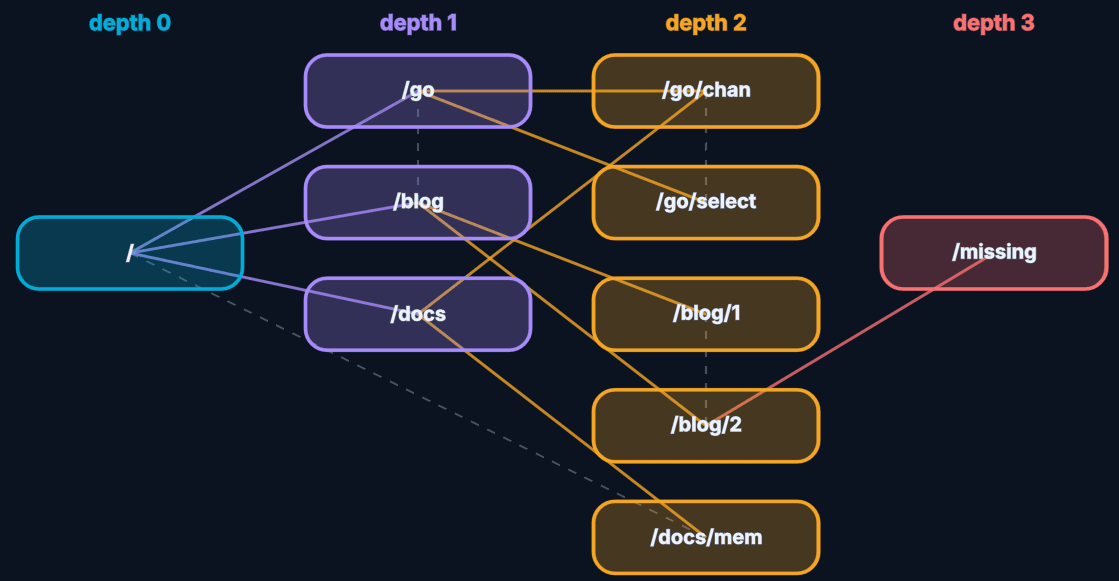
examples/ch19_crawler/crawl.go — Go

```go
// A fake web: page -> links. Every fetch takes 20ms.
var web = map[string][]string{
    "/":          {"/go", "/blog", "/docs"},
    "/go":        {"/go/chan", "/go/select", "/"},
    "/blog":      {"/blog/1", "/blog/2", "/go"},
    "/docs":      {"/docs/mem", "/go/chan"},
    "/go/chan":   {"/go/select"},
    "/go/select": {},
    "/blog/1":    {"/blog/2"},
    "/blog/2":    {"/missing"},
    "/docs/mem":  {"/"},
}
```

examples/ch19_crawler/crawl.go — Go

```go
func fetch(ctx context.Context, url string) ([]string, error) {
    select {
    case <-time.After(20 * time.Millisecond):
    case <-ctx.Done():
        return nil, ctx.Err()
    }
    links, ok := web[url]
    if !ok {
        return nil, fmt.Errorf("404 not found")
    }
    return links, nil
}
```

1



dashed = link to a page at the same or a lower depth (already seen)

NOTE

① `fetch` honours `ctx` like any real HTTP call would (`http.NewRequestWithContext`). `/missing` is linked but absent: a 404 to show error handling.

8.19 The worker pool: rate-limited, cancellable

Four workers range over tasks. Before each fetch a worker takes a tick from the shared ticker, so the whole pool makes at most 200 fetches a second.

examples/ch19_crawler/crawl.go — Go

```go
// Crawl visits pages breadth-first up to maxDepth with a pool of workers.
func Crawl(ctx context.Context, root string, maxDepth, workers int) []Page {
    tasks := make(chan task)
    results := make(chan Page)
    limiter := time.NewTicker(5 * time.Millisecond) // at most 200 fetches/s   (1)
    defer limiter.Stop()

    var wg sync.WaitGroup
    for range workers { // 1. the fetcher pool: no shared state at all
        wg.Go(func() {
            for t := range tasks {                                            (2)
                select {
                case <-limiter.C: // wait for a rate-limit token              (3)
                case <-ctx.Done():
                }
                links, err := fetch(ctx, t.url)
                results <- Page{t.url, t.depth, links, err}                   (4)
            }
        })
    }
```

A shared `time.Ticker` channel is safe for many receivers: each tick goes to exactly one worker, and ticks nobody takes are dropped.

1. **One ticker for the pool:** a tick every 5 ms is a global rate of 200/s, however many workers.
2. **range over tasks:** the worker exits when the coordinator closes `tasks`.
3. **Take a tick or give up:** if ctx is done, skip waiting; fetch will fail fast anyway.
4. **Always send a result,** even an error: the coordinator counts every task it handed out.

EVERY TASK GETS AN ANSWER

Why must a worker report a failed fetch? The coordinator's `inflight` counter only goes down on a result. A worker that silently dropped a task would make the coordinator wait forever.

8.19 The coordinator: a select that switches cases on and off

One loop, three cases. A nil channel disables a case (8.8), so the same select can stop sending when there is nothing to send, or after a cancel.

examples/ch19_crawler/crawl.go — Go

```
70      seen := map[string]bool{root: true}
71      queue := []task{{root, 0}}
72      inflight, done := 0, ctx.Done()
73      var pages []Page
74      for len(queue) > 0 || inflight > 0 {                    1
75          var send chan task // nil -> this select case is disabled   2
76          var next task
77          if len(queue) > 0 {
78              send, next = tasks, queue[0]
79          }
80          select {
81          case send <- next:                                  3
82              queue = queue[1:]
83              inflight++
84          case p := <-results:                                4
85              inflight--
86              pages = append(pages, p)
87              if p.Err != nil || p.Depth == maxDepth {
88                  continue
89              }
90              for _, l := range p.Links {
91                  if !seen[l] {
92                      seen[l] = true
93                      queue = append(queue, task{l, p.Depth + 1})
94                  }
95              }
96          case <-done: // cancelled: stop dispatching, keep draining   5
97              queue, done = nil, nil
98          }
99      }
100     close(tasks) // 3. shut the pool down and wait for it   6
101     wg.Wait()
102     return pages
103 }
```

1. **Loop** while there is work queued **or** tasks are still out.
2. **send = nil** unless the queue has a URL: an empty queue disables the send case.
3. **Send a task:** pop the queue, `inflight++`.
4. **Receive a result:** `inflight--`, record it, queue unseen links one level deeper.
5. **Cancelled:** drop the queue and set `done = nil` so this case never fires again; keep receiving until every task in flight has answered.
6. **Shut down:** `close(tasks)` ends the workers' range loops; `wg.Wait()` makes sure they exited.

★ **KEY IDEA**

No goroutine is ever left blocked: workers always have a receiver for their result, and the coordinator stops only when `inflight` is zero. That is why the leak test passes.

8.19 Running it: with a budget, and with a tight deadline

The same Crawl, two contexts. With 2 s it finishes; with 50 ms the deadline cuts it short, and it still returns cleanly.

examples/ch19_crawler/main.go — Go

```go
func run(timeout time.Duration) {
    ctx, cancel := context.WithTimeout(context.Background(), timeout)
    defer cancel()
    start := time.Now()
    pages := Crawl(ctx, "/", 3, 4)
    for _, p := range pages {
        status := fmt.Sprintf("%d links", len(p.Links))
        if p.Err != nil {
            status = p.Err.Error()
        }
        fmt.Printf("  depth %d  %-11s %s\n", p.Depth, p.URL, status)
    }
    fmt.Printf("  => %d pages in %v\n", len(pages), time.Since(start).Round(10*time.Millisecond))
}

func main() {
    fmt.Println("crawl with a 2s budget:")
    run(2 * time.Second)
    fmt.Println("crawl with a 50ms budget:")
    run(50 * time.Millisecond)
}
```

go run . — terminal

```
$ go run .
crawl with a 2s budget:
  depth 0  /           3 links
  depth 1  /docs       2 links
  depth 1  /blog       3 links
  depth 1  /go         3 links
  depth 2  /go/chan    1 links
  depth 2  /docs/mem   1 links
  depth 2  /blog/1     1 links
  depth 2  /blog/2     1 links
  depth 2  /go/select  0 links
  depth 3  /missing    404 not found
  => 10 pages in 100ms
crawl with a 50ms budget:
  depth 0  /           3 links
  depth 1  /docs       2 links
  depth 1  /go         context deadline exceeded
  depth 2  /docs/mem   context deadline exceeded
  depth 2  /go/chan    context deadline exceeded
  depth 1  /blog       context deadline exceeded
  => 6 pages in 50ms
```

2 s run: fetch waves (20 ms each)

10 pages at depths 0-3 need 4 sequential waves: about 4 × 20 ms plus rate-limit ticks. With 50 ms only 6 pages come back, the rest report the deadline.

8.19 Proving it: a race-free, leak-free test in fake time

The crawler's own test runs in a synctest bubble under -race. A leaked worker or coordinator would fail it.

examples/ch19_crawltest/crawl_test.go — Go

```go
func TestCrawl(t *testing.T) {
    synctest.Test(t, func(t *testing.T) { // fake clock: no real waiting
        ctx, cancel := context.WithTimeout(t.Context(), 2*time.Second)
        defer cancel()
        start := time.Now()
        pages := Crawl(ctx, "/", 3, 4)
        if len(pages) != 10 {
            t.Fatalf("got %d pages, want 10", len(pages))
        }
        t.Logf("10 pages in %v of fake time", time.Since(start))
    })
}

func TestCrawlDeadline(t *testing.T) {
    synctest.Test(t, func(t *testing.T) {
        ctx, cancel := context.WithTimeout(t.Context(), 30*time.Millisecond)
        defer cancel()
        start := time.Now()
        pages := Crawl(ctx, "/", 3, 4) // must return, with no leaks
        t.Logf("deadline: %d pages, returned after %v", len(pages), time.Since(start))
    })
}
```

go test -v -race . — terminal

```
$ go test -v -race .
=== RUN   TestCrawl
    crawl_test.go:19: 10 pages in 100ms of fake time
--- PASS: TestCrawl (0.00s)
=== RUN   TestCrawlDeadline
    crawl_test.go:29: deadline: 4 pages, returned after 30ms
--- PASS: TestCrawlDeadline (0.00s)
PASS
ok      gozm/module08/ch19_crawltest    1.018s
```

- **Fast:** 2 s budgets and 20 ms fetches run in fake time.
- **Race-free:** `-race` watched every access: no report.
- **Leak-free:** the bubble fails the test if any goroutine is left blocked.
- **Deadline-safe:** with 30 ms, Crawl returns at exactly 30 ms of fake time.

> **NOTE**
> Fake time is deterministic, but `select` still picks randomly among ready cases, so the total can differ by one 5 ms tick between runs (100 or 105 ms). Assert on outcomes, not on exact schedules.

8.19 A review checklist for any concurrent Go code

Ask these questions of every pull request that starts a goroutine, makes a channel or adds a lock. Here are the crawler's answers.

question	the crawler's answer
How does every goroutine **end**?	workers: `tasks` closed; coordinator: queue empty and inflight 0
Who **closes** each channel, exactly once?	coordinator closes `tasks`; `results` is never closed, nobody ranges over it
Which goroutine **owns** each piece of shared state?	coordinator owns `seen`, `queue`, `pages`; no mutex
What happens on **cancel** or deadline?	stop dispatching, drain in-flight results, close, wait
What **bounds** concurrency and rate?	4 workers; one 5 ms ticker = 200 fetches/s
Can a send or receive block **forever**?	no: every task gets a result; the coordinator always receives
Do **errors** reach a decision point?	each Page carries Err; 404s and deadline errors are reported
Is it **tested** under `-race`, fast and leak-checked?	synctest bubble + `-race`, two tests, about 1 s

★ **KEY IDEA** Most concurrency bugs are a "no" or a "not sure" to one of these eight questions. Answer them in the design, not in production.

8.19 Extend the crawler: exercises

Each exercise uses a technique from this module. Try them in order; keep every one race-free and leak-free under the synctest test.

1. Per-host limits

Give each host its own ticker or `x/time/rate` limiter, so one slow site does not starve the others.

see 8.16

2. Retries

Retry 5xx and timeouts with exponential backoff and full jitter, inside the worker, honouring ctx.

see 8.16

3. errgroup version

Rewrite the pool with `errgroup.WithContext` and `SetLimit`. What changes about error handling?

see 8.4, 8.16

4. Streaming results

Return `<-chan Page` instead of a slice, so callers process pages as they arrive. Who closes it?

see 8.15

5. Real HTTP

Fetch with `http.NewRequestWithContext`, parse links with `golang.org/x/net/html`, and test against `httptest.Server`.

see 8.14

6. Metrics

Count fetches, errors and in-flight tasks with `atomic.Int64`, and expose them while it runs. Is a mutex needed?

see 8.12

TRY IT Stretch goal: make `Crawl` resumable. Persist `seen` and `queue` when the deadline hits, and continue from them in the next run.

8.19 Recap: the whole module in one program

Every line of the crawler maps to a chapter. If you can explain each one, you know Go's concurrency model.

in the crawler	concept	chapter
`wg.Go(func() { for t := range tasks {…} })`	goroutines, WaitGroup.Go, range over a channel	8.3, 8.4, 8.6
`tasks`, `results` (unbuffered)	synchronous hand-off, happens-before	8.5, 8.13
`var send chan task // nil`	nil channels disable select cases	8.8
`limiter := time.NewTicker(…)`	timers and tickers, rate limiting	8.8, 8.16
`case <-ctx.Done():`	cancellation and deadlines	8.14
coordinator owns `seen`	share memory by communicating	8.12, 8.15
4 workers	worker pool, the scheduler's Ps	8.10, 8.15
`close(tasks); wg.Wait()`	clean shutdown, no leaks	8.16, 8.17
`synctest.Test` + `-race`	testing concurrent code	8.13, 8.18

> *Concurrency is about dealing with lots of things at once. Parallelism is about doing lots of things at once.*
>
> — Rob Pike, "Concurrency is not Parallelism", 2012

BACK MATTER

Reference

Keep these pages open while you code: the rules on one sheet, the words in one list, and the exercises to make it stick.

- Cheat sheet: goroutines, channels, select p. 186
- Cheat sheet: sync, atomic, context p. 187
- Cheat sheet: the runtime at a glance p. 188
- Glossary p. 189
- Exercises by chapter p. 191
- References and further reading p. 192
- How this book was made p. 193

Cheat sheet: goroutines, channels and select

Everything about channel operations on one page. Chapters 8.3 to 8.8 explain each line.

operation	nil channel	open, would block	open, can proceed	closed
`ch <- v`	blocks forever	blocks until a receiver or a free slot	sends	panic
`<-ch`	blocks forever	blocks until a value arrives	receives	buffered values first, then zero value, ok = false
`close(ch)`	panic	closes; wakes all receivers	closes	panic
`len`, `cap`	0, 0	values waiting, capacity	same	values still buffered, capacity

select

Waits on several operations; runs **one** ready case, chosen **at random**. `default` makes it non-blocking. A nil channel's case is never ready. `select {}` blocks forever. Timeouts: `case <- time.After(d)` or a ctx.

Ownership rules

The **sender** closes, and only one goroutine does it. Receivers never close. Multiple senders: close from a separate goroutine after `wg.Wait()`. `for v := range ch` ends when ch is closed and drained.

Goroutines

`go f(x)` evaluates f and x now, runs f later. main returning ends the program. Wait with `WaitGroup`/`wg.Go`, errgroup, or a channel. Since Go 1.22 each loop iteration has its own variable.

idiom	code
signal / broadcast	`done := make(chan struct{})` … `close(done)`
result with a timeout	`ch := make(chan T, 1)`: the late send never leaks
semaphore of n	`sem := make(chan struct{}, n)`; `sem <- struct{}{}`; `<-sem`
disable a select case	set the channel variable to `nil`

Cheat sheet: sync, atomic and context

All zero values below are ready to use; none of them may be copied after first use. Chapters 8.4, 8.12 and 8.14.

type	use it for	key methods	remember
`sync.Mutex`	exclusive access to state	`Lock`, `Unlock`, `TryLock`	not re-entrant; `defer Unlock`
`sync.RWMutex`	read-mostly state	`RLock`, `RUnlock`, `Lock`	a waiting writer blocks new readers
`sync.WaitGroup`	wait for N goroutines	`Go` (1.25), `Add`, `Done`, `Wait`	Add before `go`; never negative
`sync.Once`	one-time init	`Do`, `OnceFunc`, `OnceValue(s)`	callers wait for f to finish
`sync.Cond`	wait for a condition	`Wait`, `Signal`, `Broadcast`	always wait in a `for` loop
`sync.Pool`	reuse temporary objects	`Get`, `Put`, `New`	emptied by GC; reset objects
`sync.Map`	write-once or disjoint keys	`Load`, `Store`, `LoadOrStore`	usually: map + Mutex
`atomic.Int64` etc.	one counter, flag or pointer	`Load`, `Store`, `Add`, `CompareAndSwap`	sequentially consistent
`errgroup.Group`	tasks that can fail	`Go`, `Wait`, `SetLimit`, `WithContext`	first error cancels ctx
`context.Context`	cancel, deadline, values	`Done`, `Err`, `Deadline`, `Value`	first parameter; `defer cancel()`

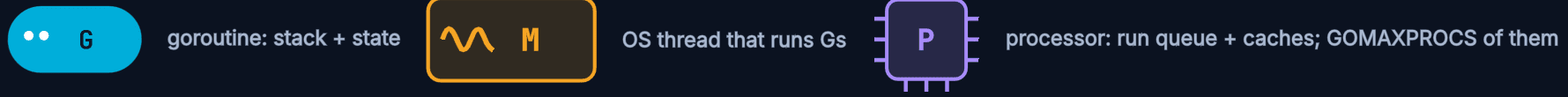
KEY IDEA Pick by the job: hand off data → channel; protect state → Mutex; one word → atomic; wait for a group → WaitGroup/errgroup; stop work → context.

Cheat sheet: the runtime at a glance

The numbers and knobs from chapters 8.3 and 8.9 to 8.11, all checked against the Go 1.27.3 runtime source.

what	value
initial goroutine stack	2 KB (`stackMin`), grows by copying
maximum stack	1 GB on 64-bit (250 MB on 32-bit)
local run queue per P	256 Gs + a `runnext` slot
global queue check	every 61st schedule on a P
work stealing	take half of a victim P's queue; 4 rounds over all Ps
sysmon sleep	20 µs, backing off to 10 ms when idle
preemption	a G running for more than 10 ms; SIGURG for async
blocking syscall hand-off	P retaken after at least one sysmon tick (20 µs)
default GOMAXPROCS	usable CPUs, also capped by the cgroup CPU limit
max OS threads	10,000 (`debug.SetMaxThreads`)

GODEBUG / env	effect
`GOMAXPROCS=n`	number of Ps
`schedtrace=1000`	print scheduler state every second
`scheddetail=1`	with schedtrace: every G, M and P
`asyncpreemptoff=1`	disable signal-based preemption
`containermaxprocs=0`	ignore the cgroup CPU limit
`updatemaxprocs=0`	do not adjust GOMAXPROCS at run time
`GOTRACEBACK=all`	dump every goroutine on a crash
`GORACE=halt_on_error=1`	stop at the first data race



Glossary (A to H)

The words of the module, with the chapter that explains each one.

Amdahl's law the serial fraction of a program limits its speed-up on any number of cores (8.1)

atomic operation a read-modify-write done by the hardware as one indivisible step (8.12)

async preemption stopping a goroutine anywhere by signalling its thread with SIGURG (8.11)

backpressure a slow consumer slowing the producer, e.g. through a full channel buffer (8.6)

bubble synctest's isolated group of goroutines with a fake clock (8.18)

buffered channel a channel with a queue of capacity > 0; sends block only when it is full (8.6)

CAS compare-and-swap: write new only if the value is still old (8.12)

cancellation telling goroutines to stop early, in Go through a context's Done channel (8.14)

concurrency structuring a program as independently executing tasks (8.1)

context a value carrying cancellation, a deadline and request-scoped values (8.14)

critical section code that must run in at most one goroutine at a time (8.12)

data race two unordered accesses to one location, at least one a non-atomic write (8.13)

deadlock goroutines waiting for each other in a cycle; nobody can proceed (8.17)

durably blocked blocked on something only another goroutine in the same bubble can release (8.18)

fan-in / fan-out merging many channels into one / spreading one stream over many goroutines (8.15)

G, M, P goroutine, OS thread (machine), logical processor: the scheduler's three structs (8.9)

generator a function returning a channel that a goroutine fills (8.15)

global run queue the scheduler's shared queue, checked every 61 schedules (8.10)

goroutine a function running concurrently, scheduled by the Go runtime (8.3)

goroutine leak a goroutine blocked forever; it and its memory are never freed (8.17)

happens before the order Go guarantees: sequenced-before plus synchronized-before (8.13)

hchan the runtime struct behind every channel: buffer, indexes, wait queues, lock (8.7)

Glossary (M to W)

The words of the module, with the chapter that explains each one.

memory model the rules for when a write in one goroutine is visible to a read in another (8.13)

mutex a lock that admits one goroutine at a time (8.12)

netpoller the runtime's epoll/kqueue loop that parks goroutines waiting on sockets (8.10)

nil channel a channel variable never made; operations block forever, close panics (8.6, 8.8)

parallelism executing several computations at the same instant on several cores (8.1)

park / ready taking a goroutine off the CPU to wait / putting it back on a run queue (8.7, 8.9)

pipeline stages connected by channels, each running concurrently (8.15)

preemption the runtime taking the CPU back from a running goroutine (8.11)

race detector ThreadSanitizer-based tool enabled with -race (8.13)

runnext a P's one-slot fast lane for the goroutine it just readied (8.10)

select a statement that waits on several channel operations at once (8.8)

semaphore a counter limiting how many goroutines do something at once (8.15)

sequential consistency all operations appear in one global order that respects each goroutine's order (8.13)

spinning thread an M actively looking for work before going to sleep (8.10)

starvation mode the Mutex mode that hands the lock straight to the oldest waiter (8.12)

sudog a waiting goroutine's entry in a channel or semaphore queue (8.7)

sysmon a runtime thread without a P: preemption, netpoll, retaking Ps (8.10, 8.11)

token bucket a rate limiter: tokens refill at a rate, requests spend them (8.16)

unbuffered channel a channel of capacity 0: every send meets a receive (8.5)

WaitGroup a counter that lets one goroutine wait for others to finish (8.4)

work stealing an idle P taking half of another P's run queue (8.10)

worker pool a fixed number of goroutines pulling jobs from one channel (8.15)

Exercises by chapter

★ check your understanding · ★★ apply it · ★★★ dig into the runtime. Every exercise starts from an example in examples/.

8.1	★	Run ch01_speedup with GOMAXPROCS 1 to 8 on your machine. Fit Amdahl's law: what serial fraction p explains your curve?
8.2	★	With ch02_goroutine_cost, estimate how many idle goroutines fit in 1 GB. How many threads would?
8.3	★★	Write a recursive function that grows a goroutine's stack to 1 MB. Measure StackInuse before and after.
8.4	★	Rewrite a WaitGroup program with wg.Go. Then make one task fail and switch to errgroup.
8.5	★	Pass a token around a ring of 5 goroutines 1,000 times with unbuffered channels. Time one hop.
8.6	★★	Three senders, one channel, one receiver ranging over it. Close it safely without a panic.
8.7	★★★	In runtime/chan.go find where a receiver copies straight from a waiting sender's stack. Why is it safe?
8.8	★★	Merge two channels with select; set each to nil when it closes; stop when both are nil.
8.9	★	Run a CPU-bound program with GODEBUG=schedtrace=500 and explain every field of one line.
8.10	★★	Start 100 goroutines doing a blocking syscall. Count OS threads, then repeat with network waits.
8.11	★	Reproduce the pre-1.14 hang with asyncpreemptoff=1. Add one function call to the loop. What changes?
8.12	★★	Fix the racy counter with a Mutex, an atomic and a channel. Benchmark all three with RunParallel.
8.13	★★★	Change the litmus test so only one goroutine uses atomics. Can 0, 0 still happen? Explain.
8.14	★★	Thread a ctx through three layers, cancel with a cause at the top, and log Cause at the bottom.
8.15	★★	Build gen → square → print with cancellation, then fan out square to 4 workers keeping order.
8.16	★★	Add graceful shutdown with signal.NotifyContext and http.Server.Shutdown to a small HTTP server.
8.17	★★	Write an ABBA deadlock, read its dump, fix it with a lock order. Then leak a goroutine and find it with goroutineleak.
8.18	★★	Turn a time.Sleep-based test for a 1-minute cache expiry into a synctest test that runs in 0 s.
8.19	★★★	Do the six crawler exercises; keep the synctest + -race test green after each.

References and further reading

Primary sources first. Every runtime number in this module was checked against the Go 1.27.3 source code.

Specifications and source

- The Go Programming Language Specification: go statements, channel types, select (go.dev/ref/spec)
- The Go Memory Model, version of June 6, 2022 (go.dev/ref/mem)
- Go 1.27.3 source: runtime/proc.go, runtime2.go, chan.go, select.go, sema.go, preempt.go, signal_unix.go, mgc.go; internal/sync/mutex.go; sync; context; testing/synctest

Design documents and proposals

- Dmitry Vyukov, "Scalable Go Scheduler Design Doc", 2012
- Austin Clements, "Proposal: Non-cooperative goroutine preemption", golang/go#24543, 2018
- Russ Cox, "Hardware Memory Models", "Programming Language Memory Models", "Updating the Go Memory Model", 2021

Go blog and talks

- Andrew Gerrand, "Share Memory By Communicating", 2010 · Rob Pike, "Go Concurrency Patterns", Google I/O 2012
- Rob Pike, "Concurrency is not Parallelism", 2012 · Sameer Ajmani, "Advanced Go Concurrency Patterns", 2013
- Dmitry Vyukov and Andrew Gerrand, "Introducing the Go Race Detector", 2013
- Sameer Ajmani, "Go Concurrency Patterns: Context" and "Pipelines and cancellation", 2014
- Damien Neil, "Testing concurrent code with testing/synctest", 2025

Papers and books

- C. A. R. Hoare, "Communicating Sequential Processes", CACM 21(8), 1978
- Leslie Lamport, "How to Make a Multiprocessor Computer That Correctly Executes Multiprocess Programs", 1979
- Hans-J. Boehm and Sarita V. Adve, "Foundations of the C++ Concurrency Memory Model", PLDI 2008
- Konstantin Serebryany and Timur Iskhodzhanov, "ThreadSanitizer: data race detection in practice", 2009
- Marc Brooker, "Exponential Backoff And Jitter", AWS Architecture Blog, 2015
- Alan Donovan and Brian Kernighan, The Go Programming Language, chapters 8-9, 2015 · Katherine Cox-Buday, Concurrency in Go, 2017

How this book was made

Every page is a ManimGL scene; every listing is a real program; every output was captured, not typed.

1. **Write the programs.** Each listing is a file in `examples/`. `tools/run_examples.py` runs them all with Go 1.27.3 and saves the real output, including the races and the deadlocks.
2. **Measure on real hardware.** Timings come from a 4-vCPU Intel Xeon 2.1 GHz Linux VM. Pages read the numbers from the saved output, so the text always matches the run.
3. **Draw with ManimGL.** Each page is a `Scene` built from a small kit of shapes: gophers, G/M/P tokens, channel tubes, lanes and code panels. One theme file sets every colour.
4. **Render and check.** `tools/build.py all` renders every page at 2560×1440, warns about anything that overflows, then builds the PDF and the HTML viewer.

rebuild everything

```
python3 tools/run_examples.py   # run every example
python3 tools/build.py all      # render, PDF, viewer
# pages/*.png  Module-08-Concurrency.pdf  index.html
```

NOTE

Your numbers will differ: other CPUs, other loads, other Go versions. That is the point of the "measured" pages: run them yourself and compare.

Happy concurrent coding.